Intrusion Detection and Monitoring System

Complete system documentation: architecture, flow, Python services, website, APIs, database, model training, operations, and source inventory.

Prepared 29 October 2026. Local-source reference; deployed state requires separate verification. Text only; no private credentials.

1. Document scope, evidence, and reading guide 2

2. Purpose, actors, and component boundaries 2

3. End-to-end system flow and timing 2

4. NetFlow collector and subscriber discovery 3

5. Traffic features and measurement semantics 4

6. Detector API, persistence, and incident assignment 4

7. Blocker, restorer, and router reconciliation 5

8. Website pages, user interface, and browser behavior 6

9. Authentication, authorization, and account lifecycle 7

10. Endpoint register and request responsibilities 8

11. Database architecture, ownership, and relationships 8

12. Migration history and data transformation rules 9

13. Machine-learning training, artifacts, and recorded results 10

14. Configuration, dependencies, and file responsibilities 11

15. Installation, startup, hosting, and change deployment 11

16. Tests, verification boundaries, and acceptance scenarios 12

17. Operational limitations and accurate interpretation 13

18. Maintenance, troubleshooting, and glossary 13

19. Source member register: Python services and training 14

20. Source member register: PHP controllers, models, and tests 17

21. Exact schema declaration register 18

22. Environment variable register without private values 22

23. Notebook and archived API evidence register 26

24. Historical research manuscripts: text extracts 27

25. Complete project-file inventory and uses 53

26. Traceability, exclusions, and documentation completion 73

1. Document scope, evidence, and reading guide

Intrusion Detection and Monitoring System: comprehensive implementation and operations reference. Prepared on 29 October 2026, using Asia/Manila as the document date context. This document describes the local project snapshot in the Intrusion-Detection-Monitoring-System repository. It is written entirely as searchable text, without screenshots, diagrams, or source-code listings. It includes detailed implementation explanations, an endpoint register, a database dictionary, operational procedures, a source member register, research material extracts, and a file inventory.

The primary evidence is executable application source: Python services under setup, Laravel controllers, middleware, models, routes, migrations, views, configuration, tests, dependency manifests, and the API ZIP archive. Training manuscripts and notebooks are supplementary historical evidence. When a manuscript, comment, or older artifact differs from active source, this reference identifies the difference rather than combining both as if they were one implementation. The archived Render API is a separate historical path, not proof of the deployment currently running on Render.

The local source can explain intended behavior, defaults, and implementation limitations. It cannot independently establish which revision is deployed on Hostinger, whether Python services are currently running, the contents of the production database, current router firewall state, actual production accuracy, or subscriber connectivity. The user's hosting screenshots establish that the role migration completed; they do not verify every deployed component. No live router operations, production schema changes, traffic generation, model retraining, or production database queries are performed to produce this document.

Secrets are deliberately excluded. Environment variable names are documented, but values from private environment files, passwords, application encryption keys, authentication tokens, connection URLs containing credentials, and runtime session contents are not included. Binary models are inventoried without deserializing them. Dependency and generated environment internals are described by category rather than reproduced as application source. Source filenames, method names, schema names, and endpoint paths remain explicit because they are necessary for maintenance.

Read sections two through seven for architecture and runtime flow; sections eight through thirteen for website, API, database, and training; sections fourteen through eighteen for deployment, testing, limitations, and maintenance. The remaining registers provide traceability to individual source members, migration definitions, notebook contents, historical research documents, and project files. The accompanying plain-text document contains the same narrative and is provided for editing and searching.

2. Purpose, actors, and component boundaries

The project combines network intrusion detection with human-directed subscriber security management. It collects MikroTik NetFlow export records, associates observed addresses with PPPoE subscriber names, computes numerical traffic features, sends those features to a two-stage classifier, stores classifications in MySQL, and presents the results through a Laravel website. An administrator can request blocking or restoration; a local router-facing reconciliation agent implements those requests. Detection alone does not automatically block a subscriber in the current main path.

The monitored subscriber generates network traffic through the MikroTik gateway. The gateway exports NetFlow records and exposes active PPP sessions and firewall management through the RouterOS API. The local monitoring computer receives those records, performs feature aggregation, runs or contacts the detector, and reaches the database and router. Hostinger hosts the Laravel website and the shared MySQL database. A browser displays database-derived dashboards and submits administrator actions. The database is the coordination point between detection, presentation, and enforcement.

The administrator has read access plus threat management permissions. The viewer account has read access to protected pages and read endpoints, including report export, but cannot submit threat changes. The detector is a machine client of the model service and a database writer. The blocker/restorer is a database reader and router controller. These machine processes do not authenticate as the website viewer; website session restrictions do not automatically secure their separate network interfaces.

The principal local scripts are setup/Mikrotek PPPoe Sniffer.py, setup/Detection Process.py, and setup/Pending Blocker.py. The historical spelling Mikrotek is part of the filename. The first script is a NetFlow collector rather than a raw packet sniffer. The second hosts a Flask inference API and writes classifications directly to MySQL. The third performs both blocking and restoration. No separate current Restorer.py service is required or present in the inspected main setup.

Laravel follows a route, middleware, controller, model, and Blade view structure. Routes select controllers; CheckSession validates the custom session and role; controllers query or update Eloquent models; Blade templates render HTML; browser JavaScript polls selected endpoints. Database migrations define persistent storage. Composer manages PHP dependencies; npm and Vite manage the source asset build; most visible administration pages also load prebuilt static template assets directly from public/admin.

3. End-to-end system flow and timing

Step one: configure MikroTik to export NetFlow version five records to the monitoring computer on UDP port 2055 and make RouterOS API access available to that computer. The repository contains router inspection and removal notes but does not automatically configure Traffic Flow export. The collector binds an IPv4 UDP socket on all local interfaces and accepts packets only when the sender address equals the configured MikroTik host.

Step two: query active PPP sessions through the RouterOS API. The collector builds a mapping from each current subscriber address to its stable PPPoE username, filters excluded address prefixes, and adds recognized addresses to a round-robin queue. A current address is an observation, not the long-term subscriber identity. The queue is refreshed using active session data, and addresses absent from a successful refreshed mapping are removed from queue state.

Step three: parse each supported NetFlow packet. Version five uses a 24-byte header followed by 48-byte records. Extract source and destination IPv4 addresses, source and destination ports, packet count, byte count, first and last flow timestamps, TCP flag bits, and protocol number. Record a local collection timestamp. Store each accepted flow in both source-address and destination-address indexes.

Step four: approximately every 60 seconds, select one queued subscriber and gather records where that subscriber appears in either direction. Require at least ten indexed flow records before analysis. Retain at most the most recent 200 outgoing records and 200 incoming records for feature calculation. Refresh subscriber mappings approximately every 300 seconds and remove stored flow records older than one hour during the periodic analysis branch.

The timing checks are inside the successful UDP receive path. A socket timeout immediately continues the loop. Consequently, these are traffic-driven checks, not independent scheduled timers: no incoming export traffic can delay subscriber refresh, analysis, and memory cleanup. A queue with N active clients normally revisits one client in roughly N times 60 seconds, plus processing and traffic delays; it does not analyze every subscriber every 60 seconds.

Step five: submit one JSON request to the configured inference URL, with a 30-second request timeout. The current detector validates presence of all required model features, coerces feature values to finite numeric values, computes Stage A attack probability, applies the configured threshold, and invokes Stage B only when Stage A predicts attack. The response includes the decision, probability, optional attack family, used threshold, and feature accounting information.

Step six: append the classification record to a local JSON-lines prediction log and attempt a database transaction. A benign result creates a detection row without an incident. An attack result either updates an eligible incident or creates a new unresolved incident, then inserts the detection referencing that incident. The website reads the same database and updates selected displays through polling or page navigation.

Step seven: an administrator selects Block. The website updates the chosen incident to blocked, records the request timestamp, and clears restoration and synchronization markers. The browser response confirms the database request, not router completion. Approximately every 20 seconds, the local agent reads all blocked PPPoE subscribers, resolves each current address, and maintains subscriber-owned forward-chain drop rules.

Step eight: an administrator selects Restore. The controller method is historically named ignore, but the operation sets the incident to resolved and records restoration time. The local agent only processes a pending restore when there is no other blocked incident for the same subscriber. It removes subscriber-owned rules, clears tracked connections for relevant addresses, and marks the resolved incident synchronized. The incident and its logs remain as historical evidence.

The website dashboard polls every 15 seconds while the document is visible. The system-status alert fragment also polls every 15 seconds. The live-monitor demonstration updates synthetic chart values every two seconds and timestamps every five seconds; those intervals do not measure the real collector or router. Overall response time combines export timing, queue position, inference time, database availability, browser polling, and enforcement polling.

4. NetFlow collector and subscriber discovery

Source: setup/Mikrotek PPPoe Sniffer.py. MikroTikConnector holds a RouterOS connection pool, API handle, and connection state. connect creates a pool using the configured router host, API port, username, and password. The current code selects no SSL and plaintext login. A failed connection is logged and reported as false. get_pppoe_clients reconnects when necessary, reads /ppp/active, extracts address and name, discards incomplete entries, and returns the accepted address-to-name mapping.

The is_client_ip helper rejects empty addresses, zero and broadcast endpoints, and a literal list of infrastructure-like string prefixes. The list includes the 100.64 through 100.127 carrier-grade address prefixes, selected 190.168 through 193.168 prefixes, loopback, link-local, and several multicast or reserved prefixes. It uses textual starts-with matching rather than a formal subnet library. It should not be described as a complete standards-based IPv4 classification policy. The separate is_infrastructure_ip helper also excludes addresses beginning with 192.168 and selected multicast or reserved prefixes.

NetFlowCollector creates the UDP socket, sets a one-second receive timeout, initializes per-source and per-destination flow lists, initializes the round-robin queue and known-client set, and maintains the current PPPoE mapping. parse_netflow_v5 checks minimum packet length and stops when an advertised record would exceed the available bytes. The main loop checks the NetFlow version before calling it. Versions other than five are ignored; template-based NetFlow version nine and IPFIX are not implemented by this parser.

Flow duration is the exported last timestamp minus the first timestamp in milliseconds. Negative or wrapped exporter counters are not explicitly repaired. The parser excludes a record if either endpoint is considered infrastructure by its filter. This may exclude traffic that an operator expects to monitor, particularly private-address traffic through a gateway. Every surviving record receives a local ISO-format timestamp; the collector does not store packet payloads or reconstruct application content.

update_client_list replaces the address-to-name mapping, prunes addresses absent from the new active set, and queues newly recognized addresses. get_next_client rotates the deque before returning its first element. store_flow indexes one flow under both endpoints, and get_client_flows concatenates both indexes for the selected address. The indexes intentionally support bidirectional observation, but they are not a strict five-tuple conversation reconstruction. Self-addressed records can appear in both indexes.

In the receive loop, subscriber discovery is refreshed only if the router query returns a nonempty mapping. An empty mapping or failed query therefore leaves the prior mapping in place. This is a relevant limitation when every PPP session disconnects or API access fails. The receive buffer is 8192 bytes. Export datagrams larger than that buffer can be truncated, while the parser processes only records that fit in the received buffer.

analyze_next_client consumes the next queue position even when it skips a subscriber with too few records. It builds a payload, logs traffic summaries, and posts it once. There is no persistent outbound queue, request retry loop, duplicate identifier, or delivery acknowledgement tied to a committed database row. Data retained in memory may be re-aggregated on a later turn, so successive classifications are overlapping observations, not necessarily disjoint network sessions.

The collector writes setup/logs/merged_ids.log and console output. MergedIDS obtains an initial PPP client mapping before entering the receive loop. The Windows launcher opens each component in a separate console. Exceptions are printed with a pause-on-error hook where applicable. Stopping the console or losing the monitoring computer stops collection; the hosted website can remain available while its data becomes stale.

5. Traffic features and measurement semantics

The current training contract contains fourteen ordered features under schema version mikrotik-netflow-v1. Feature order is taken from the loaded model bundle during inference, rather than inferred from JSON key order. Source and destination addresses, subscriber identity, ports, protocol, timestamps, TCP-flag counters, and several extra statistics are also transmitted for context, but not all are model inputs or persisted database columns.

destination_port is the latest context record's destination port. flow_duration is the sum of selected flow durations in milliseconds multiplied by one thousand, producing a microsecond representation. It is a sum of exporter durations across selected records, not the wall-clock span between first and last client activity. Concurrent flows can therefore contribute overlapping time.

total_fwd_packets and total_backward_packets are packet totals for outgoing and incoming selected records. total_length_of_fwd_packets and total_length_of_bwd_packets are their byte totals. fwd_packet_length_mean and bwd_packet_length_mean are packet-weighted means derived from each export record's bytes divided by packets. These represent aggregate estimates; NetFlow records do not supply each original packet length.

flow_bytes_s and flow_packets_s divide aggregate bytes and packets by the summed duration converted to seconds. fwd_packets_s and bwd_packets_s use the same denominator for each directional packet count. If aggregate duration is not positive, the code uses one second as a fallback denominator. down_up_ratio is incoming packets divided by the greater of outgoing packets and one. average_packet_size is aggregate bytes divided by the greater of aggregate packets and one.

The collector includes both compact database-friendly keys and normalized training keys. tot_fwd_pkts corresponds to total_fwd_packets; tot_bwd_pkts corresponds to total_backward_packets; tot_fwd_bytes corresponds to total_length_of_fwd_packets; tot_bwd_bytes corresponds to total_length_of_bwd_packets; fwd_pkt_len_mean and bwd_pkt_len_mean correspond to their expanded packet-length feature names; flow_pkts_s corresponds to flow_packets_s; dst_port corresponds to destination_port.

Additional transmitted fwd_iat_mean and bwd_iat_mean are computed by dividing the summed duration in milliseconds by the corresponding directional packet count with a minimum denominator of one. They are not measured packet-by-packet inter-arrival intervals, and they are not part of the fourteen-feature current training contract. syn_count, ack_count, fin_count, and rst_count count selected export records carrying each flag bit, rather than reconstructing the exact number of packets with that flag.

The context record prefers the latest outgoing record and falls back to the latest incoming one. src_ip is always set to the subscriber being analyzed, even when the fallback context record originally had that subscriber as its destination. Thus destination address and port metadata may not describe a canonical subscriber-to-server conversation in the incoming-only case. A classification represents a subscriber aggregate across many records and destinations, not proof that one displayed destination caused the prediction.

Packet-length means are calculated by expanding repeated mean values into lists once per packet. Large exported packet counts can consume substantial memory even though only a few hundred flow records are selected. A weighted-sum calculation would avoid this implementation cost, but this reference records the present behavior without changing it. Training data uses CICFlowMeter-derived columns, while deployment computes aggregate NetFlow approximations; matching column names and units does not alone prove identical statistical distributions.

6. Detector API, persistence, and incident assignment

Source: setup/Detection Process.py. The script loads environment settings from setup/.env, chooses a model filename under setup/data/model, and deserializes a trusted joblib bundle at startup. The default filename is netflow_ids.joblib. Required bundle entries are bin for the binary model, mul for attack-family classification, features for ordered feature names, and best_threshold. An optional schema_version is exposed to clients; missing schema metadata becomes unknown.

IDS_THRESHOLD overrides the bundle threshold and must be within zero and one inclusive. The code trims expected feature names and incoming JSON keys. It maps selected compact sender names into normalized model names when the normalized field is absent. A supplied normalized field takes precedence over its alias. Missing required feature names produce a 422 response containing error missing_model_features, schema_version, missing_expected_features, present_expected_features, and expected_features_count.

For each required feature, conversion to float is attempted. Conversion failures, NaN, and infinity become zero. Therefore feature presence validation is stricter than value validation: a present but malformed value may silently become zero. The API expects an object-like JSON body; request JSON is forced, and there is no dedicated input schema enforcing valid ranges, record size, subscriber identity, or approved source hosts. The source does not configure API authentication or rate limiting.

Stage A calls predict_proba and takes the second probability column as the attack probability. A probability at or above the selected threshold yields risk_level attack; lower values yield benign. Stage B predicts one attack-family string only for an attack. Benign attack_type remains null. The response returns risk_level, prob_attack, attack_type, used_threshold,

present_expected_features, missing_expected_features, received_features_count, api_version debug-v3, and schema_version.

The detector constructs a record containing an epoch-second timestamp, the received normalized data, and prediction results. append_log creates the configured log directory and appends a JSON record to predictions.jsonl. Log failures are printed and do not stop database saving. save_to_database constructs a selected column payload, obtains a MySQL connection with autocommit disabled, assigns an incident if appropriate, inserts the detection, and commits. On transactional failure it rolls back and prints an error; the outer inference handler can still return a successful classification response.

A successful inference response is consequently not a durable-storage guarantee. Missing database name or username explicitly skips insertion. Database exceptions are handled internally rather than converted into an HTTP failure. Neither a local prediction-log entry nor a successful collector response proves that Laravel can read a newly committed row. Operators should verify database timestamps or the logs page independently when diagnosing missing observations.

assign_incident returns no incident for non-attack records. For an attack with a PPPoE name, it searches for the same subscriber name and attack type. A blocked incident is eligible without the unresolved time-window restriction and is preferred over unresolved incidents. Otherwise an unresolved incident is eligible only if its last detection falls within INCIDENT_WINDOW_MINUTES, which defaults to fifteen and is clamped to at least one. The selected row is locked for update within the transaction.

An eligible incident increments event_count and updates last_detected_at and updated_at. Otherwise a new incident starts with status unresolved, event_count one, and first/last detection timestamps. Resolved incidents are never reopened by this query. An attack without a PPPoE name creates a new incident rather than reusing an unidentified address group. The new detection is linked through incident_id. The schema does not enforce a uniqueness constraint covering eligible incident identity; row locking does not fully prevent concurrent first-insert races when no matching incident exists.

The script starts Flask on all interfaces, port 5000, with debug enabled when executed directly. This is an implementation fact, not a hardened production deployment recommendation. The API root returns a simple running message; it does not exercise the model prediction path, database transaction, collector, or router. Loading an artifact can fail before the server starts. The bundle is not inspected dynamically by this documentation process because joblib deserialization executes trusted Python object loading behavior.

7. Blocker, restorer, and router reconciliation

Source: setup/Pending Blocker.py. The process loads shared settings, creates setup/logs/blocked_threat_sync.log, connects to RouterOS, and repeatedly polls MySQL. BLOCK_SCAN_INTERVAL defaults to twenty seconds. BLOCK_RULE_PREFIX defaults to IDS block. DB_TABLE defaults to intrusion_logs and DB_INCIDENT_TABLE defaults to threat_incidents. This service must run on a host that can reach both the database and the MikroTik API.

get_blocked_subscribers selects distinct nonempty PPPoE names with at least one blocked incident. It intentionally continues enforcing a blocked subscriber on every scan rather than treating a block as a one-time command. get_pending_restores selects resolved incidents whose firewall_synced_at is null, whose PPPoE name is nonempty, and for which no blocked incident exists for that same name. This prevents restoring one historical incident from overriding another currently blocked incident for the subscriber.

The main loop processes all blocked subscribers before pending restores, isolates individual subscriber failures with logging, catches scan-level errors, and sleeps before the next iteration. Initial ensure_connected occurs before the main loop; a startup router connection failure can terminate startup. Inside the loop, ensure_connected primarily checks whether an API object exists rather than proving an existing session is healthy. There is no general reconnect/backoff supervisor implemented around every transport failure.

Stable rule ownership is encoded as a comment consisting of the configured prefix followed by pppoe and the subscriber name, with a colon between pppoe and the name. The comment is truncated to 255 characters. Matching rules are sought in the forward chain with action drop and this exact ownership comment. These are source-address drop rules, not PPP account deletion, password changes, or a complete bidirectional quarantine policy. Existing firewall rule order and FastTrack behavior remain relevant to effective enforcement.

resolve_current_ip_for_pppoe queries /ppp/active using the supplied name. If no exact response is returned, it reads active sessions and compares trimmed lowercased names. Missing active sessions or missing assigned addresses raise a runtime error rather than falling back to an old detection address. This avoids intentionally using an obsolete observed address to block a different subscriber after dynamic reassignment.

block_subscriber first attempts legacy IP-owned rule cleanup, then resolves the current PPP address. It inspects subscriber-owned rules, removes those targeting obsolete source addresses, and distinguishes enabled and disabled rules for the current address. An enabled current rule is retained. Disabled current rules are re-enabled. If none exists, it creates a forward-chain source-address drop rule. It then clears matching tracked connections. The implementation does not move a newly added rule to a guaranteed position ahead of preexisting accepts or FastTrack rules.

_known_ips_for_pppoe reads distinct historical src_ip values for the name from intrusion_logs. _remove_legacy_subscriber_rules uses these addresses to identify older comments containing the prefix and address. This migrates from IP ownership to subscriber ownership. Historical address reuse means an address-only legacy rule may be ambiguous across subscribers. Cleanup therefore has historical assumptions that should be understood before treating ownership as perfect.

unblock_subscriber removes legacy rules, retrieves all matching subscriber-owned rules, collects their source addresses, and attempts deletion of each identifiable rule. It also resolves the subscriber's current active address when possible, but an offline subscriber does not prevent removal of already known owned rules. It clears tracked connections for collected addresses. The

ordinary current restoration path does not require SSH, Paramiko, a PPP session disconnect, or a separate restorer process.

mark_restore_synced writes the current database time into firewall_synced_at only for the selected incident while it remains resolved. However, the current unblock_subscriber path does not assert that all deletion helpers succeeded or re-query owned rules before this marker is written. _remove_firewall_rule returns false after exhausting its fallbacks, and callers do not consistently propagate that result. Accordingly, the synchronization timestamp records that the agent reached that step; it is not an independently verified guarantee that every firewall rule disappeared.

The service retains legacy block_ip and unblock_ip helpers plus direct RouterOS protocol support. RouterOsDirectClient implements variable-length word framing, sentences, socket reads, ordinary login, challenge-response fallback using MD5, and command dispatch with trap detection. _set_firewall_rule_enabled tries several identifier forms through direct API and library fallbacks. _remove_firewall_rule tries identifiers with and without the RouterOS asterisk, different parameter names, direct API removal, and a final command-style fallback. These compatibility methods support different RouterOS API representations.

clear_connections queries source and destination tracked connections and attempts to remove each unique identifier. clear_ppp_active can disconnect PPP sessions by address, but is not invoked by the current subscriber reconciliation flow. Old variable names such as dst_ip in legacy blocking methods refer to the address being blocked even though the created filter matches source address. Function names should not be used to infer destination-based blocking policy.

The browser may display blocked or resolved before the router reaches that state. Blocking is continuously reconciled, while restoration is treated as pending until the synchronization marker is set. If the agent is stopped, database statuses still change but router enforcement does not follow. If another incident for the same subscriber remains blocked, restoring one incident leaves that subscriber blocked by design. Operators should inspect both incident state and the actual owned router rules when verifying outcomes.

8. Website pages, user interface, and browser behavior

resources/views/welcome.blade.php is the public landing page and contains the login form. It includes project branding, descriptive website content, theme handling, smooth scrolling to login, and a no-account modal used by the /login fallback when a dedicated auth.login template is absent. The root route is named logout even though it only returns the welcome page. Actual logout is /logout with the name logout.action; maintainers must distinguish those names.

resources/views/admin/dashboard.blade.php renders summary counts, attack and benign series, attack-type breakdowns, destination rankings, and risk distribution from DashboardController. Time windows are all, 24h, 7d, and 30d. Attack rate is attack rows divided by all selected rows, multiplied by one hundred. The controller deliberately avoids loading a full raw log collection on the dashboard. Most metrics use intrusion_logs with a null legacy status.

DashboardController selects the earliest available timestamp for the all-time window, falling back to five years before the current time when empty. The 24-hour chart uses MySQL DATE_FORMAT to group hourly data; daily windows use DATE. The 24-hour display generates twenty-four buckets from the computed start. Seven-day and thirty-day charts generate fixed day counts. A final partial current bucket can be excluded from generated chart labels even when aggregate counters include it. All-time charts include only dates actually returned by the grouping query.

The dashboard refresh endpoint returns selectedWindowLabel, window, totalLogs, attackCount, benignCount, attackRate, chartData, attackByType, topAttackedIPs, and riskLevelDistribution. Browser code requests an initial refresh after DOM readiness and polls every fifteen seconds while visible. It uses a cache-busting query value and a no-store fetch policy and prevents overlapping refreshes. There is no WebSocket broadcast or server-sent event implementation in this path.

resources/views/admin/logs.blade.php lists raw detections with ten rows per page. LogController supports date, type, and free-text query parameters. It inspects existing columns to choose an attack-type field and search fields, preferring is_malicious, label, type, classification, verdict, then attack_type. Search candidates include PPPoE name, addresses, protocol, notes, payload, info, and ports when present. Sorting is newest created_at first. Optional notes and subscriber columns are rendered only when found in the schema.

The logs type filter is legacy-compatible rather than a strict filter on risk_level. With attack_type selected, benign rows commonly have a null family, so comparing attack_type to benign or normal does not necessarily select the same records as risk_level benign. The benign branch adds an ungrouped OR condition, which can weaken earlier predicates. These are documented implementation caveats rather than corrected in this documentation task.

resources/views/admin/manage-threats.blade.php displays incidents, not an arbitrary grouping of source-IP rows. The controller eager-loads latestLog and adapts it into a display object containing incident_id, PPPoE identity, attack family, event count, group status, and last-detected timestamp. The default status filter is unresolved, with blocked and resolved alternatives. Date filtering applies to incident last_detected_at. Each page contains ten incidents, newest activity first.

The management page shows source address, PPPoE user, destination, protocol, attack type, event count, risk label, status, and optional notes. Protocol numbers are mapped into human-readable names, including TCP, UDP, ICMP, IPv6-related values, GRE, ESP, AH, OSPF, and SCTP. Missing values use a No Value presentation. Block and Restore confirmation modals explain that enforcement is performed by the local agent. Missing subscriber identity prevents these normal actions and results in a PPPoE-unavailable or legacy-block message.

Administrators can seed two configured demonstration subscriber incidents by supplying valid destination IPs. The controller creates one unresolved incident and one attack log for each target with predefined traffic statistics and attack probability 0.99. This does not generate network packets or exercise model inference. The example data is inserted into the same tables and is not globally excluded from dashboard counts. A real subscriber with the same demo identity could later be affected by an administrator's block action.

resources/views/admin/threat-reports.blade.php presents time-window statistics and a paginated detection table. ThreatReportsController computes selected total rows, attack rows, high-risk attack rows with probability greater than 0.8, and latest row while requiring null legacy log status. The data endpoint returns up to two hundred matching rows. The CSV endpoint streams rows using a cursor and includes timestamp, addresses, risk level, attack probability, forward packet count, and byte rate.

The CSV export is not a PDF export, and it is not identical to the paginated report query: its default window is 24h, while the report index defaults to all; export does not apply the null-status filter used by index and data. The JSON search includes notes without checking whether that optional column exists. Since the inspected migrations do not create notes, a nonempty report search can fail on a schema matching only these migrations. Formula-like values are not specifically escaped for spreadsheet interpretation in the CSV writer.

resources/views/admin/system-status.blade.php renders counts, processing estimates, health indicators, and a twenty-four-hour activity timeline. It refreshes the status alert fragment every fifteen seconds using /system-status/alert. The partial maps Healthy to success styling, Caution to warning, and Warning or Critical to danger. The page does not open a live RouterOS health connection, inspect the Python processes, or issue a detector probe; its indicators are derived from stored logs.

SystemStatusController counts current day, week, and month observations; computes last-hour and last-day rates; averages forward packet count and flow_bytes_s; and calculates a probability-based health label. Critical means more than one hundred probability-at-least-0.8 rows. Otherwise Warning means more than fifty such rows or an attack rate above thirty percent. Otherwise Caution means more than twenty such rows or attack rate above fifteen percent. Otherwise Healthy is used. These indicators are mostly all-time null-status metrics, even when a selected window constrains other page counts.

The status-page uptime value is based on timestamps and is not operating-system uptime or uninterrupted service availability. In selected-window mode, the current code can treat a Carbon start-window object like a log object when accessing created_at, leaving a null timestamp and producing a failure. This path has not been validated against the live deployment. Labels describing average bytes per flow actually use the stored byte-rate field. Probability categories are dashboard conventions separate from the model's calibrated classification threshold.

resources/views/admin/live-monitor.blade.php is a demonstration page. LiveMonitorController supplies placeholder status and a current timestamp. Browser code generates random initial values and random subsequent chart changes, updating flows, packet-rate-like values, alert-like values, and timestamps. These figures are not evidence of current network activity. They should be labeled or replaced before being used in an operational report as live measurements.

Shared visual structure is repeated across the administration templates rather than centralized in a single layout file. The templates use the static Sneat administration assets, Bootstrap styling, Boxicons, ApexCharts, and shared navigation links. The public landing folder also contains a separate HTML template with Bootstrap, animations, sliders, lightboxes, counters, and PHPMailer source. Their presence does not prove that all template demonstration pages or email handlers are active Laravel features.

9. Authentication, authorization, and account lifecycle

The actual website login uses App/Models/UsersAccount mapped explicitly to users_account. LoginController validates a syntactically valid email and a required string password, finds the first matching account, and verifies the stored password hash. A successful login regenerates the session identifier and stores user, user_id, and login_time, then redirects to the dashboard. Failure redirects to the login route with an invalid-credentials message. There is no registration route or account-management interface in the inspected route file.

UsersAccount permits fullname, age, sex, email, password, and role for mass assignment. Password is hidden from model serialization. Age is cast to an integer. The name accessor maps the expected display name to fullname, allowing templates to display it. canManageThreats returns true only when role is exactly admin. The role migration adds a string column with admin as its default so existing administrator accounts retain access.

CheckSession first requires a stored user object. For every protected request it reloads the corresponding UsersAccount row, invalidates the session when the account no longer exists, and replaces the stored object with the fresh row. Role changes therefore take effect without waiting for a new login. For unsafe HTTP methods it requires canManageThreats; otherwise it aborts with 403 and a view-only message. The route group currently includes all four threat mutation endpoints.

The management Blade template separately checks canManageThreats before rendering the Actions column, per-incident action cells and modals, the demo-seed button, and the demo-seed modal. This removes the controls from viewer HTML, while middleware rejects direct crafted POST requests. Hiding a button alone would not establish authorization; both checks are used. Viewer GET requests remain permitted, including viewing blocked and resolved records, filtering, JSON refresh, and report export.

ViewOnlyUserSeeder uses updateOrCreate for user@idms.com, assigns the viewer role, sets the display name to View Only User, and hashes the requested initial password. The password itself is intentionally absent from this documentation. Re-running the seeder resets that account's password and role. DatabaseSeeder calls it and also creates the framework Test User in the separate users table. Repeated generic DatabaseSeeder runs can conflict with the unique test email; use the dedicated viewer seeder for this task.

App/Models/User is the standard Laravel Authenticatable model using the users table, with notifications, factory support, datetime verification casting, and hashed password casting. config/auth.php still points its standard provider at this model. Custom UsersAccount session login does not automatically authenticate the standard guard or establish a Sanctum token. The protected /api/user route therefore has different authentication semantics from the website pages.

Logout flushes and invalidates the session and regenerates the CSRF token. The source exposes it as GET /logout. Protected forms include CSRF tokens through the web middleware stack. The source does not configure login throttling, multifactor login, email verification enforcement, a user-account password reset flow, or a record of which administrator requested each incident

transition. The account password column is not automatically hashed by UsersAccount itself; callers other than the seeder must explicitly hash new passwords.

Session configuration defaults to database storage and a ten-minute idle lifetime unless environment settings override it. Automatic browser polling may refresh request activity, depending on session handling and whether the page remains visible. The public API ingestion endpoint is outside CheckSession and outside the viewer restriction. A viewer role should therefore not be represented as a complete machine-API access-control policy.

10. Endpoint register and request responsibilities

Website GET / returns the welcome page and is named logout. GET /login returns auth.login when that view exists, otherwise the welcome page with the missing-account modal flag. POST /login invokes LoginController login. GET /logout invokes LoginController logout and is named logout.action. These routes are outside the custom protected group.

Protected GET /dashboard invokes DashboardController gotoDashboard and accepts window. GET /dashboard/refresh invokes refreshData and returns JSON aggregates for window. GET /logs invokes LogController gotoLogs and accepts date, type, q, and pagination. GET /system-status invokes SystemStatusController gotoStatus and accepts window. GET /system-status/alert returns an HTML fragment from alertFragment. GET /live renders the demonstration monitor.

Protected GET /threat-reports invokes index and accepts window and pagination. GET /threat-reports/export invokes export and returns a streamed CSV. GET /threat-reports/data invokes data, accepts window and q, and returns an object with rows. These read endpoints are available to viewers as currently implemented.

Protected GET /manage-threats invokes ManageThreatController index and accepts status, date, and pagination. POST /threats/demo invokes demo and requires valid dst_ip_jimson and dst_ip_jimson2 values. POST /threats/{id}/block invokes block for an existing incident. POST /threats/{id}/ignore invokes ignore for restoration. POST /threats/bulk-action invokes bulkAction with action and ids. All four POST operations require the admin role through CheckSession.

The block and restore methods return 404 when the incident does not exist and 422 when its PPPoE identity is blank. They update incident state and redirect back with feedback. They do not call RouterOS directly. bulkAction treats the literal block action as blocked and any other nonempty action as resolved; it does not validate an explicit action enumeration. It filters out null subscriber names but not every possible blank string, and its success count is the number of submitted IDs rather than the actual updated-row count.

Laravel API POST /api/intrusion-log invokes IntrusionLogController store. It passes the entire request array into IntrusionLog create, with Eloquent fillable fields determining persisted attributes, and responds with status saved. There is no explicit request validation, API-key verification, authentication middleware, incident assignment, or custom error response in this controller. Extra fields are not automatically preserved if absent from fillable. Records inserted through this endpoint can lack an incident link, so they may appear in log statistics without appearing in incident management.

Laravel API GET /api/user returns the standard authenticated request user and uses auth:sanctum. The API route prefix is registered through bootstrap/app.php. GET /up is the framework health route registered in that bootstrap file; no custom detector, router, or MySQL probe is defined there by the project. Console routes contain only the scaffold inspire command, rather than an IDS scheduler.

Current Flask GET / returns a running message. GET /api/features returns count and ordered features. POST /api/analyze accepts the numerical feature payload and context, returns the classification fields described earlier, and attempts direct MySQL persistence. It returns 422 for missing expected features. No batch endpoint is defined. The Flask route root and Laravel root are different services even when both use an /api path segment.

The ZIP archive setup/ids_ensemble-api-main.zip contains a historical Flask app.py with the same three route shapes but different persistence and feature behavior. It maps compact keys to older CIC-style names with spaces and capitalization, fills missing model inputs with zero rather than rejecting the request, logs the prediction, and forwards it to LARAVEL_LOG_URL using requests. Optional LARAVEL_API_KEY is added as an X-API-KEY header, with LARAVEL_TIMEOUT defaulting to eight seconds. The current Laravel ingestion route does not verify that header. The archive's behavior should not be assumed to provide incident grouping or the fourteen-feature deployed contract.

The archived model file is only 134 bytes, consistent with a Git LFS pointer placeholder rather than a complete trained estimator. A deployment using that archive requires checking its actual model artifact. The archive requirements include gunicorn and an older scikit-learn pin, but the archive does not supply proof of a current hosting configuration or production process command. Its existence explains the Render-oriented comments and default collector URL that coexist with the newer local direct-database service.

11. Database architecture, ownership, and relationships

The operational database is shared between Laravel and Python. Laravel migrations define schema and Eloquent models handle website queries and incident actions. The detector uses PyMySQL transactions to insert observations and assign incidents. The blocker uses PyMySQL to read desired incident status and mark restoration synchronization. Website, detector, and blocker must point to the same intended database and compatible schema. Their environment files are separate; matching names in source do not enforce matching live settings.

intrusion_logs stores per-analysis observations. Its id is an auto-incrementing primary key. incident_id is a nullable indexed unsigned big integer added by the incident migration. It is an application-level relationship; the migration does not define a foreign-key constraint. pppoe_name is a nullable indexed string of length one hundred. src_ip and dst_ip are nullable strings of length forty-five, allowing textual IPv6 storage even though the current collector parses IPv4 only. Ports and protocol are nullable

integers.

Flow measurements comprise nullable doubles for flow_duration, flow_pkts_s, flow_bytes_s, fwd_pkt_len_mean, bwd_pkt_len_mean, fwd_iat_mean, and bwd_iat_mean; nullable integers for tot_fwd_pkts, tot_bwd_pkts, tot_fwd_bytes, and tot_bwd_bytes. risk_level is a required string of length twenty. prob_attack is a required double. attack_type is a nullable string of length one hundred. created_at and updated_at are framework timestamp columns. A nullable legacy status enum is added and modified by later migrations.

The current final legacy status enum accepts blocked and resolved, with null representing unmarked observations. An intermediate migration also allowed pending, followed by another migration reclassifying pending rows as blocked and narrowing the enum. Current incident actions update threat_incidents instead of intrusion_logs.status. Therefore a blocked incident can have observation rows whose legacy status remains null, and those observations continue contributing to null-status dashboard queries. Distinguish legacy row visibility from current incident enforcement state.

threat_incidents stores the management unit. id is its auto-incrementing primary key. pppoe_name is a nullable indexed string of length one hundred. attack_type is a nullable indexed string. status is an indexed string of length twenty, default unresolved. event_count is an unsigned integer default one. first_detected_at is the first associated observation time; last_detected_at is the most recent and is indexed. blocked_at records a block request; restored_at records a restore request; firewall_synced_at records restoration synchronization processing. All three are nullable timestamps, as are the first and last detection times. created_at and updated_at track record lifecycle.

ThreatIncident logs is a has-many relationship using incident_id. latestLog is a has-one latestOfMany relationship selecting the latest related row by the model's default aggregation identity, not explicitly by a custom created_at ordering. IntrusionLog incident is a belongs-to relationship. No PPP subscriber master table, router-rule table, explicit user-to-incident action foreign key, or normalized attack-family reference table is defined. Subscriber identity is duplicated as text in incident and detection rows.

users_account is the actual custom website account table. It contains id, fullname, nullable unsigned tiny-integer age, nullable sex enum male/female/other, unique email, hashed password text storage, timestamps, and role added later. The role column is a string default admin, not a database enum restricted to only admin and viewer. Application permission checks accept only admin for mutation; any other role value is effectively read-only in this check.

The standard users table is separate and contains id, name, unique email, nullable email_verified_at, password, remember_token, and timestamps. password_reset_tokens stores a primary email, token, and nullable creation timestamp. sessions stores a primary string id, nullable indexed user_id, nullable client IP and user agent, a long-text session payload, and indexed last_activity. The user_id column is declared as a foreignId type but no constrained relationship is added in that scaffold migration.

personal_access_tokens is a Sanctum-compatible polymorphic token table. Its fields include id, tokenable_type, tokenable_id, name, unique sixty-four-character token, nullable abilities, nullable last_used_at, nullable indexed expires_at, and timestamps. The table's presence does not mean the custom website login issues tokens. No project route for creating these tokens is present in the inspected API file.

cache contains a primary key string, medium-text value, and integer expiration. cache_locks contains a primary key string, owner, and integer expiration. jobs stores id, indexed queue, long-text payload, attempts, nullable reserved_at, available_at, and created_at integers. job_batches stores id, name, total/pending/failed counts, failed job identifiers, nullable options, nullable cancelled_at, created_at, and nullable finished_at. failed_jobs stores id, unique UUID, connection, queue, payload, exception, and failed_at defaulting to the current time. These support framework infrastructure, not the local polling loop.

Laravel's migrations bookkeeping table is managed by the framework, rather than explicitly created in these application migrations. It records which migrations ran and their batch numbers. The file inventory also includes an optional local SQLite file if present; its existence is not proof that production uses SQLite. Several migration and query statements are MySQL-specific, so switching the full application to SQLite requires compatibility work even though isolated tests use SQLite.

12. Migration history and data transformation rules

The scaffold migrations create users, password reset tokens, sessions, cache tables, and queue tables. December 2025 migrations create intrusion_logs, personal access tokens, users_account, and a nullable intrusion-log status enum. June 2026 migrations broaden the status enum temporarily, backfill timestamps, reclassify pending statuses, and change one demo destination address. August 2026 migrations introduce PPPoE identity and incident tracking. October 2026 adds account roles.

The timestamp backfill replaces missing created_at or updated_at using the other timestamp and then the current database time. It has no meaningful reverse operation. The pending-status reclassification changes existing data, so a down migration that reopens the enum cannot reconstruct which blocked rows were formerly pending. The demo destination migration changes every matching destination address, not only rows explicitly marked as demonstrations, and its reverse similarly changes every matching replacement address.

The PPPoE identity migration adds the nullable indexed name column only if absent, then backfills two known demonstration source addresses to their configured subscriber names. It does not discover names for all historical observations. The incident migration creates indexed incident columns and adds nullable indexed incident_id to intrusion_logs without a database foreign key.

For historical attack logs, the incident migration reads all matching rows into memory, orders by ID, and groups by lowercased trimmed subscriber name, attack family, and legacy status. Each status group becomes a separate incident. Its first and last timestamps are taken from the first and last ID-ordered logs, rather than computing strict minimum and maximum timestamps. event_count equals group size. Legacy blocked and resolved values are retained; other values become unresolved. Each original row is linked to its incident. This historical grouping does not use the detector's fifteen-minute live grouping window.

Rolling back the incident migration drops the links and incident table, losing the new incident-management state. Applying migrations on a live database should be preceded by an appropriate backup and review of pending migrations. The targeted role migration used during the hosting walkthrough changes only users_account role, preserving existing account behavior with its admin default. The viewer account is created by a separate seeder; adding the role column alone does not create it.

13. Machine-learning training, artifacts, and recorded results

The maintained deployment training path is setup/Training/train_netflow_model.py with the contract in training_schema.py. The training README explicitly says it replaces the defective notebook path for deployment. Historical notebooks and manuscripts describe stacking ensembles and different thresholds; the current maintained script uses HistGradientBoostingClassifier for Stage A and RandomForestClassifier for Stage B. The project can still be described as a two-stage model, but the exact active estimators should be named accurately.

The dataset loader scans CSV files in the selected dataset directory, normalizes column names with Unicode normalization and non-alphanumeric replacement, locates a label-like column, and requires every feature in NETFLOW_FEATURES. It retains the fourteen numerical features, a normalized attack-family label, a capture filename, and a binary label determined by whether the normalized family is BENIGN. Numeric failures become missing values and infinities are replaced with missing values.

Optional max-rows-per-file sampling processes CSV chunks of fifty thousand rows, assigns reproducible random keys, and retains the smallest keys as a uniform sample across the file rather than simply taking the first records. The random seed begins at forty-two and varies by file index. The full dataset path loads all records. The source requires both binary classes and at least two attack families before training.

The split is seventy percent training, fifteen percent validation, and fifteen percent test, formed by a thirty-percent holdout followed by a half split. Stratification is on the binary label. It is not a capture-disjoint or time-disjoint split, even though capture filenames are retained. Related traffic from the same capture can therefore appear in more than one partition. This limits claims about generalization to unseen networks or future capture periods.

Each stage has its own cloned preprocessing pipeline: median imputation, StandardScaler, and a ColumnTransformer selecting the explicit feature list and dropping other columns. Independent cloning prevents Stage B from overwriting Stage A preprocessing state. Stage A uses two hundred maximum histogram-gradient-boosting iterations, learning rate 0.08, maximum thirty-one leaf nodes, balanced class weights, and random state forty-two. Stage B uses two hundred random-forest trees, maximum depth twenty-four, balanced-subsample weighting, all available jobs, and the same random state.

Stage A trains on all training rows and binary labels. Stage B trains only on training rows labeled attack and their family labels. select_threshold computes the precision-recall curve on validation probabilities, finds thresholds meeting target recall, and chooses the highest precision among those candidates. The default target recall is 0.95; invalid target values are rejected by argument parsing. The threshold is not chosen on the final test partition in the maintained script.

The bundle stores bin, mul, features, best_threshold, schema_version, dataset_profile, trained_at, and config. Configuration includes run type, sampling limit, random state, target recall, row count, and training elapsed time. dataset_profile is cicids-bootstrap. Full training defaults to setup/data/model/netflow_ids.joblib; a sampled run defaults to netflow_ids.smoke.joblib so it does not overwrite the full artifact. An explicit output argument can change either destination.

The trainer saves and reloads the artifact before evaluation and verifies that Stage A and Stage B do not share the same fitted preprocessor object. It computes full-test predictions using the stored threshold and writes a neighboring metrics JSON. A reproducible balanced check selects up to one hundred benign and one hundred attack rows from the untouched test partition. Small smoke datasets are marked incomplete when they cannot supply two hundred rows. The animated activity bar reports activity and elapsed time, not a trustworthy completion percentage.

The checked-in netflow_ids.metrics.json records a full run with schema mikrotik-netflow-v1 and threshold approximately 0.9865455954. Its full binary test matrix records 340751 true benign, 214 false-positive benign, 3537 missed attacks, and 80110 detected attacks across 424612 rows. Accuracy is approximately 99.1166 percent. Attack precision is approximately 99.7336 percent, attack recall 95.7715 percent, and attack F1 97.7124 percent. These are stored historical evaluation results, not measurements rerun for this documentation.

The balanced check records one hundred benign and one hundred attack rows, with one hundred benign correctly classified and ninety-seven attacks detected, giving 98.5 percent binary accuracy. Its attack-family report lists DDOS, DOS GOLDENEYE, DOS HULK, PORTSCAN, and SSH-PATATOR in that sample, with perfect family predictions on the selected true-attack rows. Family evaluation on true attacks is separate from end-to-end gated performance; missed attacks do not become operational family alerts even when Stage B would label them correctly in isolation.

The metrics file explicitly warns that CICIDS bootstrap results are not production validation and calls for labeled captures from the actual MikroTik environment. The old manuscripts' claims such as 99.92 percent binary accuracy or 99.83 percent multiclass accuracy must be attributed to those manuscripts and not substituted for the maintained artifact's metrics. Neither a metrics filename nor a repository copy proves that the live detector loaded that exact artifact or threshold.

The Dataset folder contains eight capture CSV files covering Monday working hours, Tuesday, Wednesday, Thursday morning web attacks, Thursday afternoon infiltration, Friday morning, Friday afternoon port scans, and a DDoS dataset. They are input material, not application runtime tables. Their individual names and sizes are included in the inventory. The full CSV payloads and serialized estimator internals are not reproduced in a prose reference.

14. Configuration, dependencies, and file responsibilities

Laravel requires PHP version 8.2 or newer under the composer constraint and Laravel framework major version twelve. Sanctum supports token authentication infrastructure; Tinker provides interactive application inspection. Development dependencies include Faker, Pail, Pint, Sail, Mockery, Collision, Pest, and the Pest Laravel plugin. composer.lock records resolved package versions and should be distinguished from composer.json's allowed ranges. Installed vendor source is third-party framework code, not custom detection logic.

The frontend manifest includes Vite major seven, Laravel's Vite plugin, Tailwind major four and its Vite integration, Axios, and Concurrently. npm run dev starts Vite; npm run build produces deployment assets. vite.config.js defines resources/css/app.css and resources/js/app.js as entrypoints and ignores compiled Blade cache paths in its watcher. resources/js/bootstrap.js initializes Axios and its XMLHttpRequest header. The administration pages often use public static assets directly, so not every visible element depends on rebuilding Vite.

The Python requirements include Flask, Gunicorn, joblib, NumPy, pandas, scikit-learn pinned to 1.6.1, XGBoost, Requests, python-dotenv, PyMySQL, and routeros-api. Some retained libraries serve historical models or experiments rather than the maintained trainer's active estimator selection. Artifact compatibility depends on the library versions used for serialization and loading. The Windows launcher assumes python resolves to an environment containing the required packages and does not activate a named virtual environment.

bootstrap/app.php registers web, API, console routes, /up, and the check.session middleware alias. bootstrap/providers.php registers AppServiceProvider. The provider currently contains no active custom application boot behavior; a force-HTTPS example is commented out. public/index.php is the web front controller loading Composer and the application bootstrap. artisan is the command-line entrypoint used for migrations, seeders, caches, tests, and framework commands.

config/app.php defines application identity, environment, debug flag, URL, locale, encryption configuration, and a hardcoded UTC timezone in this snapshot. config/database.php defines database connections; config/session.php defines storage and cookies; config/cache.php defines caching; config/queue.php defines asynchronous job infrastructure; config/filesystems.php defines storage disks; config/logging.php defines application log channels; config/mail.php and config/services.php define external service configuration; config/auth.php defines standard authentication; config/sanctum.php defines Sanctum behavior. Environment variables can override the supported settings, but secret values are not part of this reference.

Python settings shared in setup/.env include DB_HOST, DB_PORT, DB_DATABASE, DB_USERNAME, DB_PASSWORD, DB_TABLE, DB_INCIDENT_TABLE, MIKROTIK_HOST, MIKROTIK_PORT, MIKROTIK_USER, MIKROTIK_PASSWORD, API_URL, IDS_MODEL_FILENAME, IDS_THRESHOLD, INCIDENT_WINDOW_MINUTES, BLOCK_SCAN_INTERVAL, BLOCK_RULE_PREFIX, and LOG_DIR where referenced. The environment variable register later lists source references and safe descriptions. NETFLOW_PORT, ANALYSIS_INTERVAL, and MIN_FLOWS_FOR_ANALYSIS are current script constants rather than environment-controlled settings.

Default API_URL in the collector points to a Render-hosted inference endpoint, while setup README describes a local detector. The actual selected service depends on setup/.env. No environment values are published here. This mismatch must be resolved operationally by checking the configured inference URL, feature endpoint, model schema, and persistence path. A running local detector alone does not prove the collector is sending to it.

The repository includes .gitignore entries for environment files, node_modules, vendor, generated frontend artifacts, selected model artifacts, the primary IDS virtual environment, and training Dataset. Some alternate model files and a second virtual environment may still exist locally. Git ignore rules control new tracking, not already tracked content or web-server exposure. A file under public can become downloadable after deployment; project documentation therefore omits private configuration and runtime personal records.

The two ngrok executables are optional tunneling tools by location; no main runtime script launches them. Public ZIP template bundles and framework demonstration HTML pages are distribution or design assets. A PHPMailer library in the landing template is not evidence of an implemented contact-form mail service. Compiled Python cache files, notebook checkpoints, Laravel caches, logs, sessions, and virtual environments are generated development or runtime artifacts. Their purpose is recorded without treating them as independent system features.

15. Installation, startup, hosting, and change deployment

Prepare a monitoring host with a compatible Python interpreter and install setup/data/requirements.txt into an appropriate environment. Place a trusted compatible model bundle in setup/data/model and configure setup/.env for router access, database connectivity, inference URL, and optional thresholds and polling intervals. Configure the router's Traffic Flow export to the monitoring host and verify that port 2055 traffic and RouterOS API access are reachable. The repository's current collector requires NetFlow version five.

Prepare Laravel using Composer dependencies, the application environment, an existing valid application key, and the intended database. Ensure storage and bootstrap/cache are writable by the web process. Point the site's document root to Laravel public where hosting permits. Build frontend source assets if the deployment uses their Vite outputs. Run reviewed migrations and only the intended seeders. Do not regenerate an established production application key as a routine deployment step, because it changes encryption behavior.

Start the detector before the collector when API_URL targets it, then start the collector and the blocker. setup/Start-IDS.bat performs this order using separate Windows command windows but does not wait for API readiness, validate connectivity, restart failed services, or shut them down as a supervised group. The manual run instructions name Detection Process.py, Mikrotek PPPoe Sniffer.py, and Pending Blocker.py. Keep the services running for continuous collection and enforcement.

Verify each boundary independently: the detector feature endpoint reports the expected contract; the router emits accepted NetFlow; the collector recognizes current PPPoE names; inference accepts the required features; detection rows appear in the intended database; attack rows link to incidents; Laravel displays those rows; an authorized block changes the incident; the agent resolves the current subscriber address; the correct owned rule appears; restoration removes the appropriate rules subject to other blocked incidents. Tests should use controlled authorized subscribers and distinguish demo insertions from real model outputs.

For the hosted viewer-role change, deploy LoginController, CheckSession, UsersAccount, the manage-threats Blade view, the role migration, and the dedicated viewer seeder, plus DatabaseSeeder if preserving the full repository's seed configuration. Apply the role migration, then execute ViewOnlyUserSeeder. The hosting walkthrough established a working account-relative project path of domains/dissertationsystem.online/public_html/intrusion-detection-monitoring-system/artisan in the PHP cron form. The form automatically prepends the hosting account's home path and PHP executable; entering the full home prefix again duplicates it.

The successful targeted command runs artisan migrate with the path database/migrations/2026_10_29_000001_add_role_to_users_account_table.php and the production force option. The next operation runs artisan db:seed with class ViewOnlyUserSeeder and the force option. Temporary cron scheduling used an asterisk in every field; a day-of-month value zero is invalid. Remove each temporary cron job after successful execution. Leaving the seeder scheduled repeatedly resets the viewer account credentials and role. A seeding output containing only an informational seeding message can be normal; verify successful login and role behavior separately.

The correct path was established through user-provided cron results, not by deriving a server filesystem path from the File Manager URL. A browser File Manager address is not an absolute shell path. Production credentials and host-specific PHP executables must be checked on the hosting account. This reference documents the demonstrated workflow without guaranteeing that later hosting plan changes or server moves preserve identical paths.

After Blade changes, clear stale compiled views through the normal Artisan deployment process if required. Cached routes or configuration must be refreshed appropriately when those files change. Do not use migrate:fresh or other destructive rebuild commands against the live operational database. A role migration rollback removes the role column while the new model and middleware expect it, so application and schema rollback must be coordinated. Restore access should be verified using the actual router, not solely website badges.

16. Tests, verification boundaries, and acceptance scenarios

tests/Feature/PppoeBlockingIdentityTest.php builds minimal in-memory tables and exercises ManageThreatController directly. Its first test ensures block and restore change only the selected incident, leave a newer same-subscriber incident unresolved, leave a different subscriber untouched, and do not overwrite individual log legacy statuses. Its second test verifies that a resolved historical incident and a new unresolved attack remain visible as separate management entries under their respective status filters.

tests/Feature/ViewOnlyAccessTest.php creates the custom account schema and role column, seeds the viewer, and verifies login with the requested initial credentials. It posts to demo, block, restore, and bulk-action routes and expects forbidden responses. It uses a temporary protected endpoint to verify administrators may submit and that demotion affects the next request. It verifies a viewer can read and that the management HTML omits the demo modal and Actions column, while the administrator sees the modal.

tests/Feature/ExampleTest.php checks the public page response and tests/Unit/ExampleTest.php provides a basic truth assertion. tests/TestCase.php supplies the framework base; tests/Pest.php binds feature tests to it. phpunit.xml sets a testing environment, in-memory SQLite, array cache and sessions, synchronous queues, and reduced bcrypt rounds. The feature tests intentionally construct their needed tables rather than running the entire migration chain, which contains MySQL-specific statements.

Earlier verification for the viewer change reported eight passing tests and twenty-eight assertions. Those tests demonstrate isolated login and authorization behavior, incident selection, and page fragments. They do not exercise real RouterOS APIs, NetFlow parsing, machine-learning inference, MySQL schema upgrades, hosted cron execution, external HTTP connectivity, or all dashboard/system-status controller branches. The documented test count must not be presented as end-to-end production certification.

An administrator acceptance test should establish a controlled subscriber identity and its current dynamic address, observe a real attack incident or clearly identified demonstration incident, submit a block, and verify the expected owned source-address rule. It should reconnect the subscriber with a changed address and confirm that reconciliation removes the stale owned rule and protects the intended identity. A restore test should verify deletion on the router, not only the resolved badge or synchronization marker.

A multiple-incident scenario should block two incidents for one name, restore only one, and verify that the remaining blocked incident prevents internet restoration. Restoring the final blocked incident should allow cleanup. A history scenario should resolve an incident, generate a later attack for the same name and family, and verify a new unresolved incident is created. A viewer scenario should verify read access to all intended pages and direct POST rejection for each mutation endpoint.

A persistence-failure scenario should simulate an unavailable database in a safe test environment and confirm that inference success can coexist with storage failure, as implemented. A collector-idle scenario should test the traffic-driven timer behavior. A schema-mismatch scenario should omit a required feature and expect a 422 from the maintained detector. A report-search scenario should exercise q on the production schema because the optional notes assumption is not covered by current tests.

17. Operational limitations and accurate interpretation

The system is flow-based and periodically aggregates per-subscriber records. It does not inspect full packet payloads, decrypt TLS, provide signature updates, reconstruct every session, or prove compromise from a probability alone. A subscriber aggregate can contain many unrelated conversations. Displayed destination metadata is context from one latest record rather than a complete explanation of the prediction. Exporter configuration, filtering, and feature approximations influence what is observed.

Network availability is a dependency in several directions: router to collector UDP, collector to inference HTTP, detector to MySQL, Laravel to MySQL, blocker to MySQL, and blocker to RouterOS. An online website does not imply those local links are healthy. Historical log statistics can remain visible when collection has stopped. The system-status and live-monitor pages do not provide reliable independent process-liveness checks in their current implementation.

The current Flask API lacks source authentication and runs its direct entrypoint with debug enabled on all interfaces. RouterOS pools use unencrypted API settings. The Laravel ingestion API accepts mass-assigned permitted fields without explicit validation or API authentication. These are concrete source observations. Deployment should establish appropriate access boundaries, transport protections, request validation, and production process configuration rather than assuming website roles cover machine interfaces.

Database failure is logged but not returned as an inference error. There is no durable store-and-forward buffer or end-to-end message identifier. Incident creation lacks a unique eligibility constraint. Timestamp sources differ: Python uses host-local datetime, Laravel config declares UTC, browser timestamps use browser locale, and database synchronization uses NOW. Without consistent timezone configuration and clock synchronization, time filters, incident windows, uptime estimates, and activity charts can be misleading.

The viewer role is enforced on unsafe requests inside the protected web group. It does not automatically cover future mutations added outside that group, future GET endpoints that change state, or the separate ingestion API. The account role defaults to admin for compatibility; newly created custom accounts should have an explicit intended role. The initial viewer password is a user-requested setup credential, not a recommended permanent access policy, and is omitted here.

Status has two meanings in the database: legacy observation status and current incident status. Dashboard null-status filters do not mean no currently blocked subscriber contributed those observations. Incident event_count summarizes linked detections and may diverge if logs are inserted through an alternate path or manipulated independently. No database foreign key prevents orphaned incident links. The optional notes field is referenced in some paths without a matching migration in this snapshot.

Restoration synchronization is not fully verified by the current subscriber cleanup path because deletion failures can be swallowed. Block requests are not marked with a corresponding success acknowledgement and continue reconciliation. A new drop rule may not be effective if earlier router rules accept or FastTrack the traffic. Removing an IDS-owned drop rule cannot override unrelated firewall rules, service outages, upstream failures, or PPP account restrictions; a resolved incident therefore does not guarantee working internet service.

Historical research describes broader claims and architectures, including packet-based capture, stacking ensembles, and high benchmark scores. Those materials preserve project rationale but do not override current code. Current maintained training is a NetFlow-compatible bootstrap pipeline with a documented distribution gap. Production validation should use labeled data from the actual environment and separately measure false positives, missed attacks, family confusion, decision latency, and operational impact.

18. Maintenance, troubleshooting, and glossary

When no detections appear, inspect the collector console and merged_ids.log for accepted NetFlow version-five records, verify the exporter sender matches MIKROTIK_HOST, check excluded address prefixes, verify PPP discovery, and confirm at least ten records exist for the selected client. Consider queue length and traffic-driven timers before assuming the classifier is failing. Then confirm the configured API_URL targets the intended current detector rather than an older hosted API.

When inference rejects a request, compare /api/features against the sender's normalized keys and the training schema. Missing-model-features errors indicate a contract mismatch; present but malformed values may instead be coerced to zero. When inference succeeds but the website is unchanged, check database-save errors and the intended database/table settings, since an HTTP success does not certify persistence. Compare observation timestamps and avoid using the synthetic live-monitor values as a diagnostic.

When a block badge appears but traffic continues, verify the local agent is running, inspect blocked_threat_sync.log, confirm the subscriber currently has an active PPP session, inspect owned firewall rules and their source address, review rule order and FastTrack, and inspect connectivity to both database and router. When restoration does not work, check for another blocked incident for the same name, failed removal fallbacks, unrelated router rules, and whether the synchronization marker was set despite residual rules.

Keep the classifier bundle, ordered features, schema version, training script, metrics file, and runtime dependency versions together as a release unit. Keep website code, Python writers, and database schema compatible during upgrades. Maintain backups for operational rows, account data, incident state, and trusted model artifacts; runtime prediction and enforcement logs support diagnosis but do not replace database backups. Log rotation and retention are not implemented in the Python file handlers.

PPPoE means Point-to-Point Protocol over Ethernet; its username is treated as the stable subscriber identity. RouterOS is the MikroTik router operating system and API target. NetFlow is exported traffic-summary telemetry. A flow record summarizes traffic counters and endpoints rather than holding packet content. Forward traffic is traffic where the selected subscriber is source; backward traffic is traffic where it is destination. IAT means inter-arrival time, although the collector's stored IAT fields are

aggregate approximations.

Stage A is binary benign-versus-attack inference. Stage B is attack-family inference gated by Stage A. A threshold converts attack probability to a decision. Precision describes the proportion of predicted attacks that are truly attacks in an evaluated dataset; recall describes the proportion of actual attacks detected; F1 combines precision and recall. Bootstrap metrics are development evidence from a surrogate dataset, not deployed accuracy. An incident is a management record linking observations for a subscriber and family according to grouping rules.

Unresolved means awaiting an administrator decision. Blocked means the agent should maintain a subscriber-owned drop rule. Resolved means restoration has been requested and the incident remains in history. firewall_synced_at is the agent's restoration-processing marker with the verification limitations described earlier. CSRF protects browser state-changing requests against cross-site forgery within the web stack. Sanctum is the separate Laravel token/session authentication infrastructure, not the custom UsersAccount login itself.

The following automatically derived registers anchor the narrative to specific local source files and line locations. They are a reference map, not executable instructions or an attempt to repeat every dependency's source. Research extracts are explicitly historical and preserve their own claims for comparison. The final inventory lists project files while grouping installed dependencies, private runtime data, and generated caches by category.

19. Source member register: Python services and training

Each entry below names a source-defined class or function and its line location. A class entry groups related responsibilities; a function entry records its local documentation or main operation families. Call names are a source navigation aid, not a claim that every call executes on every request. The detailed behavioral explanations in the earlier chapters take precedence over short historical docstrings.

setup/Detection Process.py

_pause_on_error, line 5: accepts exc_type, exc, tb. Referenced operations include print_exception, input.

assign_incident, line 66: accepts cursor, record, now_text. Referenced operations include execute, fetchone.

append_log, line 110: accepts record. Referenced operations include makedirs, open, write, dumps.

save_to_database, line 118: accepts record. Referenced operations include strftime, connect, now, commit, close, cursor, assign_incident, execute, rollback, list.

home, line 178: accepts no explicit positional parameters. Referenced operations include route.

features, line 183: accepts no explicit positional parameters. Referenced operations include route, jsonify.

analyze, line 190: accepts no explicit positional parameters. Referenced operations include route, DataFrame, append_log, save_to_database, jsonify, get_json, predict_proba, time, isnan, isinf, predict.

setup/Mikrotek PPPoe Sniffer.py

_pause_on_error, line 11: accepts exc_type, exc, tb. Referenced operations include print_exception, input.

is_client_ip, line 88: accepts ip. Monitor ALL IPs except known ISP/infrastructure networks. Referenced operations include startswith.

is_infrastructure_ip, line 115: accepts ip. Check if IP is infrastructure (not a client) Referenced operations include startswith.

MikroTikConnector, line 126: class grouping 4 directly declared methods. Connect to MikroTik using API (port 8728).

MikroTikConnector.__init__, line 129: accepts no explicit positional parameters. A local helper providing the return or state update described by its name and enclosing component.

MikroTikConnector.connect, line 134: accepts no explicit positional parameters. Referenced operations include RouterOsApiPool, get_api.

MikroTikConnector.get_pppoe_clients, line 154: accepts no explicit positional parameters. Return active PPPoE clients as {current_ip: stable_username}. Referenced operations include get_resource, connect, is_client_ip.

MikroTikConnector.disconnect, line 176: accepts no explicit positional parameters. Referenced operations include disconnect.

NetFlowCollector, line 187: class grouping 10 directly declared methods.

NetFlowCollector.__init__, line 188: accepts no explicit positional parameters. Referenced operations include socket, bind, settimeout, defaultdict, deque, set.

NetFlowCollector.parse_netflow_v5, line 210: accepts data. Parse NetFlow v5 packet Referenced operations include unpack, inet_ntoa, is_infrastructure_ip, isoformat, now.

NetFlowCollector.update_client_list, line 258: accepts api_clients. Update client list from MikroTik API Referenced operations include dict, set, deque, intersection_update, add.

NetFlowCollector.get_next_client, line 281: accepts no explicit positional parameters. Get the next client in round-robin queue Referenced operations include rotate.

NetFlowCollector.calculate_api_payload, line 288: accepts client_ip, all_flows. Convert NetFlow data to match Render API expected format Referenced operations include isoformat, extend, now.

NetFlowCollector.store_flow, line 423: accepts flow. Store flow in both source and destination indices A local helper providing the return or state update described by its name and enclosing component.

NetFlowCollector.get_client_flows, line 428: accepts client_ip. Get all flows where client is either source or destination A local helper providing the return or state update described by its name and enclosing component.

NetFlowCollector.send_to_api, line 434: accepts payload. Send to Render API Referenced operations include post, json.

NetFlowCollector.analyze_next_client, line 481: accepts no explicit positional parameters. Analyze the next client in round-robin queue Referenced operations include get_next_client, get_client_flows, calculate_api_payload, send_to_api.

NetFlowCollector.run, line 523: accepts no explicit positional parameters. Main collector loop Referenced operations include time, MikroTikConnector, recvfrom, get_pppoe_clients, analyze_next_client, list, unpack, parse_netflow_v5, update_client_list, store_flow, timestamp, fromisoformat.

MergedIDS, line 589: class grouping 2 directly declared methods.

MergedIDS.__init__, line 590: accepts no explicit positional parameters. Referenced operations include NetFlowCollector.

MergedIDS.run, line 593: accepts no explicit positional parameters. Referenced operations include MikroTikConnector, get_pppoe_clients, run, update_client_list.

setup/Pending Blocker.py

_pause_on_error, line 11: accepts exc_type, exc, tb. Referenced operations include print_exception, input.

RouterOsDirectClient, line 68: class grouping 11 directly declared methods.

RouterOsDirectClient.__init__, line 69: accepts host, port, username, password, timeout. A local helper providing the return or state update described by its name and enclosing component.

RouterOsDirectClient.__enter__, line 78: accepts no explicit positional parameters. Referenced operations include create_connection, settimeout, _login.

RouterOsDirectClient.__exit__, line 86: accepts exc_type, exc, tb. Referenced operations include close.

RouterOsDirectClient._write_len, line 94: accepts length. Referenced operations include sendall, bytes.

RouterOsDirectClient._write_word, line 124: accepts word. Referenced operations include encode, _write_len, sendall.

RouterOsDirectClient._write_sentence, line 129: accepts words. Referenced operations include _write_word.

RouterOsDirectClient._read_len, line 134: accepts no explicit positional parameters. Referenced operations include recv, ConnectionError.

RouterOsDirectClient._read_word, line 159: accepts no explicit positional parameters. Referenced operations include _read_len, decode, recv, ConnectionError.

RouterOsDirectClient._read_sentence, line 171: accepts no explicit positional parameters. Referenced operations include _read_word.

RouterOsDirectClient._login, line 179: accepts no explicit positional parameters. Referenced operations include _write_sentence, _read_sentence, fromhex, hexdigest, any, startswith, RuntimeError, md5, split, encode.

RouterOsDirectClient.run, line 210: accepts command, args. Referenced operations include _write_sentence, _read_sentence, any, RuntimeError.

db_connect, line 225: accepts no explicit positional parameters. Referenced operations include connect.

get_blocked_subscribers, line 238: accepts no explicit positional parameters. Referenced operations include RuntimeError, db_connect, cursor, execute, fetchall.

get_pending_restores, line 257: accepts no explicit positional parameters. Referenced operations include RuntimeError, db_connect, cursor, execute, fetchall.

MikroTikBlocker, line 280: class grouping 24 directly declared methods.

MikroTikBlocker.__init__, line 281: accepts no explicit positional parameters. A local helper providing the return or state update described by its name and enclosing component.

MikroTikBlocker._rule_comment, line 286: accepts dst_ip. A local helper providing the return or state update described by its name and enclosing component.

MikroTikBlocker._format_rule, line 291: accepts row. A local helper providing the return or state update described by its name and enclosing component.

MikroTikBlocker._get_rule_id, line 301: accepts row. Get rule ID from either 'id' or '.id' key, preserving the asterisk A local helper providing the return or state update described by its name and enclosing component.

MikroTikBlocker.connect, line 310: accepts no explicit positional parameters. Referenced operations include RouterOsApiPool, get_api.

MikroTikBlocker.ensure_connected, line 323: accepts no explicit positional parameters. Referenced operations include connect.

MikroTikBlocker.is_already_blocked, line 330: accepts dst_ip. Referenced operations include ensure_connected, get_resource, bool, _rule_comment, _format_rule.

MikroTikBlocker._get_matching_rules, line 351: accepts dst_ip. Referenced operations include ensure_connected, get_resource, _rule_comment.

MikroTikBlocker.block_ip, line 364: accepts dst_ip. Referenced operations include _get_matching_rules, _rule_comment, clear_connections, add, _get_rule_id, _set_firewall_rule_enabled, _format_rule.

MikroTikBlocker.unblock_ip, line 428: accepts dst_ip. Referenced operations include _get_matching_rules, enumerate, clear_connections, _get_rule_id, _remove_firewall_rule, _rule_comment, _format_rule.

MikroTikBlocker._subscriber_rule_comment, line 504: accepts pppoe_name. A local helper providing the return or state update described by its name and enclosing component.

MikroTikBlocker._get_subscriber_rules, line 507: accepts pppoe_name. Referenced operations include ensure_connected, get_resource, _subscriber_rule_comment.

MikroTikBlocker._known_ips_for_pppoe, line 514: accepts pppoe_name. Referenced operations include db_connect, cursor, execute, fetchall.

MikroTikBlocker._remove_legacy_subscriber_rules, line 527: accepts pppoe_name. Referenced operations include _known_ips_for_pppoe, ensure_connected, get_resource, _get_rule_id, any, _remove_firewall_rule.

MikroTikBlocker.resolve_current_ip_for_pppoe, line 546: accepts pppoe_name. Referenced operations include ensure_connected, get_resource, RuntimeError.

MikroTikBlocker.block_subscriber, line 565: accepts pppoe_name. Referenced operations include _remove_legacy_subscriber_rules, resolve_current_ip_for_pppoe, _get_subscriber_rules, _subscriber_rule_comment, clear_connections, _get_rule_id, _remove_firewall_rule, add, _set_firewall_rule_enabled.

MikroTikBlocker.unblock_subscriber, line 614: accepts pppoe_name. Referenced operations include _remove_legacy_subscriber_rules, _get_subscriber_rules, set, _get_rule_id, add, clear_connections, _remove_firewall_rule, resolve_current_ip_for_pppoe.

MikroTikBlocker.mark_restore_synced, line 635: accepts incident_id. Referenced operations include RuntimeError, db_connect, cursor, execute.

MikroTikBlocker._set_firewall_rule_enabled, line 652: accepts resource, rule_id, dst_ip, enabled. Referenced operations include lstrip, enumerate, enable, disable, RouterOsDirectClient, run.

MikroTikBlocker._remove_firewall_rule, line 697: accepts resource, rule_id, dst_ip. Remove a firewall rule completely (delete it) Referenced operations include lstrip, method_func, RouterOsDirectClient, remove, _remove_via_direct_api, run.

MikroTikBlocker._remove_via_direct_api, line 743: accepts params. Remove a firewall rule using direct RouterOS API Referenced operations include RouterOsDirectClient, run.

MikroTikBlocker.clear_connections, line 749: accepts dst_ip. Referenced operations include ensure_connected, get_resource, set, _get_rule_id, add, remove, _format_rule.

MikroTikBlocker.clear_ppp_active, line 783: accepts dst_ip. Referenced operations include ensure_connected, get_resource, _get_rule_id, remove, _format_rule.

MikroTikBlocker.close, line 809: accepts no explicit positional parameters. Referenced operations include disconnect.

main, line 818: accepts no explicit positional parameters. Referenced operations include MikroTikBlocker, ensure_connected, sleep, get_pending_restores, get_blocked_subscribers, block_subscriber, unblock_subscriber, mark_restore_synced, format_exc.

setup/Training/train_netflow_model.py

activity_bar, line 47: accepts label. Show an indeterminate progress bar while a blocking ML operation runs. Referenced operations include Event, monotonic, Thread, start, set, write, is_set, flush, wait.

normalize_name, line 81: accepts value. Referenced operations include sub, normalize.

normalize_label, line 87: accepts value. Referenced operations include normalize, replace, upper, sub.

read_csv_sample, line 93: accepts csv_path, sample_size, seed. Referenced operations include default_rng, DataFrame, read_csv, reset_index, random, concat, nsmallest, drop.

load_dataset, line 109: accepts dataset_dir, max_rows_per_file. Referenced operations include sorted, enumerate, concat, replace, astype, glob, FileNotFoundError, read_csv_sample, next, copy, map, to_numeric, normalize_name, ValueError.

select_threshold, line 144: accepts y_true, probabilities, target_recall. Referenced operations include precision_recall_curve, flatnonzero, argmax.

main, line 153: accepts no explicit positional parameters. Referenced operations include ArgumentParser, add_argument, parse_args, load_dataset, train_test_split, ColumnTransformer, Pipeline, time, mkdir, default_rng, to_numpy, flatnonzero, concatenate, shuffle, astype, predict, with_suffix, write_text, nunique, ValueError, activity_bar, fit, select_threshold, list, strftime, dump, load, RuntimeError, tolist, classification_report, dumps, predict_proba, choice, clone, HistGradientBoostingClassifier, RandomForestClassifier, confusion_matrix, SimpleImputer, StandardScaler.

setup/Training/training_schema.py

Declares the ordered NetFlow feature contract and schema identifier used by the maintained training path.

20. Source member register: PHP controllers, models, and tests

This register lists every declared PHP method in the custom application and test files, together with its source line. Earlier chapters explain runtime behavior; these entries make source navigation and maintenance assignment precise. An endpoint is only active when its route registration and middleware permit it; a helper's existence does not establish a public endpoint.

app/Http/Controllers/DashboardController.php

gotoDashboard, line 11. Builds initial dashboard metrics and time-window labels.

getWindowLabel, line 44. Internal component helper for get Window Label.

getDateRange, line 55. Internal component helper for get Date Range.

getOldestCreatedAt, line 69. Internal component helper for get Oldest Created At.

getPeriodLabel, line 76. Internal component helper for get Period Label.

getChartData, line 96. Internal component helper for get Chart Data.

getAttacksByType, line 172. Internal component helper for get Attacks By Type.

getTopAttackedIPs, line 193. Internal component helper for get Top Attacked I Ps.

getRiskLevelDistribution, line 207. Internal component helper for get Risk Level Distribution.

refreshData, line 225. Returns selected dashboard metrics for browser polling.

getDashboardMetrics, line 249. Internal component helper for get Dashboard Metrics.

app/Http/Controllers/IntrusionLogController.php

store, line 10. Creates one mass-assignable intrusion log from request data and returns a saved status.

app/Http/Controllers/LiveMonitorController.php

gotoLive, line 9. Returns placeholder data for the simulated live-monitor page.

app/Http/Controllers/LogController.php

gotoLogs, line 11. Applies optional log filters, schema-compatible searches, and ten-row pagination.

app/Http/Controllers/LoginController.php

login, line 14. Validates email and password, verifies the custom account hash, and establishes a regenerated website session.

logout, line 46. Flushes and invalidates the session, regenerates the CSRF token, and redirects to login.

app/Http/Controllers/ManageThreatController.php

index, line 12. Builds the page query, selected filters, statistics or incident display data, and returns the associated view.

block, line 84. Marks one identified subscriber incident blocked and resets restoration synchronization state.

ignore, line 98. Marks one identified subscriber incident resolved and pending router restoration processing.

bulkAction, line 111. Applies a submitted block or resolution action to selected incident IDs.

demo, line 136. Validates two destination addresses and creates demonstration incidents and observation rows.

app/Http/Controllers/SystemStatusController.php

gotoStatus, line 11. Builds system statistics, processing metrics, health labels, and the activity timeline.

alertFragment, line 29. Renders only the database-derived health alert for AJAX polling.

getSystemStats, line 40. Internal component helper for get System Stats.

getProcessingMetrics, line 126. Internal component helper for get Processing Metrics.

getHealthIndicators, line 151. Internal component helper for get Health Indicators.

getActivityTimeline, line 184. Internal component helper for get Activity Timeline.

app/Http/Controllers/ThreatReportsController.php

index, line 13. Builds the page query, selected filters, statistics or incident display data, and returns the associated view.

data, line 54. Returns up to two hundred filtered report observations in a JSON rows object.

export, line 103. Streams a CSV using a database cursor and the selected time window.

app/Http/Middleware/CheckSession.php

handle, line 17. Reloads the custom account and enforces session existence and admin-only unsafe requests.

app/Models/IntrusionLog.php

incident, line 21. Defines the observation-to-incident belongs-to relation.

app/Models/ThreatIncident.php

casts, line 17. Defines attribute conversions used by Eloquent.

logs, line 28. Defines the incident-to-observations has-many relation.

latestLog, line 33. Defines a latest-of-many related observation for management display.

app/Models/User.php

casts, line 41. Defines attribute conversions used by Eloquent.

app/Models/UsersAccount.php

canManageThreats, line 51. Returns whether the role is exactly admin.

getNameAttribute, line 60. Maps display name to fullname for the custom account.

app/Providers/AppServiceProvider.php

register, line 13. Framework service registration hook; no custom behavior in this provider.

boot, line 21. Framework provider startup hook; no active custom HTTPS forcing in this snapshot.

tests/Feature/PppoeBlockingIdentityTest.php

setUp, line 15. Builds the isolated test preconditions and minimal database schema.

test_block_and_restore_change_only_the_selected_incident, line 48. Regression test for block and restore change only the selected incident.

test_resolved_history_and_new_unresolved_attack_coexist, line 73. Regression test for resolved history and new unresolved attack coexist.

incident, line 101. Defines the observation-to-incident belongs-to relation.

log, line 114. Internal component helper for log.

tests/Feature/ViewOnlyAccessTest.php

setUp, line 12. Builds the isolated test preconditions and minimal database schema.

test_requested_credentials_can_log_in, line 21. Regression test for requested credentials can log in.

test_viewer_cannot_submit_any_threat_changes, line 28. Regression test for viewer cannot submit any threat changes.

test_permissions_are_reloaded_and_admin_can_submit, line 37. Regression test for permissions are reloaded and admin can submit.

test_viewer_can_read_and_mutating_controls_are_absent, line 47. Regression test for viewer can read and mutating controls are absent.

database/seeders/DatabaseSeeder.php

run, line 16. Executes the seeder account creation or update operations.

database/seeders/ViewOnlyUserSeeder.php

run, line 11. Executes the seeder account creation or update operations.

21. Exact schema declaration register

The following declarations are transcribed automatically into prose from every application migration. They supplement the consolidated database dictionary with field sizes, explicit defaults, indexes, and rollback history. Entries from later migrations may alter earlier declarations; they must be read in filename order. The register describes declared schema, not a queried production schema. Row transformations are explained in the migration-history chapter.

database/migrations/0001_01_01_000000_create_users_table.php

Schema operation create concerns table users at line 14.

Schema operation create concerns table password_reset_tokens at line 24.

Schema operation create concerns table sessions at line 30.

Schema operation dropIfExists concerns table users at line 45.

Schema operation dropIfExists concerns table password_reset_tokens at line 46.

Schema operation dropIfExists concerns table sessions at line 47.

Declaration id: framework default columns.

Declaration string: name.

Declaration string: email. Modifiers: unique .

Declaration timestamp: email_verified_at. Modifiers: nullable .

Declaration string: password.

Declaration rememberToken: framework default columns.

Declaration timestamps: framework default columns.

Declaration string: email. Modifiers: primary .

Declaration string: token.

Declaration timestamp: created_at. Modifiers: nullable .

Declaration string: id. Modifiers: primary .

Declaration foreignId: user_id. Modifiers: nullable ; index .

Declaration string: ip_address, 45. Modifiers: nullable .

Declaration text: user_agent. Modifiers: nullable .

Declaration longText: payload.

Declaration integer: last_activity. Modifiers: index .

database/migrations/0001_01_01_000001_create_cache_table.php

Schema operation create concerns table cache at line 14.

Schema operation create concerns table cache_locks at line 20.

Schema operation dropIfExists concerns table cache at line 32.

Schema operation dropIfExists concerns table cache_locks at line 33.

Declaration string: key. Modifiers: primary .

Declaration mediumText: value.

Declaration integer: expiration.

Declaration string: key. Modifiers: primary .

Declaration string: owner.

Declaration integer: expiration.

database/migrations/0001_01_01_000002_create_jobs_table.php

Schema operation create concerns table jobs at line 14.

Schema operation create concerns table job_batches at line 24.

Schema operation create concerns table failed_jobs at line 37.

Schema operation dropIfExists concerns table jobs at line 53.

Schema operation dropIfExists concerns table job_batches at line 54.

Schema operation dropIfExists concerns table failed_jobs at line 55.

Declaration id: framework default columns.

Declaration string: queue. Modifiers: index .

Declaration longText: payload.

Declaration unsignedTinyInteger: attempts.

Declaration unsignedInteger: reserved_at. Modifiers: nullable .

Declaration unsignedInteger: available_at.

Declaration unsignedInteger: created_at.

Declaration string: id. Modifiers: primary .

Declaration string: name.

Declaration integer: total_jobs.

Declaration integer: pending_jobs.

Declaration integer: failed_jobs.

Declaration longText: failed_job_ids.

Declaration mediumText: options. Modifiers: nullable .

Declaration integer: cancelled_at. Modifiers: nullable .

Declaration integer: created_at.

Declaration integer: finished_at. Modifiers: nullable .

Declaration id: framework default columns.

Declaration string: uuid. Modifiers: unique .

Declaration text: connection.

Declaration text: queue.

Declaration longText: payload.

Declaration longText: exception.

Declaration timestamp: failed_at. Modifiers: useCurrent .

database/migrations/2025_12_14_101328_create_intrusion_logs_table.php

Schema operation create concerns table intrusion_logs at line 14.

Schema operation dropIfExists concerns table intrusion_logs at line 49.

Declaration id: framework default columns.

Declaration string: src_ip, 45. Modifiers: nullable .

Declaration string: dst_ip, 45. Modifiers: nullable .

Declaration integer: src_port. Modifiers: nullable .

Declaration integer: dst_port. Modifiers: nullable .

Declaration integer: protocol. Modifiers: nullable .

Declaration double: flow_duration. Modifiers: nullable .

Declaration double: flow_pkts_s. Modifiers: nullable .

Declaration double: flow_bytes_s. Modifiers: nullable .

Declaration integer: tot_fwd_pkts. Modifiers: nullable .

Declaration integer: tot_bwd_pkts. Modifiers: nullable .

Declaration integer: tot_fwd_bytes. Modifiers: nullable .

Declaration integer: tot_bwd_bytes. Modifiers: nullable .

Declaration double: fwd_pkt_len_mean. Modifiers: nullable .

Declaration double: bwd_pkt_len_mean. Modifiers: nullable .

Declaration double: fwd_iat_mean. Modifiers: nullable .

Declaration double: bwd_iat_mean. Modifiers: nullable .

Declaration string: risk_level, 20.

Declaration double: prob_attack.

Declaration string: attack_type, 100. Modifiers: nullable .

Declaration timestamps: framework default columns.

database/migrations/2025_12_14_102846_create_personal_access_tokens_table.php

Schema operation create concerns table personal_access_tokens at line 14.

Schema operation dropIfExists concerns table personal_access_tokens at line 31.

Declaration id: framework default columns.

Declaration morphs: tokenable.

Declaration text: name.

Declaration string: token, 64. Modifiers: unique .

Declaration text: abilities. Modifiers: nullable .

Declaration timestamp: last_used_at. Modifiers: nullable .

Declaration timestamp: expires_at. Modifiers: nullable ; index .

Declaration timestamps: framework default columns.

database/migrations/2025_12_17_123456_create_users_account_table.php

Schema operation create concerns table users_account at line 16.

Schema operation dropIfExists concerns table users_account at line 34.

Declaration id: framework default columns.

Declaration string: fullname.

Declaration unsignedTinyInteger: age. Modifiers: nullable .

Declaration enum: sex, [male, female, other]. Modifiers: nullable .

Declaration string: email. Modifiers: unique .

Declaration string: password.

Declaration timestamps: framework default columns.

database/migrations/2025_12_19_000000_add_status_to_intrusion_logs_table.php

Schema operation table concerns table intrusion_logs at line 15.

Schema operation table concerns table intrusion_logs at line 26.

Declaration enum: status, [blocked, resolved]. Modifiers: nullable ; default null; after attack_type.

Declaration dropColumn: status.

database/migrations/2026_06_23_000001_update_intrusion_logs_status_enum.php

SQL migration operation: ALTER TABLE intrusion_logs MODIFY status ENUM('pending', 'blocked', 'resolved') NULL DEFAULT NULL.

SQL migration operation: ALTER TABLE intrusion_logs MODIFY status ENUM('blocked', 'resolved') NULL DEFAULT NULL.

database/migrations/2026_06_23_000002_backfill_intrusion_logs_timestamps.php

SQL migration operation: UPDATE intrusion_logs SET created_at = COALESCE(created_at, updated_at, NOW()), updated_at = COALESCE(updated_at, created_at, NOW()) WHERE created_at IS NULL OR updated_at IS NULL.

database/migrations/2026_06_23_000003_reclassify_pending_intrusion_logs_to_blocked.php

SQL migration operation: UPDATE intrusion_logs SET status = NULL WHERE status = 'pending'.

SQL migration operation: ALTER TABLE intrusion_logs MODIFY status ENUM('blocked', 'resolved') NULL DEFAULT NULL.

SQL migration operation: ALTER TABLE intrusion_logs MODIFY status ENUM('pending', 'blocked', 'resolved') NULL DEFAULT NULL.

database/migrations/2026_06_24_000001_update_demo_dst_ip_in_intrusion_logs.php

database/migrations/2026_08_02_000001_add_pppoe_name_to_intrusion_logs.php

Schema operation table concerns table intrusion_logs at line 13.

Schema operation table concerns table intrusion_logs at line 26.

Declaration string: pppoe_name, 100. Modifiers: nullable ; after src_ip; index .

Declaration dropIndex: [pppoe_name].

Declaration dropColumn: pppoe_name.

database/migrations/2026_08_02_000002_create_threat_incidents_table.php

Schema operation create concerns table threat_incidents at line 12.

Schema operation table concerns table intrusion_logs at line 26.

Schema operation table concerns table intrusion_logs at line 73.

Schema operation dropIfExists concerns table threat_incidents at line 77.

Declaration id: framework default columns.

Declaration string: pppoe_name, 100. Modifiers: nullable ; index .

Declaration string: attack_type. Modifiers: nullable ; index .

Declaration string: status, 20. Modifiers: default unresolved; index .

Declaration unsignedInteger: event_count. Modifiers: default 1.

Declaration timestamp: first_detected_at. Modifiers: nullable .

Declaration timestamp: last_detected_at. Modifiers: nullable ; index .

Declaration timestamp: blocked_at. Modifiers: nullable .

Declaration timestamp: restored_at. Modifiers: nullable .

Declaration timestamp: firewall_synced_at. Modifiers: nullable .

Declaration timestamps: framework default columns.

Declaration unsignedBigInteger: incident_id. Modifiers: nullable ; after id; index .

Declaration dropIndex: [incident_id].

Declaration dropColumn: incident_id.

database/migrations/2026_10_29_000001_add_role_to_users_account_table.php

Schema operation table concerns table users_account at line 11.

Schema operation table concerns table users_account at line 19.

Declaration string: role. Modifiers: default admin.

Declaration dropColumn: role.

22. Environment variable register without private values

This list is derived from getenv and env references in application configuration, bootstrap, and current setup scripts. Values in private environment files are not read into the document. Variables marked as access material must be configured privately. A variable referenced only in a historical archive belongs to that archive's deployment, not automatically the active detector.

Configuration names and consumers

API_URL. Target inference request URL; determines whether the collector uses local or hosted inference. Referenced by setup/Mikrotek PPPoe Sniffer.py.

APP_DEBUG. Framework or component setting; its exact consumers are the referenced source files. Referenced by config/app.php.

APP_ENV. Framework or component setting; its exact consumers are the referenced source files. Referenced by config/app.php.

APP_FAKER_LOCALE. Framework or component setting; its exact consumers are the referenced source files. Referenced by config/app.php.

APP_FALLBACK_LOCALE. Framework or component setting; its exact consumers are the referenced source files. Referenced by config/app.php.

APP_KEY. Private access or cryptographic configuration; do not publish its configured value. Referenced by config/app.php.

APP_LOCALE. Framework or component setting; its exact consumers are the referenced source files. Referenced by config/app.php.

APP_MAINTENANCE_DRIVER. Framework or component setting; its exact consumers are the referenced source files. Referenced by config/app.php.

APP_MAINTENANCE_STORE. Framework or component setting; its exact consumers are the referenced source files. Referenced by config/app.php.

APP_NAME. Framework or component setting; its exact consumers are the referenced source files. Referenced by config/app.php, config/cache.php, config/database.php, config/session.php.

APP_PREVIOUS_KEYS. Private access or cryptographic configuration; do not publish its configured value. Referenced by config/app.php.

APP_URL. Framework or component setting; its exact consumers are the referenced source files. Referenced by config/app.php, config/filesystems.php, config/mail.php.

AUTH_GUARD. Framework or component setting; its exact consumers are the referenced source files. Referenced by config/auth.php.

AUTH_MODEL. Framework or component setting; its exact consumers are the referenced source files. Referenced by config/auth.php.

AUTH_PASSWORD_BROKER. Private access or cryptographic configuration; do not publish its configured value. Referenced by config/auth.php.

AUTH_PASSWORD_RESET_TOKEN_TABLE. Private access or cryptographic configuration; do not publish its configured value. Referenced by config/auth.php.

AUTH_PASSWORD_TIMEOUT. Private access or cryptographic configuration; do not publish its configured value. Referenced by config/auth.php.

AWS_ACCESS_KEY_ID. Private access or cryptographic configuration; do not publish its configured value. Referenced by config/cache.php, config/filesystems.php, config/queue.php, config/services.php.

AWS_BUCKET. Optional framework mail, cloud, or external service configuration; not proof that the service is used by the IDS path. Referenced by config/filesystems.php.

AWS_DEFAULT_REGION. Optional framework mail, cloud, or external service configuration; not proof that the service is used by the IDS path. Referenced by config/cache.php, config/filesystems.php, config/queue.php, config/services.php.

AWS_ENDPOINT. Optional framework mail, cloud, or external service configuration; not proof that the service is used by the IDS path. Referenced by config/filesystems.php.

AWS_SECRET_ACCESS_KEY. Private access or cryptographic configuration; do not publish its configured value. Referenced by config/cache.php, config/filesystems.php, config/queue.php, config/services.php.

AWS_URL. Optional framework mail, cloud, or external service configuration; not proof that the service is used by the IDS path. Referenced by config/filesystems.php.

AWS_USE_PATH_STYLE_ENDPOINT. Optional framework mail, cloud, or external service configuration; not proof that the service is used by the IDS path. Referenced by config/filesystems.php.

BEANSTALKD_QUEUE. Framework or component setting; its exact consumers are the referenced source files. Referenced by config/queue.php.

BEANSTALKD_QUEUE_HOST. Framework or component setting; its exact consumers are the referenced source files. Referenced by config/queue.php.

BEANSTALKD_QUEUE_RETRY_AFTER. Framework or component setting; its exact consumers are the referenced source files. Referenced by config/queue.php.

BLOCK_RULE_PREFIX. Ownership prefix used in managed router comments. Referenced by setup/Pending Blocker.py.

BLOCK_SCAN_INTERVAL. Seconds between enforcement scans; default twenty. Referenced by setup/Pending Blocker.py.

CACHE_PREFIX. Framework cache or backing-store configuration. Referenced by config/cache.php.

CACHE_STORE. Framework cache or backing-store configuration. Referenced by config/cache.php.

DB_CACHE_CONNECTION. Database connection, storage, or transport configuration. Referenced by config/cache.php.

DB_CACHE_LOCK_CONNECTION. Database connection, storage, or transport configuration. Referenced by config/cache.php.

DB_CACHE_LOCK_TABLE. Database connection, storage, or transport configuration. Referenced by config/cache.php.

DB_CACHE_TABLE. Database connection, storage, or transport configuration. Referenced by config/cache.php.

DB_CHARSET. Database connection, storage, or transport configuration. Referenced by config/database.php.

DB_COLLATION. Database connection, storage, or transport configuration. Referenced by config/database.php.

DB_CONNECTION. Database connection, storage, or transport configuration. Referenced by config/database.php, config/queue.php.

DB_DATABASE. Database connection, storage, or transport configuration. Referenced by config/database.php, setup/Detection Process.py, setup/Pending Blocker.py.

DB_ENCRYPT. Database connection, storage, or transport configuration. Referenced by config/database.php.

DB_FOREIGN_KEYS. Private access or cryptographic configuration; do not publish its configured value. Referenced by config/database.php.

DB_HOST. Database connection, storage, or transport configuration. Referenced by config/database.php, setup/Detection Process.py, setup/Pending Blocker.py.

DB_INCIDENT_TABLE. Incident-management table name for Python grouping and enforcement. Referenced by setup/Detection Process.py, setup/Pending Blocker.py.

DB_PASSWORD. Private access or cryptographic configuration; do not publish its configured value. Referenced by config/database.php, setup/Detection Process.py, setup/Pending Blocker.py.

DB_PORT. Database connection, storage, or transport configuration. Referenced by config/database.php, setup/Detection Process.py, setup/Pending Blocker.py.

DB_QUEUE. Database connection, storage, or transport configuration. Referenced by config/queue.php.

DB_QUEUE_CONNECTION. Database connection, storage, or transport configuration. Referenced by config/queue.php.

DB_QUEUE_RETRY_AFTER. Database connection, storage, or transport configuration. Referenced by config/queue.php.

DB_QUEUE_TABLE. Database connection, storage, or transport configuration. Referenced by config/queue.php.

DB_SOCKET. Database connection, storage, or transport configuration. Referenced by config/database.php.

DB_TABLE. Observation table name for Python persistence and lookup. Referenced by setup/Detection Process.py, setup/Pending Blocker.py.

DB_TRUST_SERVER_CERTIFICATE. Database connection, storage, or transport configuration. Referenced by config/database.php.

DB_URL. Database connection, storage, or transport configuration. Referenced by config/database.php.

DB_USERNAME. Database connection, storage, or transport configuration. Referenced by config/database.php, setup/Detection Process.py, setup/Pending Blocker.py.

DYNAMODB_CACHE_TABLE. Framework or component setting; its exact consumers are the referenced source files. Referenced by config/cache.php.

DYNAMODB_ENDPOINT. Framework or component setting; its exact consumers are the referenced source files. Referenced by config/cache.php.

FILESYSTEM_DISK. Framework or component setting; its exact consumers are the referenced source files. Referenced by config/filesystems.php.

IDS_MODEL_FILENAME. Trusted model bundle filename under setup/data/model. Referenced by setup/Detection Process.py.

IDS_THRESHOLD. Optional override of the stored binary classification threshold. Referenced by setup/Detection Process.py.

INCIDENT_WINDOW_MINUTES. Eligibility window for reusing unresolved incidents; default fifteen minutes. Referenced by setup/Detection Process.py.

LOG_CHANNEL. Laravel logging channel, level, or transport configuration. Referenced by config/logging.php.

LOG_DAILY_DAYS. Laravel logging channel, level, or transport configuration. Referenced by config/logging.php.

LOG_DEPRECATIONS_CHANNEL. Laravel logging channel, level, or transport configuration. Referenced by config/logging.php.

LOG_DEPRECATIONS_TRACE. Laravel logging channel, level, or transport configuration. Referenced by config/logging.php.

LOG_DIR. Prediction log output directory for the detector. Referenced by setup/Detection Process.py.

LOG_LEVEL. Laravel logging channel, level, or transport configuration. Referenced by config/logging.php.

LOG_PAPERTRAIL_HANDLER. Laravel logging channel, level, or transport configuration. Referenced by config/logging.php.

LOG_SLACK_EMOJI. Laravel logging channel, level, or transport configuration. Referenced by config/logging.php.

LOG_SLACK_USERNAME. Laravel logging channel, level, or transport configuration. Referenced by config/logging.php.

LOG_SLACK_WEBHOOK_URL. Laravel logging channel, level, or transport configuration. Referenced by config/logging.php.

LOG_STACK. Laravel logging channel, level, or transport configuration. Referenced by config/logging.php.

LOG_STDERR_FORMATTER. Laravel logging channel, level, or transport configuration. Referenced by config/logging.php.

LOG_SYSLOG_FACILITY. Laravel logging channel, level, or transport configuration. Referenced by config/logging.php.

MAIL_EHLO_DOMAIN. Optional framework mail, cloud, or external service configuration; not proof that the service is used by the IDS path. Referenced by config/mail.php.

MAIL_FROM_ADDRESS. Optional framework mail, cloud, or external service configuration; not proof that the service is used by the IDS path. Referenced by config/mail.php.

MAIL_FROM_NAME. Optional framework mail, cloud, or external service configuration; not proof that the service is used by the IDS path. Referenced by config/mail.php.

MAIL_HOST. Optional framework mail, cloud, or external service configuration; not proof that the service is used by the IDS path. Referenced by config/mail.php.

MAIL_LOG_CHANNEL. Optional framework mail, cloud, or external service configuration; not proof that the service is used by the IDS path. Referenced by config/mail.php.

MAIL_MAILER. Optional framework mail, cloud, or external service configuration; not proof that the service is used by the IDS path. Referenced by config/mail.php.

MAIL_PASSWORD. Private access or cryptographic configuration; do not publish its configured value. Referenced by config/mail.php.

MAIL_PORT. Optional framework mail, cloud, or external service configuration; not proof that the service is used by the IDS path. Referenced by config/mail.php.

MAIL_SCHEME. Optional framework mail, cloud, or external service configuration; not proof that the service is used by the IDS path. Referenced by config/mail.php.

MAIL_SENDMAIL_PATH. Optional framework mail, cloud, or external service configuration; not proof that the service is used by the IDS path. Referenced by config/mail.php.

MAIL_URL. Optional framework mail, cloud, or external service configuration; not proof that the service is used by the IDS path. Referenced by config/mail.php.

MAIL_USERNAME. Optional framework mail, cloud, or external service configuration; not proof that the service is used by the IDS path. Referenced by config/mail.php.

MEMCACHED_HOST. Framework cache or backing-store configuration. Referenced by config/cache.php.

MEMCACHED_PASSWORD. Private access or cryptographic configuration; do not publish its configured value. Referenced by config/cache.php.

MEMCACHED_PERSISTENT_ID. Framework cache or backing-store configuration. Referenced by config/cache.php.

MEMCACHED_PORT. Framework cache or backing-store configuration. Referenced by config/cache.php.

MEMCACHED_USERNAME. Framework cache or backing-store configuration. Referenced by config/cache.php.

MIKROTIK_HOST. Router address and required NetFlow sender address. Referenced by setup/Mikrotek PPPoe Sniffer.py, setup/Pending Blocker.py.

MIKROTIK_PASSWORD. Private access or cryptographic configuration; do not publish its configured value. Referenced by setup/Mikrotek PPPoe Sniffer.py, setup/Pending Blocker.py.

MIKROTIK_PORT. RouterOS API port; default 8728 in the scripts. Referenced by setup/Mikrotek PPPoe Sniffer.py, setup/Pending Blocker.py.

MIKROTIK_USER. Framework or component setting; its exact consumers are the referenced source files. Referenced by setup/Mikrotek PPPoe Sniffer.py, setup/Pending Blocker.py.

MYSQL_ATTR_SSL_CA. Framework or component setting; its exact consumers are the referenced source files. Referenced by config/database.php.

PAPERTRAIL_PORT. Framework or component setting; its exact consumers are the referenced source files. Referenced by config/logging.php.

PAPERTRAIL_URL. Framework or component setting; its exact consumers are the referenced source files. Referenced by config/logging.php.

POSTMARK_API_KEY. Private access or cryptographic configuration; do not publish its configured value. Referenced by config/services.php.

POSTMARK_MESSAGE_STREAM_ID. Optional framework mail, cloud, or external service configuration; not proof that the service is used by the IDS path. Referenced by config/mail.php.

QUEUE_CONNECTION. Framework asynchronous queue configuration; separate from the Python polling service. Referenced by config/queue.php.

QUEUE_FAILED_DRIVER. Framework asynchronous queue configuration; separate from the Python polling service. Referenced by config/queue.php.

REDIS_BACKOFF_ALGORITHM. Framework cache or backing-store configuration. Referenced by config/database.php.

REDIS_BACKOFF_BASE. Framework cache or backing-store configuration. Referenced by config/database.php.

REDIS_BACKOFF_CAP. Framework cache or backing-store configuration. Referenced by config/database.php.

REDIS_CACHE_CONNECTION. Framework cache or backing-store configuration. Referenced by config/cache.php.

REDIS_CACHE_DB. Framework cache or backing-store configuration. Referenced by config/database.php.

REDIS_CACHE_LOCK_CONNECTION. Framework cache or backing-store configuration. Referenced by config/cache.php.

REDIS_CLIENT. Framework cache or backing-store configuration. Referenced by config/database.php.

REDIS_CLUSTER. Framework cache or backing-store configuration. Referenced by config/database.php.

REDIS_DB. Framework cache or backing-store configuration. Referenced by config/database.php.

REDIS_HOST. Framework cache or backing-store configuration. Referenced by config/database.php.

REDIS_MAX_RETRIES. Framework cache or backing-store configuration. Referenced by config/database.php.

REDIS_PASSWORD. Private access or cryptographic configuration; do not publish its configured value. Referenced by config/database.php.

REDIS_PERSISTENT. Framework cache or backing-store configuration. Referenced by config/database.php.

REDIS_PORT. Framework cache or backing-store configuration. Referenced by config/database.php.

REDIS_PREFIX. Framework cache or backing-store configuration. Referenced by config/database.php.

REDIS_QUEUE. Framework cache or backing-store configuration. Referenced by config/queue.php.

REDIS_QUEUE_CONNECTION. Framework cache or backing-store configuration. Referenced by config/queue.php.

REDIS_QUEUE_RETRY_AFTER. Framework cache or backing-store configuration. Referenced by config/queue.php.

REDIS_URL. Framework cache or backing-store configuration. Referenced by config/database.php.

REDIS_USERNAME. Framework cache or backing-store configuration. Referenced by config/database.php.

RESEND_API_KEY. Private access or cryptographic configuration; do not publish its configured value. Referenced by config/services.php.

SANCTUM_STATEFUL_DOMAINS. Framework or component setting; its exact consumers are the referenced source files. Referenced by config/sanctum.php.

SANCTUM_TOKEN_PREFIX. Private access or cryptographic configuration; do not publish its configured value. Referenced by config/sanctum.php.

SESSION_CONNECTION. Website session persistence, lifetime, or cookie policy configuration. Referenced by config/session.php.

SESSION_COOKIE. Website session persistence, lifetime, or cookie policy configuration. Referenced by config/session.php.

SESSION_DOMAIN. Website session persistence, lifetime, or cookie policy configuration. Referenced by config/session.php.

SESSION_DRIVER. Website session persistence, lifetime, or cookie policy configuration. Referenced by config/session.php.

SESSION_ENCRYPT. Website session persistence, lifetime, or cookie policy configuration. Referenced by config/session.php.

SESSION_EXPIRE_ON_CLOSE. Website session persistence, lifetime, or cookie policy configuration. Referenced by config/session.php.

SESSION_HTTP_ONLY. Website session persistence, lifetime, or cookie policy configuration. Referenced by config/session.php.

SESSION_LIFETIME. Website session persistence, lifetime, or cookie policy configuration. Referenced by config/session.php.

SESSION_PARTITIONED_COOKIE. Website session persistence, lifetime, or cookie policy configuration. Referenced by config/session.php.

SESSION_PATH. Website session persistence, lifetime, or cookie policy configuration. Referenced by config/session.php.

SESSION_SAME_SITE. Website session persistence, lifetime, or cookie policy configuration. Referenced by config/session.php.

SESSION_SECURE_COOKIE. Website session persistence, lifetime, or cookie policy configuration. Referenced by config/session.php.

SESSION_STORE. Website session persistence, lifetime, or cookie policy configuration. Referenced by config/session.php.

SESSION_TABLE. Website session persistence, lifetime, or cookie policy configuration. Referenced by config/session.php.

SLACK_BOT_USER_DEFAULT_CHANNEL. Optional framework mail, cloud, or external service configuration; not proof that the service is used by the IDS path. Referenced by config/services.php.

SLACK_BOT_USER_OAUTH_TOKEN. Private access or cryptographic configuration; do not publish its configured value. Referenced by config/services.php.

SQS_PREFIX. Framework asynchronous queue configuration; separate from the Python polling service. Referenced by config/queue.php.

SQS_QUEUE. Framework asynchronous queue configuration; separate from the Python polling service. Referenced by config/queue.php.

SQS_SUFFIX. Framework asynchronous queue configuration; separate from the Python polling service. Referenced by config/queue.php.

23. Notebook and archived API evidence register

Notebook contents are development evidence, not automatically executed runtime components. Code cells are summarized as prose; stored outputs are not used to certify live behavior. Empty notebooks and checkpoints are identified as such. The historical API ZIP is inspected as text without executing it or loading its binary artifact.

setup/Training/.ipynb_checkpoints/Check Attr-checkpoint.ipynb

The notebook contains 0 cells and 0 code cells.

No substantive code source is stored in this notebook snapshot.

setup/Training/.ipynb_checkpoints/Network Attack Simulation-checkpoint.ipynb

The notebook contains 3 cells and 3 code cells.

Defined helpers or classes: normalize_colname, clean_label, safe_numeric, IDSDecision, IDSInference, __init__, _risk_label, _prepare_X, predict_df, predict_one, batch_test_bot, plot_probability_distribution.

Contains an offline BOT-label prediction or probability inspection experiment, not an active network attack generator.

setup/Training/.ipynb_checkpoints/Training Model-checkpoint.ipynb

The notebook contains 1 cells and 1 code cells.

Defined helpers or classes: normalize_colname, clean_label, safe_numeric.

Contains historical watch and suspicious severity gates, which are not implemented as those categories in the current Flask detector.

setup/Training/artifacts/.ipynb_checkpoints/Untitled-checkpoint.ipynb

The notebook contains 0 cells and 0 code cells.

No substantive code source is stored in this notebook snapshot.

setup/Training/artifacts/Untitled.ipynb

The notebook contains 2 cells and 2 code cells.

Contains an offline BOT-label prediction or probability inspection experiment, not an active network attack generator.

setup/Training/Check Attr.ipynb

The notebook contains 1 cells and 1 code cells.

No substantive code source is stored in this notebook snapshot.

setup/Training/Network Attack Simulation.ipynb

The notebook contains 3 cells and 3 code cells.

Defined helpers or classes: normalize_colname, clean_label, safe_numeric.

Runs inference on CSV rows, retains predicted attacks, adds predicted family and optional family confidence, and exports a results CSV. The filename Network Attack Simulation does not establish actual packet generation.

setup/Training/Training Model.ipynb

The notebook contains 1 cells and 1 code cells.

Defined helpers or classes: step, spinner, spin, normalize_colname, clean_label, safe_numeric, load_all_csvs.

Historical training experiment using a stacking classifier with random forest, gradient boosting, optional XGBoost, and a logistic-regression meta-model. It is distinct from the maintained NetFlow trainer.

Contains a fixed prediction-threshold experiment at 0.80 and writes a predicted-attacks CSV; this is not the maintained artifact threshold.

The historical notebook reuses a preprocessor object across stage definitions; the maintained trainer deliberately clones preprocessing state and evaluates after artifact reload.

Archived API member inventory

ids_ensemble-api-main/.gitattributes: 62 bytes. Historical API source or dependency/distribution metadata.

ids_ensemble-api-main/README.md: 18 bytes. Historical API source or dependency/distribution metadata.

ids_ensemble-api-main/app.py: 4788 bytes. Historical API source or dependency/distribution metadata.

ids_ensemble-api-main/model/ids_ensemble.joblib: 134 bytes. Model placeholder, not loaded for documentation.

ids_ensemble-api-main/requirements.txt: 72 bytes. Historical API source or dependency/distribution metadata.

24. Historical research manuscripts: text extracts

The following extracts preserve text from the project-owned Word manuscripts found in setup/Training so the documentation includes the original rationale, literature discussion, methodology ideas, and evaluation claims as well as the current implementation. Their claims are historical manuscript claims and have not been independently verified. They can conflict with active source and checked-in metrics. The current-source chapters must be used to describe what this project revision actually implements. Private email addresses and credential assignments are omitted. Tables are flattened into sentences so this remains a text-only reference; embedded figures and images are not reproduced.

alpha_word.docx

Historical source manuscript: setup/Training/alpha_word.docx. This appendix preserves source text and does not endorse its numerical results or treat its architecture as the current deployment.

Two-Stage Ensemble Machine Learning for Network Intrusion Detection System

ABSTRACT

This paper presents a novel two-stage ensemble machine learning approach for network intrusion detection systems (IDS). The proposed method employs a calibrated stacking ensemble for binary classification (Stage A) followed by a Random Forest classifier for multi-class attack categorization (Stage B). Evaluated on the CIC-IDS2017 benchmark dataset, our approach achieves 99.92% binary classification accuracy and 99.83% multi-class accuracy across 14 attack types, significantly outperforming existing methods. The system demonstrates robust performance on network-layer attacks including DDoS, PortScan, and various DoS variants, providing an effective solution for modern network security challenges.

Keywords: intrusion detection, ensemble learning, machine learning, network security

INTRODUCTION

The rapid expansion of network infrastructures and the increasing sophistication of cyber threats have made intrusion detection systems (IDS) a critical component of organizational security frameworks (Ring et al., 2019). Traditional signature-based detection methods struggle to identify novel attacks and zero-day exploits, leading to the adoption of machine learning approaches for anomaly detection (Buczak & Guven, 2016). As noted by Sommer & Paxson (2010), machine learning for intrusion detection must operate outside the 'closed world' assumption, dealing with unpredictable and novel attack behaviors—a challenge our two-stage ensemble is designed to address.

Several benchmark datasets have been developed for evaluating IDS performance, including KDD CUP 99 (Tavallaee et al., 2009), UNSW-NB15 (Moustafa & Slay, 2015), and more recently, CIC-IDS2017 (Sharafaldin et al., 2018). The CIC-IDS2017 dataset addresses limitations of earlier datasets by providing realistic network traffic with diverse attack scenarios (Ring et al., 2019). However, effective utilization of these datasets requires sophisticated machine learning approaches that can handle class imbalance and complex attack patterns.

Ensemble learning methods have shown remarkable success in intrusion detection by combining multiple classifiers to improve overall performance (Kevric et al., 2017). Random Forests (Breiman, 2001) and Gradient Boosting (Friedman, 2001) have been particularly effective, with recent advancements in stacking ensembles (Wolpert, 1992) and XGBoost (Chen & Guestrin, 2016) demonstrating superior performance. However, many existing approaches suffer from poor probability calibration (Niculescu-Mizil & Caruana, 2005) and suboptimal threshold selection (Zadrozny & Elkan, 2002). Chandola et al. (2009) conducted a seminal survey on anomaly detection, framing many of the foundational challenges that our two-stage ensemble aims to overcome through calibrated probability estimation and threshold optimization.

This paper proposes a two-stage ensemble framework that addresses these limitations through calibrated probability estimation and optimized threshold tuning. Our contributions include: (1) a novel two-stage architecture separating binary detection from multi-class classification, (2) implementation of calibrated stacking ensembles for robust probability estimation, and (3) comprehensive evaluation on the CIC-IDS2017 dataset demonstrating state-of-the-art performance.

LITERATURE REVIEW

Intrusion Detection Datasets

The evolution of intrusion detection datasets has significantly influenced machine learning approaches in cybersecurity. Early datasets like KDD CUP 99 (Tavallaee et al., 2009) established foundational benchmarks but suffered from redundancy and unrealistic traffic patterns. The UNSW-NB15 dataset (Moustafa & Slay, 2015) addressed some limitations by incorporating modern

attack scenarios, while CIC-IDS2017 (Sharafaldin et al., 2018) provided more realistic network behavior and diverse attack profiles (Ring et al., 2019).

Ring et al. (2019) conducted a comprehensive survey of network-based intrusion detection datasets, highlighting the importance of realistic traffic generation and proper attack characterization. Their analysis emphasized that dataset quality directly impacts the practical applicability of developed IDS solutions. Khraisat et al. (2019) provided a comprehensive survey of intrusion detection systems, highlighting the challenges of imbalanced datasets and evolving attack patterns, which further underscores the need for robust ensemble methods like the one proposed in this work. Sperotto et al. (2010) provided an overview of IP flow-based intrusion detection, which informs our feature selection process focused on flow-level statistics.

Ensemble Learning in Intrusion Detection

Ensemble methods have gained prominence in intrusion detection due to their ability to improve accuracy and robustness. Breiman's (2001) Random Forests provided the foundation for many ensemble approaches, while Friedman's (2001) Gradient Boosting introduced sequential improvement of weak learners. Chen and Guestrin (2016) further advanced gradient boosting with XGBoost, incorporating regularization and efficient implementation.

Recent studies have explored stacking ensembles (Wolpert, 1992) for intrusion detection. Kevric et al. (2017) demonstrated that combining classifier approaches using tree algorithms can be effective for network intrusion detection. Similarly, Aljawarneh et al. (2018) proposed an ensemble-learning framework that improved IDS efficiency through classifier combination and feature selection. These approaches align with Wolpert's (1992) stacked generalization theory, which forms the theoretical basis for our ensemble design. Ahmed et al. (2016) surveyed network anomaly detection techniques and emphasized the growing role of machine learning in identifying subtle and sophisticated threats, reinforcing the relevance of our ensemble-based approach. Almseidin et al. (2017) evaluated multiple machine learning algorithms for IDS, demonstrating the superiority of ensemble methods, which supports our choice of a stacking-based architecture.

Probability Calibration and Threshold Optimization

Probability calibration is crucial for reliable intrusion detection, as noted by Niculescu-Mizil and Caruana (2005). Platt scaling (Platt, 1999) and isotonic regression (Zadrozny & Elkan, 2001) have been widely used to transform classifier scores into accurate probability estimates. Zadrozny and Elkan (2002) demonstrated that proper calibration significantly improves classification performance, particularly for imbalanced datasets common in intrusion detection.

Recent works by Mirsky et al. (2018) and Kasongo & Sun (2020) have incorporated calibration techniques in ensemble frameworks, showing improved detection rates. Our approach builds upon these foundations by integrating calibration directly into the stacking ensemble architecture.

RESEARCH METHODOLOGY

Dataset and Preprocessing

We utilized the CIC-IDS2017 dataset (Sharafaldin et al., 2018), which contains realistic network traffic with both benign and attack flows. The dataset includes 2.8 million samples across 80 features, capturing various attack scenarios over multiple days. Following the dataset characterization by Sharafaldin et al. (2018), we focused on network-layer attacks detectable through flow statistics. Haider et al. (2017) discussed the importance of realistic dataset generation using fuzzy qualitative modeling, which aligns with our use of the CIC-IDS2017 dataset to ensure real-world applicability.

Figure 1. Dataset distribution across traffic types for training and testing sets (CIC-IDS2017)

Data preprocessing involved several steps. First, we normalized column names and handled missing values by replacing them with zeros. Numerical features were standardized using StandardScaler, while non-numeric identifiers were excluded to prevent data leakage. The label column was processed using Unicode normalization to ensure consistency across different attack naming conventions.

Feature Selection

Based on the analysis by Garcia et al. (2014), we selected 78 numerical features representing network flow characteristics. Features related to pure identifiers (IP addresses, ports, timestamps) were excluded, while behavioral features like flow bytes per second, packets per second, and inter-arrival times were retained. This approach aligns with feature selection strategies discussed by Aljawarneh et al. (2018) for handling high-dimensional network data.

Two-Stage Architecture

Stage A: Binary Classification (Benign vs Attack)

We implemented a stacking ensemble combining Random Forests (Breiman, 2001), Gradient Boosting (Friedman, 2001), and XGBoost (Chen & Guestrin, 2016). The ensemble employs logistic regression as a meta-classifier, following the stacked generalization framework (Wolpert, 1992). To address probability calibration, we incorporated CalibratedClassifierCV with sigmoid calibration (Niculescu-Mizil & Caruana, 2005; Platt, 1999), which has shown effectiveness in cybersecurity applications (Zadrozny & Elkan, 2001).

The binary classifier is formulated as:

Stage B: Multi-class Classification (Attack Categorization)

For attack type classification, we employed a Random Forest classifier (Breiman, 2001) trained exclusively on attack samples. This separation prevents label leakage and ensures that the multi-class classifier focuses on distinguishing between different attack patterns rather than benign traffic.

Threshold Optimization

Experimental Setup

The dataset was split using stratified sampling with 70% for training and 30% for testing, ensuring proportional representation of all attack types in both sets. We implemented the system in Python using scikit-learn, with hyperparameters optimized for balanced performance and computational efficiency. The experiments were conducted with FAST_MODE enabled to facilitate rapid iteration while maintaining competitive performance.

RESULT AND DISCUSSION

Stage A: Binary Classification Performance

Our stacking ensemble achieved exceptional performance in distinguishing between benign and attack traffic. As shown in Table 1, the system attained 99.92% accuracy with balanced precision and recall across both classes. The ROC-AUC score of 0.99987 demonstrates near-perfect separability between benign and malicious traffic.

Figure 2. ROC curve for Stage A showing near-perfect class separability (AUC = 0.99987).

Table 1 Stage A Binary Classification Results

Figure 3. Stage A binary classification performance showing precision, recall, and F1-score for benign and attack classes.

Threshold optimization yielded an optimal operating point at 0.256, providing balanced performance between false positives and false negatives. This calibrated threshold significantly outperforms the default 0.5 threshold, particularly in recall for attack detection.

Figure 4. Precision–Recall curve for Stage A showing optimal threshold selection (0.256).

Stage B: Multi-class Attack Classification

The multi-class classifier demonstrated outstanding performance across 14 attack categories, achieving 99.83% overall accuracy. As detailed in Table 2, most attack types were classified with near-perfect accuracy, particularly high-volume attacks like DDoS (100%), DoS HULK (100%), and PortScan (100%).

Table 2 Stage B Multi-class Classification Performance (Selected Attacks)

Figure 5. Stage B multi-class attack classification accuracies across 14 attack types.

The confusion matrix revealed minimal misclassification between attack types, with most errors occurring in low-sample categories. This performance surpasses recently reported results in the literature, demonstrating the effectiveness of our two-stage approach.

Figure 6. Confusion matrix for Stage B showing classification performance across network-layer attacks.

Comparative Analysis

Our results significantly outperform existing approaches in the literature. As shown in Table 3, our method achieves superior performance compared to recent ensemble-based IDS proposals.

Table 3 Performance Comparison with Recent Works

The performance improvement can be attributed to several factors: (1) effective probability calibration addressing dataset imbalance (Niculescu-Mizil & Caruana, 2005), (2) optimal threshold tuning maximizing F1-score (Zadrozny & Elkan, 2002), and (3) the two-stage architecture preventing interference between binary and multi-class classification tasks.

Computational Efficiency

Despite the sophisticated ensemble architecture, our system maintains practical computational requirements. Stage A training required approximately 3.3 hours, while Stage B training completed in 43 seconds. Inference performance supports real-time deployment, with the capability to process thousands of network flows per second. This efficiency aligns with the requirements discussed by Shone et al. (2018) for practical intrusion detection systems.

Figure 7. Computational efficiency comparison between training and inference stages.

CONCLUSION

This paper presented a novel two-stage ensemble approach for network intrusion detection that achieves state-of-the-art performance on the CIC-IDS2017 dataset. Our method demonstrates that carefully designed ensemble architectures with proper calibration and threshold optimization can significantly improve intrusion detection accuracy.

The key contributions of this work include:

A two-stage classification framework that separates binary detection from multi-class attack categorization

Implementation of calibrated stacking ensembles for robust probability estimation

Comprehensive evaluation showing 99.92% binary accuracy and 99.83% multi-class accuracy

Demonstration that the approach outperforms recent ensemble-based IDS methods

Future work will focus on extending the approach to web application attack detection through integration of payload analysis features, adapting to evolving attack patterns through continuous learning, and optimizing for resource-constrained environments following approaches discussed by Javaid et al. (2016) and Yin et al. (2017). Wang & Lu (2013) discussed cybersecurity challenges in smart grids, illustrating the broader need for adaptable IDS solutions like our two-stage ensemble in critical infrastructure.

REFERENCES

Sharafaldin, I., Lashkari, A. H., & Ghorbani, A. A. (2018). Toward generating a new intrusion detection dataset and intrusion traffic characterization. Proceedings of the 4th International Conference on Information Systems Security and Privacy, 108–116. https://doi.org/10.5220/0006639801080116

Moustafa, N., & Slay, J. (2015). UNSW-NB15: A comprehensive data set for network intrusion detection systems. 2015 Military Communications and Information Systems Conference (MilCIS), 1–6. https://doi.org/10.1109/MilCIS.2015.7348942

Tavallaee, M., Bagheri, E., Lu, W., & Ghorbani, A. A. (2009). A detailed analysis of the KDD CUP 99 data set. 2009 IEEE Symposium on Computational Intelligence for Security and Defense Applications, 1–6. https://doi.org/10.1109/CISDA.2009.5356528

Breiman, L. (2001). Random forests. Machine Learning, 45(1), 5–32. https://doi.org/10.1023/A:1010933404324

Friedman, J. H. (2001). Greedy function approximation: A gradient boosting machine. The Annals of Statistics, 29(5), 1189–1232. https://doi.org/10.1214/aos/1013203451

Chen, T., & Guestrin, C. (2016). XGBoost: A scalable tree boosting system. Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, 785–794. https://doi.org/10.1145/2939672.2939785

Wolpert, D. H. (1992). Stacked generalization. Neural Networks, 5(2), 241–259. https://doi.org/10.1016/S0893-6080(05)80023-1

Niculescu-Mizil, A., & Caruana, R. (2005). Predicting good probabilities with supervised learning. Proceedings of the 22nd International Conference on Machine Learning, 625–632. https://doi.org/10.1145/1102351.1102430

Platt, J. (1999). Probabilistic outputs for support vector machines and comparisons to regularized likelihood methods. Advances in Large Margin Classifiers, 10(3), 61–74.

Zadrozny, B., & Elkan, C. (2002). Transforming classifier scores into accurate multiclass probability estimates. Proceedings of the 8th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, 694–699. https://doi.org/10.1145/775047.775151

Buczak, A. L., & Guven, E. (2016). A survey of data mining and machine learning methods for cyber security intrusion detection. IEEE Communications Surveys & Tutorials, 18(2), 1153–1176. https://doi.org/10.1109/COMST.2015.2494502

Ring, M., Wunderlich, S., Scheuring, D., Landes, D., & Hotho, A. (2019). A survey of network-based intrusion detection data sets. Computers & Security, 86, 147–167. https://doi.org/10.1016/j.cose.2019.06.005

Khraisat, A., Gondal, I., Vamplew, P., & Kamruzzaman, J. (2019). Survey of intrusion detection systems: techniques, datasets and challenges. Cybersecurity, 2(1), 1–22. https://doi.org/10.1186/s42400-019-0038-7

Zadrozny, B., & Elkan, C. (2001). Obtaining calibrated probability estimates from decision trees and naive Bayesian classifiers. Proceedings of the 18th International Conference on Machine Learning, 609–616.

Garcia, S., Zunino, A., & Campo, M. (2014). Survey on network-based botnet detection methods. Security and Communication Networks, 7(5), 878–903. https://doi.org/10.1002/sec.800

Aljawarneh, S., Aldwairi, M., & Yassein, M. B. (2018). Anomaly-based intrusion detection system through feature selection analysis and building hybrid efficient model. Journal of Computational Science, 25, 152–160. https://doi.org/10.1016/j.jocs.2017.03.006

Vinayakumar, R., Alazab, M., Soman, K. P., Poornachandran, P., Al-Nemrat, A., & Venkatraman, S. (2019). Deep learning approach for intelligent intrusion detection system. IEEE Access, 7, 41525–41550. https://doi.org/10.1109/ACCESS.2019.2895334

Kasongo, S. M., & Sun, Y. (2020). A deep learning method with wrapper based feature extraction for wireless intrusion detection system. Computers & Security, 92, 101752. https://doi.org/10.1016/j.cose.2020.101752

Sommer, R., & Paxson, V. (2010). Outside the closed world: On using machine learning for network intrusion detection. 2010 IEEE Symposium on Security and Privacy, 305–316. https://doi.org/10.1109/SP.2010.25

Yin, C., Zhu, Y., Fei, J., & He, X. (2017). A deep learning approach for intrusion detection using recurrent neural networks. IEEE Access, 5, 21954–21961. https://doi.org/10.1109/ACCESS.2017.2762418

Ahmed, M., Mahmood, A. N., & Hu, J. (2016). A survey of network anomaly detection techniques. Journal of Network and Computer Applications, 60, 19–31. https://doi.org/10.1016/j.jnca.2015.11.016

Mirsky, Y., Doitshman, T., Elovici, Y., & Shabtai, A. (2018). Kitsune: An ensemble of autoencoders for online network intrusion detection. Proceedings of the 2018 Network and Distributed System Security Symposium. https://doi.org/10.14722/ndss.2018.23204

Shone, N., Ngoc, T. N., Phai, V. D., & Shi, Q. (2018). A deep learning approach to network intrusion detection. IEEE Transactions on Emerging Topics in Computational Intelligence, 2(1), 41-50. https://doi.org/10.1109/TETCI.2017.2772792

Javaid, A., Niyaz, Q., Sun, W., & Alam, M. (2016). A deep learning approach for network intrusion detection system. *Proceedings of the 9th EAI International Conference on Bio-inspired Information and Communications Technologies*, 21-26. https://doi.org/10.4108/eai.3-12-2015.2262516

Haider, W., Hu, J., Slay, J., Turnbull, B. P., & Xie, Y. (2017). Generating realistic intrusion detection system dataset based on fuzzy qualitative modeling. Journal of Network and Computer Applications, 87, 185-192. https://doi.org/10.1016/j.jnca.2017.03.018

Sperotto, A., Schaffrath, G., Sadre, R., Morariu, C., Pras, A., & Stiller, B. (2010). An overview of IP flow-based intrusion detection. IEEE Communications Surveys & Tutorials, 12(3), 343-356. https://doi.org/10.1109/SURV.2010.032210.00054

Wang, W., & Lu, Z. (2013). Cyber security in the smart grid: Survey and challenges. Computer Networks, 57(5), 1344-1371. https://doi.org/10.1016/j.comnet.2012.12.017

Almseidin, M., Alzubi, M., Kovacs, S., & Alkasassbeh, M. (2017). Evaluation of machine learning algorithms for intrusion detection system. 2017 IEEE 15th International Symposium on Intelligent Systems and Informatics (SISY), 000277-000282. https://doi.org/10.1109/SISY.2017.8080566

Kevric, J., Jukic, S., & Subasi, A. (2017). An effective combining classifier approach using tree algorithms for network intrusion detection. Neural Computing and Applications, 28(1), 1051-1058. https://doi.org/10.1007/s00521-016-2418-1

Chandola, V., Banerjee, A., & Kumar, V. (2009). Anomaly detection: A survey. ACM Computing Surveys, 41(3), 1-58. https://doi.org/10.1145/1541880.1541882

Manuscript table 1, flattened text follows.

Metric ; Benign ; Attack ; Weighted Avg

Precision ; 99.97% ; 99.73% ; 99.92%

Recall ; 99.93% ; 99.86% ; 99.92%

F1-Score ; 99.95% ; 99.79% ; 99.92%

Manuscript table 2, flattened text follows.

Metric ; ; DDoS ; DoS HULK ; PortScan ; DoS GoldenEye ; FTP-Patator ; Overall

Precision ; ; 100.0% ; 99.96% ; 99.99% ; 99.61% ; 99.83%

Recall ; ; 100.0% ; 99.99% ; 99.97% ; 99.67% ; 99.83%

F1-Score ; ; 100.0% ; 99.97% ; 99.98% ; 99.64% ; 99.83%

Support ; ; 38,428 ; 69,409 ; 47,614 ; 3,063 ; 2,376 ; 167,294

Manuscript table 3, flattened text follows.

Study ; Approach ; Binary Accuracy ; Multi-class Accuracy

Our Work ; Two-stage Ensemble ; 99.92% ; 99.83%

Kevric et al. (2017) ; Combining Classifier Approach ; 99.20% ; 98.10%

Aljawarneh et al. (2018) ; Hybrid Efficient Model ; 99.50% ; 98.70%

Vinayakumar et al. (2019) ; Deep Learning Approach ; 99.10% ; 97.90%

Kasongo & Sun (2020) ; Deep Learning with Feature Extraction ; 99.30% ; 98.20%

beta.docx

Historical source manuscript: setup/Training/beta.docx. This appendix preserves source text and does not endorse its numerical results or treat its architecture as the current deployment.

Machine Learning–Based Intrusion Detection System using Stacking Ensemble Method

INTRODUCTION

Cybersecurity has become a critical issue in the digital age, as networks grow increasingly complex and vulnerable to attacks. Traditional signature-based intrusion detection systems (IDS) are limited because they cannot recognize new or evolving attack types. As a result, machine learning (ML)–based IDS have emerged as a promising solution for detecting anomalies and threats by learning traffic behavior patterns rather than relying solely on known signatures.

Modern algorithms such as Random Forest (Breiman, 2001), Gradient Boosting (Friedman, 2001), and XGBoost (Chen & Guestrin, 2016) are now widely applied to network intrusion detection. However, despite their strengths, single-model IDS still face challenges in achieving consistent accuracy across different attack categories and in maintaining generalization to unseen threats.

This study proposes an enhanced IDS architecture using ensemble learning—specifically a stacking model that combines multiple base classifiers (Random Forest, Gradient Boosting, and XGBoost) with a meta-learner (Logistic Regression). By leveraging the CICIDS2017 dataset (Sharafaldin et al., 2018), this research evaluates both binary (benign vs. attack) and multi-class (attack family) classification to determine how stacking ensembles improve detection accuracy and robustness.

STATEMENT OF THE PROBLEM

Machine learning models for IDS often exhibit high binary accuracy but fail when distinguishing between specific attack families. This issue is amplified by imbalanced datasets and temporal differences between training and testing data. The problems addressed by this study are:

How can stacking ensemble learning improve IDS accuracy and generalization compared with single classifiers?

What are the detection results of the ensemble model when distinguishing between benign and attack traffic?

Why does multi-class performance drop despite strong binary detection accuracy, and what factors cause it?

How can probability calibration and threshold optimization improve classification decisions?

OBJECTIVE OF THE STUDY

To develop a machine learning–based network intrusion detection system using stacking ensemble models for enhanced performance in detecting and classifying network attacks.

To preprocess and analyze the CICIDS2017 dataset for machine learning-based IDS development.

To design a stacking ensemble architecture integrating Random Forest, Gradient Boosting, and XGBoost classifiers.

To train and evaluate binary and multi-class models using accuracy, precision, recall, F1-score, and ROC-AUC metrics.

To assess the effect of probability calibration on improving decision thresholds.

To compare the ensemble's results against known single-model baselines in existing studies.

SIGNIFICANT OF THE STUDY

This research contributes to the continuous evolution of intrusion detection by showing how ensemble learning enhances classification reliability and detection precision. Stacking models allow the combination of multiple algorithms to capture both linear and nonlinear relationships in data, reducing bias and variance simultaneously (Wolpert, 1992).

The findings will benefit cybersecurity practitioners by offering a scalable model for real-time IDS deployment and aid academic researchers by providing a structured methodology for building robust, calibrated ensemble models. It also highlights the role of threshold tuning (Zadrozny & Elkan, 2002) and probabilistic calibration (Platt, 1999) in improving confidence scoring for IDS decisions.

SCOPE AND DELIMITATION

This study focuses on the use of supervised learning and ensemble methods for IDS development using the CICIDS2017 dataset. The dataset includes a wide range of network traffic representing benign and malicious behaviors. Only numeric flow-based features were analyzed; packet payloads and protocol headers were excluded.

The study is delimited by:

Dataset: CICIDS2017 only.

Algorithms: Random Forest, Gradient Boosting, and XGBoost combined via stacking.

Platform: Python (scikit-learn, XGBoost).

Evaluation: Offline performance testing; no live or hardware-integrated IDS deployment.

Deep learning and hybrid neural network methods are excluded, as the primary goal is to validate the efficacy of tree-based ensembles for classical IDS.

METHODOLOGY

A. Dataset Preparation

The dataset (Sharafaldin et al., 2018) consists of over 2.8 million instances from eight separate capture files. Each CSV file represents a unique network scenario. Data cleaning involved normalization of column names, removal of identifiers (IP addresses, timestamps), conversion of all features to numeric types, and imputation of missing values.

B. Labeling

Each record was labeled as either Benign or Attack (binary) and further assigned a detailed attack type for multi-class analysis. Unicode and inconsistent naming issues were corrected to standardize labels.

C. Model Design

A stacking ensemble was implemented using Random Forest (Breiman, 2001), Gradient Boosting (Friedman, 2001), and XGBoost (Chen & Guestrin, 2016) as base learners, and Logistic Regression as the meta-learner. Calibration (Niculescu-Mizil & Caruana, 2005) with the sigmoid method was applied to optimize probability outputs.

D. Training and Testing

A time-based split was used: the final capture ("Wednesday-workingHours") served as the test set. Stage A trained a binary classifier; Stage B trained a multi-class classifier. Model evaluation used accuracy, precision, recall, F1-score, and ROC-AUC metrics.

E. Implementation Tools

The implementation was coded in Python using scikit-learn, xgboost, and supporting libraries (pandas, numpy, joblib). Training was performed on a multi-core CPU environment.

RESULT AND DISCUSSION

A. Binary Classification (Stage A)

The ensemble achieved outstanding performance:

Accuracy: 96.48%

Precision: 0.945

Recall: 0.959

F1-score: 0.952

ROC-AUC: 0.972

These results confirm the capability of ensemble stacking to improve detection performance, surpassing typical single-model baselines such as Decision Trees or Logistic Regression (Ali et al., 2023; Alotaibi, 2023). The calibrated probability threshold of 0.053 optimized recall while maintaining high precision.

B. Multi-Class Classification (Stage B)

The multi-class model failed to generalize effectively because the test set contained unseen attack types. This outcome aligns with the findings of Buczak and Guven (2016) and Ring et al. (2019), who observed significant class imbalance and label mismatch issues in IDS datasets. The model correctly identified general attack behavior but failed to classify new attack subtypes—highlighting a dataset limitation rather than a model defect.

C. Interpretation and Comparison

Studies such as Mahfouz et al. (2022), Talukder et al. (2024), and Pramanick et al. (2025) reported similar results where ensemble methods achieved excellent binary detection but struggled in open-set conditions. The proposed model's strong ROC-AUC validates that ensemble probability calibration improves confidence scores, as previously discussed by Zadrozny and Elkan (2001, 2002).

D. Implications

These results emphasize the need for consistent attack label mapping (Lazzarini, 2023), integration of diverse datasets (Moustafa & Slay, 2015), and hierarchical classification approaches. The system's strength in binary detection demonstrates potential for real-time IDS integration once retrained with broader data coverage.

CONCLUSION AND RECOMMENDATION

This study developed and evaluated a stacking ensemble-based network intrusion detection model using the CICIDS2017 dataset. The system achieved 96.48% accuracy in binary detection, demonstrating that ensemble learning effectively improves detection precision and robustness. However, its poor performance in multi-class classification underscores the importance of dataset balance and representativeness.

Recommendations

Group related attack subtypes into unified families (e.g., all DoS variants → DDoS).

Expand dataset diversity by merging other benchmark datasets (e.g., UNSW-NB15, NSL-KDD).

Explore hybrid ensemble-deep learning models for enhanced feature extraction.

Incorporate open-set recognition techniques to handle unseen attacks.

Implement real-time experiments to evaluate response latency and deployment feasibility.

REFERENCES

 Sharafaldin, I., Lashkari, A. H., & Ghorbani, A. A. (2018). Toward generating a new intrusion detection dataset and intrusion traffic characterization. Proceedings of the 4th International Conference on Information Systems Security and Privacy (ICISSP), 108–116. https://doi.org/10.5220/0006639801080116

 Moustafa, N., & Slay, J. (2015). UNSW-NB15: A comprehensive data set for network intrusion detection systems. Military Communications and Information Systems Conference (MilCIS). https://doi.org/10.1109/MilCIS.2015.7348942

 Tavallaee, M., Bagheri, E., Lu, W., & Ghorbani, A. A. (2009). A detailed analysis of the KDD CUP 99 data set. Proceedings of 2009 IEEE International Conference on Computational Intelligence for Security and Defense Applications (CISDA). https://doi.org/10.1109/CISDA.2009.5356528

 Breiman, L. (2001). Random forests. Machine Learning, 45(1), 5–32. https://doi.org/10.1023/A:1010933404324

 Friedman, J. H. (2001). Greedy function approximation: A gradient boosting machine. The Annals of Statistics, 29(5), 1189–1232. https://doi.org/10.1214/aos/1013203451

 Chen, T., & Guestrin, C. (2016). XGBoost: A scalable tree boosting system. Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD), 785–794. https://doi.org/10.1145/2939672.2939785

 Wolpert, D. H. (1992). Stacked generalization. Neural Networks, 5(2), 241–259. https://doi.org/10.1016/S0893-6080(05)80023-1

 Niculescu-Mizil, A., & Caruana, R. (2005). Predicting good probabilities with supervised learning. Proceedings of the 22nd International Conference on Machine Learning (ICML). https://doi.org/10.1145/1102351.1102430

 Platt, J. C. (1999). Probabilistic outputs for support vector machines and comparisons to regularized likelihood methods. In A. J. Smola, P. Bartlett, B. Schölkopf, & D. Schuurmans (Eds.), Advances in Large Margin Classifiers (pp. 61–74). MIT Press. (Original work published 1999)

 Zadrozny, B., & Elkan, C. (2002). Transforming classifier scores into accurate multiclass probability estimates. Proceedings of the 8th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD), 694–699. https://doi.org/10.1145/775047.775151

 Buczak, A. L., & Guven, E. (2016). A survey of data mining and machine learning methods for cyber security intrusion detection. IEEE Communications Surveys & Tutorials, 18(2), 1153–1176. https://doi.org/10.1109/COMST.2015.2494502

 Ring, M., Wunderlich, S., Scheuring, D., Landes, D., & Hotho, A. (2019). A survey of network-based intrusion detection data sets. Computers & Security, 86, 147–167. https://doi.org/10.1016/j.cose.2019.06.005

 García, S., Grill, M., Stiborek, J., & Zunino, A. (2014). An empirical comparison of botnet detection methods. Computers & Security, 45, 100–123. https://doi.org/10.1016/j.cose.2014.06.008

 Zadrozny, B., & Elkan, C. (2001). Obtaining calibrated probability estimates from decision trees and naive Bayesian classifiers. Proceedings of the 18th International Conference on Machine Learning (ICML), 609–616.

 Lin, Z., Shi, Y., & Xue, Z. (2018). IDSGAN: Generative adversarial networks for attack generation against intrusion detection. arXiv Preprint. https://doi.org/10.48550/arXiv.1809.02077

 Ali, M., Haque, M.-u., Durad, M. H., Usman, A., Mohsin, S. M., & Mujlid, H. (2023). Effective network intrusion detection using stacking-based ensemble approach. Data & Knowledge Engineering. https://doi.org/10.1007/s10207-023-00718-7

 Alotaibi, Y. (2023). Ensemble-learning framework for intrusion detection to improve IDS efficiency. Applied Sciences. https://doi.org/10.3390/app13111195

 Lazzarini, R. (2023). A stacking ensemble of deep learning models for IoT intrusion detection. Pattern Recognition Letters. https://doi.org/10.1016/j.patrec.2023.12.005

 Akhtar, M. A., et al. (2023). Robust genetic machine learning ensemble model for intrusion detection. Scientific Reports. https://doi.org/10.1038/s41598-023-43816-1

 Tang, Y. (2021). Deep stacking network for intrusion detection. Sensors, 22(1), 25. https://doi.org/10.3390/s22010025

 Abdullah, A. (2025). Development of an intrusion detection system using an ensemble voting machine learning technique. ETASR (European Transactions on Electrical Systems Research), 15(3), 23917–23922. https://doi.org/10.48084/etasr.10764

 Alalwany, E. (2025). Stacking ensemble deep learning for real-time intrusion detection in IoMT networks. Sensors. https://doi.org/10.3390/s25030624

 Mahfouz, A. M., Abuhussein, A., Alsubaei, F. S., & Shiva, S. G. (2022). Toward a holistic, efficient, stacking ensemble intrusion detection system using a real cloud-based dataset. International Journal of Advanced Computer Science and Applications (IJACSA), 13(9). https://doi.org/10.14569/IJACSA.2022.01309110

 Sowmya, T. (2023). A comprehensive review of AI based intrusion detection systems. Science of Computer Programming. https://doi.org/10.1016/j.scico.2022.102963

 Talukder, M. A., Islam, M. M., Uddin, M. A., Hasan, K. F., Sharmin, S., & Alyami, S. A. (2024). Machine learning-based network intrusion detection for big and imbalanced data using oversampling, stacking feature embedding and feature extraction. Journal of Big Data. https://doi.org/10.48550/arXiv.2401.12262

 (2023). Ensemble-based approach for efficient intrusion detection system based on stacking ensemble learning. Intelligent Automation & Soft Computing, 37(2). https://doi.org/10.32604/iasc.2023.024515

 Pramanick, N., et al. (2025). Leveraging stacking machine learning models and feature selection for scalable intrusion detection. Scientific Reports. https://doi.org/10.1038/s41598-025-01052-9

 Yang, L., Shami, A., Stevens, G., & De Rusett, S. (2022). LCCDE: A decision-based ensemble framework for intrusion detection in the Internet of Vehicles. Proceedings of the IEEE Global Communications Conference (GLOBECOM). https://doi.org/10.1109/GLOBECOM48099.2022.10001280

 Bibers, I., Arreche, O., & Abdallah, M. (2024). A comprehensive comparative study of individual ML models and ensemble strategies for network intrusion detection systems. arXiv Preprint. https://doi.org/10.48550/arXiv.2410.15597

 Lin, Z. Z., Pike, T. D., Bailey, M. M., & Bastian, N. D. (2022). A hypergraph-based machine learning ensemble network intrusion detection system. arXiv Preprint. https://doi.org/10.48550/arXiv.2211.03933

Design and Development of a Real-Time Flow-Based Intrusion Detection System Using a Two-Stage Ensemble Model.docx

Historical source manuscript: setup/Training/Design and Development of a Real-Time Flow-Based Intrusion Detection System Using a Two-Stage Ensemble Model.docx. This appendix preserves source text and does not endorse its numerical results or treat its architecture as the current deployment.

Design and Development of a Real-Time Flow-Based Intrusion Detection and Security Management System Using a Two-Stage Ensemble Model

Jimson A. Olaybar1, Patrick D. Cerna2

1,2College of ICT and Engineering, State University of Northern Negros, Sagay City, Negros Occidental, Philippines

1FCSIT, Southern Leyte State University – Main Campus, Sogod, Southern Leyte, Philippines

[email address omitted]

Abstract— The increasing complexity of modern network environments has heightened the need for practical and scalable intrusion detection solutions. This study presents the development of a real-time, flow-based Intrusion Detection and Security Management System (IDSMS) that integrates live traffic monitoring, machine learning-based intrusion analysis, centralized data storage, and web-based visualization. The system captures network traffic from a MikroTik-based gateway using a passive flow sniffer deployed on a Kali Linux monitoring node. Packets are aggregated into bidirectional flows and transformed into statistical feature vectors, which are analyzed by a machine learning inference service through an application programming interface. Detection results, including risk level and attack classification, are stored in a centralized database and presented through an interactive dashboard. Development-stage testing using simulated attack traffic demonstrates the system's ability to capture, analyze, and visualize network threats in real time. The proposed system provides a practical foundation for deployment in shared-access networks, small enterprises, and IoT-enabled environments.

Keywords— Intrusion Detection System, Flow-Based Analysis, Machine Learning, Network Security, Real-Time Monitoring

INTRODUCTION

The rapid growth of networked systems, including enterprise networks, Internet cafés, public Wi-Fi services, and IoT-enabled environments, has significantly increased exposure to cyber threats. Modern attacks such as port scanning, brute-force login attempts, DNS amplification, and distributed denial-of-service (DDoS) attacks continue to evolve in scale and sophistication, making traditional signature-based intrusion detection systems (IDS) insufficient for real-time protection [1], [4].

Recent studies emphasize the effectiveness of flow-based intrusion detection, which analyzes statistical properties of network flows rather than packet payloads. This approach enables efficient monitoring of encrypted traffic while reducing computational overhead, making it suitable for real-time and resource-constrained environments [2], [3], [7]. Machine learning (ML) and deep learning techniques further enhance flow-based IDS by learning complex traffic patterns and improving anomaly detection accuracy [6], [14], [22].

Despite advances in detection models, many existing IDS studies focus primarily on offline datasets and lack full system integration, limiting their applicability in operational networks [4], [10]. Real-world deployment requires not only accurate models but also live traffic capture, scalable inference, centralized logging, and actionable security management interfaces. Furthermore, public-access networks such as pisonet or shared Wi-Fi services demand IDS solutions capable of identifying malicious client devices while preserving normal user traffic. Recent studies have demonstrated the feasibility of anomaly detection and traffic pattern analysis in real-world Pisonet environments using dynamic time warping and seasonal decomposition techniques [25], [26].

To address these challenges, this study presents the development and implementation of a real-time flow-based Intrusion Detection and Security Management System (IDSMS). The proposed system integrates live network traffic capture, flow-level feature extraction, machine learning-based attack classification, and a web-based security management platform. The system is designed to operate with a MikroTik gateway and downstream routers, enabling visibility into client-generated traffic while supporting real-time detection and response.

Unlike dataset-only approaches, this work emphasizes end-to-end system development, demonstrating how flow-based IDS concepts can be operationalized into a working security platform. The system architecture supports live monitoring, historical analysis, and threat management, providing a practical foundation for deployment in small enterprise networks, IoT environments, and public internet access services.

RELATED WORKS

Flow-based intrusion detection has become a widely adopted approach for modern network security due to its scalability, privacy-preserving nature, and compatibility with encrypted traffic. Unlike payload inspection, flow-based methods rely on statistical representations of communication behaviors, enabling real-time monitoring with reduced computational overhead. Satheesh et al. demonstrated early integration of flow-based anomaly detection with SDN/OpenFlow environments using machine learning, highlighting the feasibility of flow-driven security monitoring in programmable networks [1]. Similarly, Pontes et al. proposed a novel flow-based detection method using the inverse Potts model, showing that flow representations can capture complex network behaviors beyond traditional feature engineering [2]. For Wi-Fi environments, Segura-Garcia et al. introduced HeaderHunter, a real-time flow-based IDS for IEEE 802.11 networks, supporting the practicality of flow-level monitoring for wireless systems [3].

A major challenge in flow-based IDS research is the inconsistency of feature definitions and dataset variability across studies. Sarhan et al. emphasized the need for standardized feature sets, reporting that differences in feature extraction and labeling procedures can limit reproducibility and deployment readiness [4]. In parallel, dataset contributions have improved realism and diversity for IDS evaluation. Ferrag et al. introduced Edge-IIoTset, a comprehensive dataset reflecting realistic IoT and IIoT cyberattacks for centralized and federated learning scenarios [5]. Benchmarking studies using widely adopted datasets such as CICIDS2017 further demonstrate the importance of feature relevance and model robustness. Kurniabudi et al. analyzed CICIDS2017 features using information gain for anomaly detection, while Maseer et al. provided benchmarking of machine learning models for anomaly-based IDS, reinforcing the need for systematic model evaluation when using flow-style datasets [10], [11]. Komisarek and Pawlicki also introduced a refined dataset specifically oriented toward real-time NIDS settings, highlighting the continued effort to align IDS datasets with operational requirements [12].

Deep learning has increasingly been applied to flow-based intrusion detection due to its capability to model nonlinear relationships and hierarchical traffic patterns. Vishwakarma and Kesswani proposed a deep neural network-based real-time IDS for IoT, reporting improved detection performance in IoT contexts [6]. Additional studies have focused on flow-driven deep learning models for IoT anomaly detection, reflecting the shift toward learning-based approaches that operate effectively under constrained device environments [7], [8]. More recent developments include transformer-based architectures designed explicitly for flow representations. Manocchio et al. proposed FlowTransformer, demonstrating that attention-based models can effectively learn temporal and contextual dependencies within flow records [20]. These studies suggest that advanced deep learning can enhance detection accuracy; however, deploying such models in practical settings requires stable feature extraction pipelines and system-level integration.

Beyond centralized learning, federated learning (FL) has emerged as a promising technique for privacy-preserving intrusion detection across distributed networks. de Oliveira et al. presented F-NIDS, demonstrating a federated approach for intrusion detection while reducing the need to centralize sensitive data [13]. Doriguzzi-Corin and Siracusa proposed FLAD, which adapts federated learning for DDoS detection using adaptive mechanisms to address dynamic network behaviors [21]. Other works further explored combining FL with generative or representation learning methods. Nugraha et al. investigated federated learning with variational autoencoders for anomaly detection, indicating that hybrid FL approaches can improve generalization across heterogeneous sites [22]. Devine et al. also discussed federated ML for enabling IDS in IoT networks, while Buyuktanir et al. reviewed FL applications in IDS, outlining research trends and open challenges such as non-IID traffic distributions and communication overhead [23], [24].

Several studies have also proposed enhancement techniques to improve intrusion detection reliability, including data augmentation and improved feature encoding. Mohammad et al. explored deep learning combined with data augmentation to enhance IDS

performance, supporting the importance of improving training robustness and generalization [14]. Alitbi et al. proposed a generalized real-time IDS using incremental feature encoding and similarity embedding, addressing real-time constraints while improving discrimination capability [15]. These contributions reinforce the direction of building models and representations that are more suitable for live deployment and continuous monitoring.

Despite extensive research on flow-based IDS models, feature engineering, and learning strategies, many studies remain focused on algorithmic accuracy and offline experiments, with limited emphasis on full operational deployment. Practical deployment requires an integrated pipeline covering real-time flow capture, feature extraction consistency, scalable inference, centralized storage, and a management interface to support monitoring and response. The present work addresses this gap by developing an end-to-end Intrusion Detection and Security Management System (IDSMS) that connects live flow sniffing, machine learning inference via API, database logging, and a web-based dashboard for monitoring and threat management. By focusing on system development and implementation, this study complements model-centric IDS research and supports practical applicability in environments such as small enterprise networks and public-access Wi-Fi infrastructures. Recent work has further extended flow-based analysis to real-world scenarios, including Pisonet traffic analysis [25] and dynamic network traffic visualization using machine learning and seasonal decomposition [26].

METHODOLOGY

The detection models integrated into the system are based on the previously developed and validated two-stage ensemble intrusion detection framework.

System Architecture Overview

The proposed IDSMS follows a modular, flow-based architecture composed of five primary components: (1) live traffic capture, (2) flow aggregation and feature extraction, (3) machine learning-based intrusion analysis, (4) centralized data storage, and (5) web-based visualization and threat management. Figure 1 illustrates the complete system workflow, from packet capture to security decision-making.

Figure 1. Overall System Architecture Diagram

Network traffic originates from client devices connected to routers and passes through a MikroTik gateway. A dedicated monitoring node running Kali Linux captures and processes traffic flows without interfering with network operations.

Live Traffic Capture and Flow Aggregation

Traffic capture is performed using a Python-based flow sniffer deployed on the Kali Linux virtual machine. The sniffer passively listens on the designated network interface connected to the MikroTik router. Packets are aggregated into bidirectional flows using a five-tuple identifier consisting of source IP address, destination IP address, source port, destination port, and transport protocol.

Flow-based monitoring is selected due to its proven effectiveness in detecting network anomalies while remaining compatible with encrypted traffic [1], [3]. Idle timeout mechanisms are employed to determine flow termination, ensuring timely feature export for real-time analysis.

Feature Extraction and Preprocessing

For each completed flow, the system extracts a comprehensive set of approximately 78 statistical features, following best practices in flow-based IDS research [4], [7], [10]. These features capture temporal, volumetric, and directional characteristics of traffic, including flow duration, packet counts, byte rates, inter-arrival times, and forward–backward packet statistics.

Figure 2. Runtime Data Processing Pipeline of the Proposed IDSMS

Prior to analysis, flows are filtered to remove incomplete or insignificant traffic, such as flows with insufficient packet counts. Feature normalization is applied to ensure compatibility with the trained machine learning models and to prevent dominance of high-magnitude features during inference.

Machine Learning Model Design and Training

The intrusion detection engine integrated into the proposed IDSMS employs a two-stage ensemble machine learning architecture. Stage A performs binary classification to distinguish benign and malicious traffic using a calibrated stacking ensemble composed of Random Forest, Gradient Boosting, and XGBoost classifiers, with Logistic Regression as the meta-classifier. Stage B performs multi-class attack classification using a Random Forest classifier trained exclusively on malicious traffic samples.

The models were trained offline using the CIC-IDS2017 dataset, which contains realistic benign and attack traffic flows represented by 78 numerical flow-based features. Prior to training, feature normalization was applied using standard scaling, and identifier-based fields such as IP addresses and ports were removed to prevent information leakage.

Probability calibration and threshold optimization were applied in Stage A to improve detection reliability under class imbalance. The trained models were then deployed as an inference service and integrated into the proposed system through an API, enabling real-time classification of live network flows.

Machine Learning-Based Intrusion Analysis

Extracted flow features are transmitted to a remote machine learning inference service through a secure API. The detection engine employs supervised learning models trained on labeled flow-based intrusion datasets, enabling classification of traffic into benign or malicious categories.

The inference service outputs a risk level, attack probability, and predicted attack type, consistent with modern IDS frameworks that emphasize probabilistic decision-making rather than binary classification [6], [14], [22]. This decoupled design allows independent updating of detection models without modifying the traffic capture layer.

Data Storage and Logging Mechanism

All analyzed flows and prediction results are stored in a centralized database. Each record includes flow metadata, extracted statistical features, predicted risk classification, probability score, and timestamp. This persistent storage supports forensic analysis, trend evaluation, and system performance assessment.

Figure 3 presents the database schema used by the proposed IDSMS. Captured network flows are stored with their extracted statistical features and corresponding prediction outputs. The schema supports traceability between observed traffic, model decisions, and logged security events, enabling real-time dashboard updates and retrospective forensic analysis. The ER diagram represents the logical data model; implementation may store selected fields in a consolidated logging table to simplify real-time insertion and dashboard querying.

Figure 3. Database Schema of the Proposed IDSMS

Flow-level logging aligns with recent IDS research advocating standardized and structured data storage to improve transparency and reproducibility [4], [12].

Web-Based Visualization and Threat Management

A web-based Intrusion Detection and Security Management System (IDSMS) provides real-time and historical insights into network activity. The dashboard displays key metrics such as total network flows, detected attacks, attack rate, and risk-level distribution. Visual components include time-series graphs, attack distribution charts, and top attacked IP summaries.

Detected threats are presented with associated source IP addresses, destination IPs, protocol types, and attack classifications. Administrative controls enable operators to review, manage, and respond to threats, supporting practical deployment scenarios in public and enterprise networks.

Experimental Deployment Environment

The system is deployed in a virtualized environment using VMware. The Kali Linux monitoring node operates alongside the MikroTik gateway, routers, and client devices generating both benign and simulated attack traffic. This setup ensures that captured flows closely reflect real-world network behavior while allowing controlled experimentation.

Such deployment-focused evaluation addresses a key limitation identified in prior IDS studies, which often lack real-time operational validation [4], [23].

SYSTEM ARCHITECTURE AND DESIGN

Overall System Architecture

The proposed Intrusion Detection and Security Management System (IDSMS) is designed using a modular and layered architecture to support real-time network monitoring, scalable analysis, and centralized security management. The architecture separates traffic acquisition, intrusion analysis, data storage, and visualization into independent components, ensuring flexibility, maintainability, and ease of deployment.

The system operates within a gateway-based network environment where client devices connect through routers and a MikroTik gateway. Traffic generated by client devices is passively observed by a dedicated monitoring node, enabling intrusion detection without disrupting normal network operations.

Figure 4. Network Deployment Topology

Network Layer and Traffic Flow

At the network layer, client devices such as mobile phones, laptops, and public-access terminals generate traffic that traverses downstream routers before passing through the MikroTik gateway. This gateway acts as the central routing point and reflects typical configurations used in small enterprises, internet cafés, and pisonet Wi-Fi services.

The MikroTik router provides IP addressing, network address translation (NAT), and routing functions. Importantly, the IDS does not rely on MikroTik’s internal packet inspection; instead, it observes traffic externally to avoid performance degradation and configuration complexity on the gateway device.

Flow Monitoring and Capture Design

A Kali Linux-based monitoring node is connected to the same local network segment as the MikroTik gateway. This node runs a Python-based flow sniffer that captures packets passively from the designated network interface. The sniffer aggregates packets into bidirectional network flows using a five-tuple identifier composed of source IP address, destination IP address, source port, destination port, and transport protocol.

To ensure accurate detection of client-originated threats, the system supports configurable filtering rules that exclude infrastructure-related traffic, such as router management packets and monitoring host traffic. This design ensures that analysis focuses on user-generated traffic, which is critical in shared-access network environments.

Flow Representation and Feature Layer

Each completed flow is transformed into a structured feature vector representing statistical properties of the communication session. The system extracts approximately 78 flow-level features, including duration metrics, packet and byte counts, packet rates, inter-arrival times, and directional flow statistics.

This feature-based representation abstracts away packet payloads, enabling compatibility with encrypted traffic and reducing privacy concerns. The feature design aligns with established flow-based intrusion detection practices and supports consistent input to the machine learning inference layer.

Machine Learning Inference Integration

The intrusion analysis component is implemented as a remote machine learning inference service accessed through a secure API. Extracted flow features are transmitted from the monitoring node to the inference service, where trained models classify each flow as benign or malicious.

The inference service returns structured outputs including predicted risk level, probability of attack, and attack category. By decoupling traffic capture from model inference, the system allows independent updates to detection models without modifying the monitoring infrastructure. This architectural choice supports scalability and long-term system evolution.

Data Storage and Logging Architecture

All analyzed flows and prediction results are stored in a centralized relational database. Each database record includes flow metadata, extracted features, classification results, and timestamps. Centralized storage enables historical analysis, auditing, and system performance evaluation.

This logging architecture supports both real-time monitoring and retrospective investigation, addressing operational requirements for network administrators and security analysts.

Web-Based Security Management Interface

A web-based dashboard serves as the system’s visualization and management layer. The interface provides real-time summaries of network activity, including total flows processed, detected attacks, and traffic trends. Graphical elements such as charts and tables enable intuitive analysis of security events.

Administrators can inspect individual flow records, review detected threats, and analyze historical patterns. The dashboard is designed to support decision-making rather than automated enforcement during development, allowing administrators to validate detection behavior before enabling response actions.

Figure 5 illustrates the operational workflow of the security analyst or administrator when handling detected intrusion events. After an alert appears on the dashboard, the administrator reviews the event, inspects the corresponding flow details, and decides whether to block the source or ignore the alert. This workflow highlights the role of the proposed system not only as a detection platform but also as a security management and decision-support tool.

Figure 5. Security Analyst/Admin Decision-Making Workflow

IMPLEMENTATION

Development Environment and Tools

The proposed Intrusion Detection and Security Management System (IDSMS) was implemented using a combination of networking hardware, virtualization tools, and open-source software frameworks. A MikroTik router running RouterOS served as the network gateway, while the intrusion monitoring component was deployed on a Kali Linux virtual machine hosted in a VMware environment. This setup enabled controlled experimentation while closely emulating real-world deployment conditions.

Python was selected as the primary programming language for system development due to its extensive support for network analysis, machine learning integration, and rapid prototyping. Supporting tools included packet capture libraries, HTTP communication modules, and database connectors, facilitating seamless interaction between system components.

Flow Sniffer Implementation

The flow monitoring component was implemented as a Python-based sniffer executed through the command-line interface of the Kali Linux environment. Packet capture was performed on a designated network interface connected to the MikroTik gateway. The sniffer passively observes live traffic and aggregates packets into bidirectional flows using a five-tuple identifier consisting of source IP address, destination IP address, source port, destination port, and transport protocol.

To prevent interference from infrastructure traffic, configurable environment variables were used to exclude router and monitoring host IP addresses from analysis. Flow termination was determined using idle timeouts and minimum packet thresholds to ensure that only meaningful traffic sessions were exported for analysis.

Figure 6. Flow Capture and Feature Extraction Pipeline

Feature Extraction and Flow Processing

For each completed flow, the system computes a comprehensive set of statistical features that characterize the behavior of the communication session. These features include flow duration, packet counts, byte counts, packet rates, inter-arrival times, and directional statistics. Feature extraction is performed locally on the monitoring node to minimize data transmission overhead and ensure real-time performance.

Extracted features are formatted into structured data objects compatible with the machine learning inference service. Basic validation checks are applied to ensure feature completeness before transmission.

Machine Learning API Integration

The machine learning inference component was implemented as an external API service. The monitoring node sends extracted flow features to the API using secure HTTP requests. This service processes the input features using trained machine learning models and returns classification results.

Each response includes the predicted traffic label (benign or malicious), attack probability score, and identified attack type when applicable. This API-based integration allows the detection models to be updated or replaced independently from the flow capture logic, supporting extensibility and continuous improvement.

During development, network connectivity constraints were addressed by configuring appropriate routing and interface modes to ensure reliable communication between the monitoring node and the inference service.

Database and Data Management Implementation

A centralized relational database was used to store analyzed flows and prediction results. Each database entry includes flow identifiers, extracted statistical features, prediction outputs, and timestamps. The database schema was designed to support efficient querying for real-time dashboard updates and historical analysis.

Data persistence enables tracking of network behavior over time and supports post-analysis of detected threats. This storage layer plays a critical role in validating detection performance during development and testing.

Web-Based Dashboard Implementation

A web-based security dashboard was developed to provide visualization and management capabilities. The dashboard displays real-time statistics such as total flows processed, detected attacks, attack rates, and risk-level distributions. Interactive tables allow administrators to inspect individual flow records and review detection results.

Figure 7. Threat Reports module of the IDSMS showing summarized statistics and per-flow event records.

The interface was designed with development-stage validation in mind, emphasizing transparency and interpretability rather than automated enforcement. This approach allows system behavior to be closely monitored and verified before enabling active response mechanisms.

Testing and Development Configuration

Testing was conducted using a combination of normal network activities and simulated attack traffic. Generated traffic patterns were used to validate flow capture accuracy, feature extraction correctness, API communication, and database logging. These tests confirmed that the system components operate cohesively under development conditions.

The implementation supports future integration of automated mitigation actions, such as blocking malicious IP addresses through the MikroTik router, although such enforcement mechanisms were not enabled during the development phase.

RESULTS AND DISCUSSION

Overview of Development-Stage Results

This section presents and discusses the results obtained during the development and functional testing phase of the proposed Intrusion Detection and Security Management System (IDSMS). The evaluation focuses on validating system functionality, data flow correctness, and integration performance rather than benchmarking detection accuracy against large-scale real-world datasets.

The network traffic used in this phase consisted of a mixture of benign activities and intentionally generated attack traffic designed to test the system’s capability to capture, analyze, classify, and store flow-level information. The results demonstrate that the system operates as intended across all architectural layers, from live traffic capture to dashboard visualization.

Flow Capture and Traffic Visibility

During testing, the flow sniffer successfully captured live traffic traversing the MikroTik gateway and aggregated packets into bidirectional flows. Each flow was uniquely identified using source IP address, destination IP address, source port, destination port, and protocol.

The system consistently recorded flow metadata such as flow duration, packet count, and byte volume. The presence of multiple distinct source IP addresses in the captured data confirms that the system is capable of differentiating traffic generated by individual client devices. This behavior is essential for deployment in shared-access environments, such as pisonet Wi-Fi services, where multiple users generate concurrent traffic.

The successful extraction of flow-level statistics aligns with findings in flow-based IDS literature, which emphasize the reliability of flow representations for real-time monitoring and anomaly detection [1], [3].

Real-Time Monitoring Performance

The real-time monitoring capability of the proposed system is shown in Figure 8. The Live Monitor interface displays current incoming flows, packet rates, and alert counters, and updates continuously as traffic is observed from the network. The interface provides immediate visibility into ongoing network activity, allowing the administrator to track traffic fluctuations and verify that the monitoring and flow capture components are operating correctly. During testing, changes in traffic volume were instantly reflected in the live chart and counters, confirming that the system is capable of processing and visualizing network data in real time. This real-time feedback is essential for operational environments, as it enables rapid situational awareness and supports timely detection of abnormal traffic behavior.

Figure 8. Real-time monitoring interface of the proposed IDSMS showing live flow rate and packet activity.

Feature Generation and Statistical Metrics

For each completed flow, the system generated a structured feature vector consisting of approximately 78 statistical attributes. Key computed metrics included flow duration, packets per second, bytes per second, forward and backward packet counts, and inter-arrival time statistics.

The consistency of these metrics across repeated traffic patterns indicates that the feature extraction process is stable and deterministic. During simulated attack scenarios, abnormal traffic behaviors—such as unusually high packet rates or short inter-arrival times—were reflected in elevated flow-level metrics, demonstrating the sensitivity of the feature set to anomalous behavior.

This result supports prior research indicating that statistical flow features are effective descriptors for distinguishing benign and malicious traffic patterns [4], [10].

Machine Learning Inference Results

Extracted flow features were successfully transmitted to the machine learning inference API and processed without data loss. The inference engine returned prediction results including traffic classification (benign or malicious), probability scores, and identified attack types.

During development testing, most flows were classified as benign, while simulated attack traffic produced elevated probability scores and corresponding attack labels. Although these results do not represent real-world attack prevalence, they confirm that the inference pipeline correctly processes input features and returns interpretable outputs.

The probabilistic output format enables nuanced risk assessment rather than rigid binary decisions, which is consistent with modern IDS design principles [6], [14].

Detection Performance of the Integrated Machine Learning Engine

The integrated two-stage ensemble detection engine, which was previously trained and evaluated using the CIC-IDS2017 dataset, achieved strong performance. In the present system, the trained model was integrated and validated at the functional level using simulated attack traffic. Stage A (binary classification) achieved an accuracy of 99.92% with a ROC-AUC score of 0.99987, indicating near-perfect separation between benign and malicious traffic. Stage B (multi-class classification) achieved an overall accuracy of 99.83% across fourteen attack categories.

These results confirm that the machine learning component of the proposed IDSMS is not merely a conceptual module but a validated and high-performing detection engine suitable for real-time deployment. It should be noted that these quantitative performance results were obtained during offline training and evaluation of the detection models, while the present system-level experiments focus on validating real-time integration, data flow correctness, and operational functionality.

Table 1. Detection Performance of the Integrated Machine Learning Engine

Table 1 presents the performance of the two-stage ensemble machine learning engine integrated into the proposed IDSMS. Stage A, which performs binary classification between benign and malicious traffic, achieved an accuracy of 99.92% with a ROC-AUC score of 0.99987, indicating near-perfect discrimination capability. Stage B, which performs multi-class classification among fourteen attack types, achieved an overall accuracy of 99.83%, demonstrating that the system is not only effective at detecting malicious traffic but also reliable in identifying specific attack categories, making it suitable for real-time intrusion detection and security monitoring.

Database Logging and Data Integrity

All analyzed flows and corresponding prediction results were successfully stored in the centralized database. Each record included flow identifiers, extracted features, classification outputs, and timestamps.

Database inspection confirmed that data fields were populated correctly and consistently across multiple test runs. This persistent storage enables historical analysis and supports forensic review of detected events. The ability to store both benign and malicious flow records ensures comprehensive visibility into network behavior.

The logging mechanism adheres to best practices recommended for real-time IDS deployment, where traceability and auditability are critical [12].

Dashboard Metrics and Visualization Analysis

The web-based dashboard provided real-time visualization of network activity and detection outcomes. Metrics such as total flows processed, number of detected attacks, and attack rate were displayed using graphical components.

Figure 9. Dashboard Screenshot (Overview Page)

Time-series graphs illustrated fluctuations in traffic volume and attack occurrences, while tabular views enabled detailed inspection of individual flow records. During simulated attack periods, the dashboard reflected corresponding increases in detected attack counts, confirming synchronization between the monitoring, analysis, and visualization layers

Figure 10: Sample table of Detected Testing Attacks

Because the attack traffic was artificially generated for testing, the displayed attack statistics should not be interpreted as indicative of real-world network conditions. Instead, they demonstrate the system’s ability to surface and contextualize security events during development.

Discussion of System Effectiveness

The results confirm that the proposed IDSMS effectively integrates flow-based traffic monitoring, machine learning-based analysis, and centralized security management. The system demonstrates reliable end-to-end operation, from packet capture through prediction and visualization.

Compared to prior studies that focus primarily on offline datasets and isolated detection models [4], [10], this work emphasizes operational deployment and system integration. The development results indicate that the architecture is suitable for real-world environments, where continuous traffic monitoring and actionable insights are required.

However, the current evaluation is limited to development-scale testing with simulated attacks and a constrained number of client devices. Large-scale deployment with diverse traffic patterns and genuine attack scenarios is identified as a necessary next step to fully assess detection accuracy and scalability.

CONCLUSION AND FUTURE WORK

Conclusion

This study presented the development and implementation of a real-time, flow-based Intrusion Detection and Security Management System (IDSMS) that integrates live network traffic monitoring, machine learning-based intrusion analysis, centralized data storage, and web-based visualization. Unlike many existing IDS studies that focus primarily on offline datasets and model accuracy, this work emphasized system-level design and operational deployment, demonstrating how flow-based intrusion detection can be implemented in a practical network environment.

The system successfully captured live traffic from a MikroTik-based network, aggregated packets into bidirectional flows, extracted statistical features, and transmitted these features to a machine learning inference service for classification. The results were persistently stored in a database and presented through an interactive dashboard, enabling real-time monitoring and historical analysis. Development-stage testing using simulated attack traffic confirmed that the system components function cohesively and that intrusion events are correctly detected, logged, and visualized.

Overall, the findings validate the feasibility of deploying a modular, flow-based IDS capable of supporting shared-access network environments such as pisonet Wi-Fi services, small enterprises, and IoT-enabled infrastructures. The proposed system provides a solid foundation for real-world intrusion detection by combining research-driven methodologies with practical engineering implementation.

Limitations

Although the system demonstrates correct functionality and integration, the evaluation was conducted under development-stage conditions using simulated attack traffic and a limited number of client devices. As such, the results do not fully represent large-scale operational environments with diverse traffic patterns and real-world attack behaviors. Additionally, automated response mechanisms, such as real-time blocking or quarantine of malicious devices, were intentionally disabled during development to prioritize validation and system stability.

Future Work

Future research and development will focus on extending the system’s capabilities and evaluating its performance under real-world conditions. This includes deployment in shared-access environments similar to those studied in recent Pisonet traffic analyses [25], [26], to validate detection robustness under diverse and dynamic user behaviors. Planned enhancements include large-scale deployment involving multiple client devices and prolonged monitoring periods to assess scalability, detection accuracy, and system robustness. Integrating automated mitigation mechanisms, such as dynamic firewall rule enforcement through the MikroTik router, will enable real-time response to detected threats.

Further work will also explore adaptive and federated learning approaches to continuously improve detection models while preserving data privacy. Expanding the system to support additional network environments, including cloud-based and distributed infrastructures, represents another promising direction. Finally, comprehensive benchmarking against established IDS datasets and comparative evaluation with existing systems will be conducted to quantify performance gains and validate detection effectiveness.

REFERENCES

[1]. Satheesh, N., Rathnamma, M. V., Rajeshkumar, G., et al. (2020). Flow-based anomaly intrusion detection using machine learning model with software defined networking for OpenFlow network. Microprocessors and Microsystems, 79, 103285. https://doi.org/10.1016/j.micpro.2020.103285 ScienceDirect

[2]. Pontes, C. F. T., de Souza, M. M. C., Gondim, J. J. C., Bishop, M., & Marotta, M. A. (2021). A New Method for Flow-Based Network Intrusion Detection Using the Inverse Potts Model. IEEE Transactions on Network and Service Management, 18(2), 1125–1136. https://doi.org/10.1109/TNSM.2021.3075503 ResearchGate+1

[3]. Segura-Garcia, J., Garcia-Pineda, M., & Felici-Castell, S. (2021). HeaderHunter: A flow-based, real-time intrusion detection system for IEEE 802.11 networks. Sensors, 21(4), 1140. https://doi.org/10.3390/s21041140 ScienceDirect

[4]. Sarhan, M., Layeghy, S., & Portmann, M. (2022). Towards a Standard Feature Set for Network Intrusion Detection System Datasets. Mobile Networks and Applications, 27(1), 357–370. https://doi.org/10.1007/s11036-021-01843-0 ACM Digital Library+1

[5]. Ferrag, M. A., Friha, O., Hamouda, D., Maglaras, L., & Janicke, H. (2022). Edge-IIoTset: A New Comprehensive Realistic Cyber Security Dataset of IoT and IIoT Applications for Centralized and Federated Learning. IEEE Access. https://doi.org/10.1109/ACCESS.2022.3165809 semanticscholar.org+1

[6]. Vishwakarma, M., & Kesswani, N. (2022). DIDS: A deep neural network based real-time intrusion detection system for IoT. Decision Analytics Journal, 5, 100142. https://doi.org/10.1016/j.dajour.2022.100142 ScienceDirect+1

[7]. Rathore, H., et al. (2023). An anomaly-based IDS for IoT using NetFlow features and deep learning. Neural Computing and Applications. https://doi.org/10.1007/s00521-023-08376-5 Springer

[8]. Fraihat, S., et al. (2023). Intelligent intrusion detection in IoT networks using NetFlow-based traffic and ML (IoT-focused). Internet of Things. https://doi.org/10.1016/j.iot.2023.100819

[9]. Jablaoui, R., Cheikhrouhou, O., Hamdi, M., & Liouane, N. (2025). Deep learning enabled intrusion detection system for IoT security. EURASIP Journal on Wireless Communications and Networking, 2025(1). https://doi.org/10.1186/s13638-025-02477-6

[10]. Kurniabudi, Stiawan, D., Darmawijoyo, Idris, M. Y., & Bamhdi, A. M. (2020). CICIDS-2017 Dataset Feature Analysis with Information Gain for Anomaly Detection. IEEE Access, 8, 132911–132921. https://doi.org/10.1109/ACCESS.2020.3009843 Scinapse

[11]. Maseer, Z. K., Yusof, R., Bahaman, N., Mostafa, S. A., & Foozy, C. F. M. (2021). Benchmarking of Machine Learning for Anomaly Based Intrusion Detection Systems in the CICIDS2017 Dataset. IEEE Access, 9, 22351–22370. https://doi.org/10.1109/ACCESS.2021.3056614 semanticscholar.org

[12]. Komisarek, M., & Pawlicki, M. (2022). A novel, refined dataset for real-time Network Intrusion Detection. In Proceedings of the 17th International Conference on Availability, Reliability and Security (ARES 2022). https://doi.org/10.1145/3538969.3544486 ACM Digital Library

[13]. de Oliveira, J. A., Gonçalves, V. P., Meneguette, R. I., de Sousa Jr, R. T., Guidoni, D. L., Oliveira, J. C. M., & Rocha Filho, G. P. (2023). F-NIDS — A Network Intrusion Detection System based on federated learning. Computer Networks, 236, 110010. https://doi.org/10.1016/j.comnet.2023.110010

[14]. Mohammad, R., Saeed, F., Almazroi, A. A., Alsubaei, F. S., & Almazroi, A. A. (2024). Enhancing Intrusion Detection Systems Using a Deep Learning and Data Augmentation Approach. Systems, 12(3), 79. https://doi.org/10.3390/systems12030079 MDPI

[15]. Alitbi, Z. K., Hosseini Seno, S. A., Ghaemi Bafghi, A., & Zabihzadeh, D. (2025). A Generalized and Real-Time Network Intrusion Detection System Through Incremental Feature Encoding and Similarity Embedding Learning. Sensors, 25(16), 4961. https://doi.org/10.3390/s25164961

[16]. Mutambik, I. (2024). An efficient flow-based anomaly detection system for enhanced security in IoT networks. Sensors, 24(22), 7408. https://doi.org/10.3390/s24227408 PubMed

[17]. Callegari, C., Giordano, S., & Pagano, M. (2024). A real-time deep learning-based approach for detecting network attacks. Big Data Research, 36, 100446. https://doi.org/10.1016/j.bdr.2024.100446 ResearchGate

[18]. Gheni, H. Q., & Al-Yaseen, W. L. (2024). Two-step data clustering for improved intrusion detection system using CICIoT2023 dataset. e-Prime – Advances in Electrical Engineering, Electronics and Energy, 9, 100673. https://doi.org/10.1016/j.prime.2024.100673 ScienceDirect

[19]. Sayed, N., Baskar, S., Ramachandran, M., et al. (2023). Augmenting IoT intrusion detection system performance using deep neural network. CMC—Computers, Materials & Continua, 74(1). https://doi.org/10.32604/cmc.2023.030831 techscience.com

[20]. Manocchio, L. D., Layeghy, S., Lo, W. W., et al. (2023). FlowTransformer: A transformer framework for flow-based network intrusion detection systems. Expert Systems with Applications, 241, 122564. https://doi.org/10.1016/j.eswa.2023.122564 ResearchGate

[21]. Doriguzzi-Corin, R., & Siracusa, D. (2023). FLAD: Adaptive federated learning for DDoS attack detection. Computers & Security, 137, 103597. https://doi.org/10.1016/j.cose.2023.103597 ResearchGate

[22]. Nugraha, K. D., Kota, V. R., & Rupp, M. (2024). Leveraging federated learning and variational autoencoders for an enhanced anomaly detection system. IEEE NetSoft 2024. https://doi.org/10.1109/NetSoft60951.2024.10588934 ResearchGate

[23]. Devine, M., Ardakani, S. P., Al-Khafajiy, M., & James, Y. (2025). Federated machine learning to enable intrusion detection systems in IoT networks. Electronics, 14(6), 1176. https://doi.org/10.3390/electronics14061176 MDPI

[24]. Buyuktanir, O., Nandi, D., & Goyal, N. (2025). A review of federated learning applications in intrusion detection systems. Cluster Computing. https://doi.org/10.1007/s10586-025-05325-w

[25] Saligue, R. Y., & Saligue, E. T. (2025). Real-World Traffic Analysis in Pisonet using DTW and Anomaly Detection. In 2025 7th International Conference on Innovative Data Communication Technologies and Application (ICIDCA). IEEE. https://doi.org/10.1109/ICIDCA66325.2025.11280496

[26] Saligue, R. Y., Saligue, E. T., Teofilo, L. M., Sy, K. N. L., & Cerna, P. D. (2025). Dynamic Analysis and Visualization of Network Traffic Patterns using Machine Learning and Seasonal Decomposition Techniques. In 2025 5th International Conference on Soft Computing for Security Applications (ICSCSA). IEEE. https://doi.org/10.1109/ICSCSA66339.2025.11170757

Manuscript table 1, flattened text follows.

Stage ; Task ; Accuracy ; ROC-AUC

Stage A ; Binary (Benign vs Attack) ; 99.92% ; 0.99987

Stage B ; Multi-class (14 attack types) ; 99.83% ;

paper 2 development sir jimson.docx

Historical source manuscript: setup/Training/paper 2 development sir jimson.docx. This appendix preserves source text and does not endorse its numerical results or treat its architecture as the current deployment.

Design and Development of a Real-Time Flow-Based Intrusion Detection and Security Management System Using a Two-Stage Ensemble Model

Jimson A. Olaybar1, Patrick D. Cerna2

1,2College of ICT and Engineering, State University of Northern Negros, Sagay City, Negros Occidental, Philippines

1FCSIT, Southern Leyte State University – Main Campus, Sogod, Southern Leyte, Philippines

[email address omitted]

ABSTRACT

The increasing complexity of modern network environments has heightened the need for practical and scalable intrusion detection solutions. This study presents the development of a real-time, flow-based Intrusion Detection and Security Management System (IDSMS) that integrates live traffic monitoring, machine learning-based intrusion analysis, centralized data storage, and web-based visualization. The system captures network traffic from a MikroTik-based gateway using a passive flow sniffer deployed on a Kali Linux monitoring node. Packets are aggregated into bidirectional flows and transformed into statistical feature vectors, which are analyzed by a machine learning inference service through an application programming interface. Detection results, including risk

level and attack classification, are stored in a centralized database and presented through an interactive dashboard. Development-stage testing using simulated attack traffic demonstrates the system's ability to capture, analyze, and visualize network threats in real time. The proposed system provides a practical foundation for deployment in shared-access networks, small enterprises, and IoT-enabled environments.

Keywords — Intrusion Detection System, Flow-Based Analysis, Machine Learning, Network Security, Real-Time Monitoring

1. INTRODUCTION

The rapid growth of networked systems, including enterprise networks, Internet cafés, public Wi-Fi services, and IoT-enabled environments, has significantly increased exposure to cyber threats. Modern attacks such as port scanning, brute-force login attempts, DNS amplification, and distributed denial-of-service (DDoS) attacks continue to evolve in scale and sophistication, making traditional signature-based intrusion detection systems (IDS) insufficient for real-time protection [1], [4].

Recent studies emphasize the effectiveness of flow-based intrusion detection, which analyzes statistical properties of network flows rather than packet payloads. This approach enables efficient monitoring of encrypted traffic while reducing computational overhead, making it suitable for real-time and resource-constrained environments [2], [3], [7]. Machine learning (ML) and deep learning techniques further enhance flow-based IDS by learning complex traffic patterns and improving anomaly detection accuracy [6], [14], [22].

Despite advances in detection models, many existing IDS studies focus primarily on offline datasets and lack full system integration, limiting their applicability in operational networks [4], [10]. Real-world deployment requires not only accurate models but also live traffic capture, scalable inference, centralized logging, and actionable security management interfaces. Furthermore, public-access networks such as pisonet or shared Wi-Fi services demand IDS solutions capable of identifying malicious client devices while preserving normal user traffic. Recent studies have demonstrated the feasibility of anomaly detection and traffic pattern analysis in real-world Pisonet environments using dynamic time warping and seasonal decomposition techniques [25], [26].

To address these challenges, this study presents the development and implementation of a real-time flow-based Intrusion Detection and Security Management System (IDSMS). The proposed system integrates live network traffic capture, flow-level feature extraction, machine learning-based attack classification, and a web-based security management platform. The system is designed to operate with a MikroTik gateway and downstream routers, enabling visibility into client-generated traffic while supporting real-time detection and response.

Unlike dataset-only approaches, this work emphasizes end-to-end system development, demonstrating how flow-based IDS concepts can be operationalized into a working security platform. The system architecture supports live monitoring, historical analysis, and threat management, providing a practical foundation for deployment in small enterprise networks, IoT environments, and public internet access services.

2. OBJECTIVES OF THE STUDY

This study aims to design, develop, and evaluate a comprehensive flow-based intrusion detection system with integrated security management capabilities. Specifically, the objectives are:

To design and implement a real-time flow-based network monitoring system capable of capturing live traffic from a MikroTik gateway environment without disrupting normal network operations.

To develop a machine learning-based intrusion detection engine that utilizes a two-stage ensemble approach for binary classification (benign vs. malicious traffic) and multi-class attack classification.

To create an integrated security management platform featuring centralized data storage, web-based visualization, and real-time threat monitoring capabilities.

To validate the system functionality through development-stage testing using simulated attack traffic and evaluate its operational feasibility for deployment in shared-access network environments.

3. SCOPE OF THE STUDY

This research focuses on the following scope:

Geographical and Institutional Scope: The study is conducted within the network infrastructure of the State University of Northern Negros, Philippines, with applicability to similar educational institutions and small-to-medium enterprise environments.

Technical Scope:

Network environment: MikroTik router-based gateway with downstream routers

Monitoring platform: Kali Linux virtual machine

Detection approach: Flow-based analysis using statistical features

Machine learning framework: Two-stage ensemble model

System integration: Python-based implementation with web dashboard

Temporal Scope: The development and testing phase was conducted during the 2024-2025 academic year, with system validation performed under controlled laboratory conditions.

Functional Scope: The system includes live traffic capture, flow aggregation, feature extraction, machine learning inference, database logging, and web-based visualization components. Automated mitigation actions are excluded from the current implementation.

Limitation Scope: Evaluation is limited to development-stage testing with simulated attack traffic; large-scale real-world deployment and performance benchmarking are designated as future work.

4. SIGNIFICANCE OF THE STUDY

This research holds significant value for multiple stakeholders in the cybersecurity and network management domains:

4.1 For Educational Institutions

The system provides a cost-effective intrusion detection solution for universities and colleges with limited cybersecurity budgets, particularly relevant for Philippine higher education institutions facing resource constraints.

4.2 For Network Administrators

The integrated security management platform offers real-time visibility into network threats, enabling proactive security monitoring and informed decision-making for network administrators in small-to-medium enterprises.

4.3 For Cybersecurity Researchers

The study contributes to the body of knowledge on practical implementation of flow-based IDS, addressing the research gap between theoretical models and operational deployment. The modular architecture serves as a reference for future system development.

4.4 For Public Internet Service Providers

The system's design accommodates shared-access environments like pisonet and public Wi-Fi services, providing a framework for monitoring client-originated threats while maintaining service availability.

4.5 For IoT Network Operators

The flow-based approach's compatibility with encrypted traffic makes the system applicable to IoT environments where payload inspection is impractical or impossible.

4.6 For Developing Economies

The use of open-source tools and modular design provides a blueprint for affordable cybersecurity solutions in resource-constrained settings, contributing to digital security in developing regions.

5. RELATED STUDIES

5.1 Flow-Based Intrusion Detection Foundations

Flow-based intrusion detection has emerged as a scalable alternative to payload inspection, particularly for monitoring encrypted traffic. Satheesh et al. [1] demonstrated early integration of flow-based anomaly detection with SDN/OpenFlow environments, establishing the feasibility of flow-driven security monitoring in programmable networks. Pontes et al. [2] introduced a novel detection method using the inverse Potts model, showing that flow representations can capture complex network behaviors beyond traditional feature engineering. For wireless environments, Segura-Garcia et al. [3] developed HeaderHunter, a real-time flow-based IDS for IEEE 802.11 networks, validating the practicality of flow-level monitoring in wireless systems.

5.2 Standardization and Dataset Development

A persistent challenge in flow-based IDS research is feature definition inconsistency and dataset variability. Sarhan et al. [4] emphasized the need for standardized feature sets, noting that differences in extraction and labeling procedures limit reproducibility. Several datasets have been developed to address these issues, including Edge-IIoTset by Ferrag et al. [5] for IoT/IIoT scenarios and the widely adopted CICIDS2017 dataset used in benchmarking studies by Kurniabudi et al. [10] and Maseer et al. [11]. Komisarek and Pawlicki [12] further contributed a refined dataset specifically designed for real-time NIDS settings.

5.3 Machine Learning Integration in IDS

Deep learning approaches have gained prominence for their ability to model complex traffic patterns. Vishwakarma and Kesswani [6] proposed a deep neural network-based real-time IDS for IoT, reporting improved detection performance in constrained environments. Rathore et al. [7] and Fraihat et al. [8] explored flow-driven deep learning models for IoT anomaly detection, while Manocchio et al. [20] introduced FlowTransformer, an attention-based model for learning temporal dependencies within flow records.

5.4 Federated Learning for Distributed Detection

Federated learning has emerged as a privacy-preserving approach for distributed intrusion detection. de Oliveira et al. [13] presented F-NIDS, demonstrating federated intrusion detection without centralized data collection. Doriguzzi-Corin and Siracusa [21] proposed FLAD for adaptive DDoS detection, while Nugraha et al. [22] investigated federated learning with variational autoencoders for anomaly detection.

5.5 Enhancement Techniques and Practical Applications

Several studies have focused on improving IDS reliability through techniques like data augmentation and feature encoding. Mohammad et al. [14] explored deep learning combined with data augmentation, while Alitbi et al. [15] proposed incremental feature encoding for real-time constraints. Recent work has extended flow-based analysis to practical scenarios, including Pisonet traffic analysis [25] and dynamic network visualization [26], highlighting the growing relevance of operational deployment considerations.

5.6 Research Gap Addressed by This Study

While extensive research exists on flow-based IDS models and algorithms, few studies address comprehensive system integration and operational deployment. Most existing work focuses on offline evaluation using datasets, with limited emphasis on end-to-end system implementation. This study addresses this gap by developing a complete Intrusion Detection and Security Management System (IDSMS) that integrates live traffic capture, machine learning inference, centralized storage, and web-based management

interfaces.

6. METHODOLOGY

6.1 System Architecture Overview

The proposed IDSMS follows a modular architecture with five primary components: (1) live traffic capture from client devices through router and MikroTik gateway infrastructure, (2) flow aggregation and feature extraction by a dedicated monitoring node, (3) machine learning-based intrusion analysis through an inference service, (4) centralized data storage in a relational database, and (5) web-based visualization and threat management via an interactive dashboard. The system operates within a gateway-based network environment where client traffic passes through routers and a MikroTik gateway, with a dedicated monitoring node performing passive traffic observation and feature extraction before analysis results are stored and visualized.

Figure 1. Overall System Architecture Diagram

6.2 Live Traffic Capture and Flow Aggregation

The traffic processing pipeline, illustrated in Figure 2, begins with packet capture using a Python-based flow sniffer on a Kali Linux VM. The system performs flow aggregation by grouping packets into bidirectional flows based on five-tuple identifiers (source/destination IP/port + protocol). Upon flow completion, feature extraction generates approximately 78 statistical attributes per flow. These feature vectors are sent via REST API to a machine learning inference service, which returns JSON-formatted classification results (e.g., {'data': value, 'risk_level': 'high', 'attack_type': 'DDoS'}). Idle timeout mechanisms ensure timely flow termination and feature export.

Figure 2. Flow Capture and Feature Extraction Pipeline

6.3 Feature Extraction and Preprocessing

For each completed flow, the system extracts approximately 78 statistical features capturing temporal, volumetric, and directional characteristics. These include flow duration, packet counts, byte rates, inter-arrival times, and forward-backward packet statistics. Following feature extraction, the data undergoes JSON formatting to create structured API payloads. These formatted feature vectors are transmitted to the ML inference API, which performs real-time classification and returns prediction results (including risk level, attack probability, and classification labels). All results are then stored in the centralized database and made available for visualization through the web dashboard. Feature normalization is applied to ensure compatibility with machine learning models, and filtering removes incomplete or insignificant traffic. This complete runtime processing pipeline from packet aggregation through database storage to dashboard visualization is shown in Figure 3.

Figure 3. Runtime Data Processing Pipeline of the Proposed IDSMS

6.4 Machine Learning Model Design and Training

6.4.1 Two-Stage Ensemble Architecture

The intrusion detection engine employs a two-stage ensemble approach:

Stage A: Binary classification (benign vs. malicious) using a calibrated stacking ensemble of Random Forest, Gradient Boosting, and XGBoost classifiers with Logistic Regression as meta-classifier

Stage B: Multi-class attack classification using a Random Forest classifier trained exclusively on malicious samples

6.4.2 Training Process

Models were trained offline using the CIC-IDS2017 dataset with 78 numerical flow-based features. Preprocessing included feature normalization using standard scaling and removal of identifier-based fields (IP addresses, ports) to prevent information leakage. Probability calibration and threshold optimization were applied in Stage A to address class imbalance.

6.5 Machine Learning Inference Integration

Extracted flow features are transmitted to a remote inference service via secure API. The service returns structured outputs including risk level, attack probability, and attack classification. This decoupled design enables independent model updates without modifying the traffic capture infrastructure.

6.6 Data Storage and Logging

A centralized relational database implements the storage layer with a normalized schema consisting of five primary tables: Flows (flow metadata), Features (extracted attributes), Users (access control), Predictions (classification outputs), and Attacks (threat records). This structure ensures comprehensive logging of all processed traffic while supporting efficient queries for both real-time monitoring and historical forensic analysis, as detailed in Figure 4.

Figure 4. Database Schema of the Proposed IDSMS

6.7 Web-Based Security Management Interface

A web dashboard provides visualization and management capabilities, displaying metrics such as total flows processed, detected attacks, attack rates, and risk-level distributions. The interface includes time-series graphs, attack distribution charts, and interactive tables for detailed flow inspection. The dashboard supports an interactive security analyst workflow where: (1) an alert appears in the interface, (2) the admin reviews the alert details, (3) checks flow-specific information, (4) decides on an action, and (5) selects either to block the malicious traffic or ignore the alert. This decision-making process is illustrated in Figure 5.

Figure 5. Security Analyst/Admin Decision-Making Workflow

6.8 Experimental Deployment Environment

The system was deployed in a virtualized VMware environment with the following network topology: client devices (1) connect to a router (2), which forwards traffic through a MikroTik gateway (3) linked to an ISP for external access. A dedicated Kali Linux sniffer node is positioned to monitor traffic via a switch/LAN connection, enabling passive observation of all network activity. This setup facilitates controlled experimentation while emulating real-world network behavior, as illustrated in Figure 6.

Figure 6. Network Deployment Topology

7. RESULTS AND DISCUSSIONS

7.1 Flow Capture and Traffic Visibility Validation

During testing, the flow sniffer successfully captured live traffic and aggregated packets into bidirectional flows. The system consistently recorded flow metadata (duration, packet count, byte volume) and differentiated traffic from multiple client devices, confirming its suitability for shared-access environments. The successful extraction of flow-level statistics aligns with established flow-based IDS research [1], [3].

7.2 Real-Time Monitoring Performance

Figure 7 illustrates the Live Monitor interface of the IDSMS, which provides immediate visibility into network activity through real-time metrics and graphical displays. The dashboard shows live statistics such as incoming flows (38), packets per second (31), and alerts per second (0), alongside a time-series graph depicting flow rate variations over a monitoring period. This real-time visualization confirms the system’s capability to process and display traffic fluctuations instantly, a critical feature for environments requiring continuous situational awareness and prompt threat response.

Figure 7. Real-time monitoring interface of the proposed IDSMS showing live flow rate and packet activity

7.3 Feature Generation Consistency

The system generated structured feature vectors with approximately 78 statistical attributes per flow. Key metrics including flow duration, packets per second, bytes per second, and inter-arrival time statistics showed consistency across repeated traffic patterns. During simulated attacks, abnormal behaviors (high packet rates, short inter-arrival times) produced elevated metrics, demonstrating feature sensitivity to anomalies as supported by prior research [4], [10].

7.4 Machine Learning Inference Results

Feature transmission to the inference API occurred without data loss, with the service returning classification results, probability scores, and attack labels. Simulated attack traffic produced elevated probability scores and appropriate attack classifications, validating the inference pipeline's functionality. The probabilistic output format enables nuanced risk assessment, consistent with modern IDS design principles [6], [14].

7.5 Detection Performance of Integrated ML Engine

The integrated two-stage ensemble achieved exceptional performance metrics:

Table 1. Detection Performance of Integrated Machine Learning Engine

Stage A demonstrated near-perfect discrimination capability with 99.92% accuracy and ROC-AUC of 0.99987. Stage B achieved 99.83% accuracy across fourteen attack categories, confirming reliable specific attack identification. These results validate the detection engine's suitability for real-time deployment.

7.6 Database Logging and Data Integrity

All analyzed flows and predictions were successfully stored in the centralized database with correct field population across multiple test runs. The persistent storage enables historical analysis and forensic review, adhering to best practices for real-time IDS deployment [12].

7.7 Dashboard Visualization Effectiveness

Figures 8 illustrate the dashboard’s comprehensive visualization capabilities, which include real-time attack summaries (118 malicious flows detected), traffic overview graphs, and detailed breakdowns of attack types and risk distributions. The interface successfully reflected simulated attack patterns showing spikes in threat activity and updating attack rate metrics dynamically thereby validating the end-to-end synchronization of the IDSMS pipeline.

Figure 8. Dashboard Screenshot (Overview Page)

The Threat Reports module, shown in Figure 10, demonstrated real-time synchronization across the IDSMS by displaying live summary metrics—including 118 detected attacks out of 5,214 total events—and populating a detailed event log table with timestamped flow records, source and destination IPs, risk classifications, probability scores, and traffic statistics as soon as simulated attacks were processed. This immediate update of both numerical counters and forensic data validated the system’s end-to-end operational pipeline, confirming that packet capture, feature extraction, machine learning inference, database storage, and dashboard visualization were seamlessly integrated with negligible latency, thereby proving the system’s readiness for real-world security monitoring where timely threat awareness is essential.

Figure 9. Threat Reports module of the IDSMS showing summarized statistics and per-flow event records.

Figure 10 presents a tabular view of specific attack instances detected during testing, displayed within the Manage Threats interface. The table includes detailed forensic records such as timestamps, source and destination IP addresses, protocol identifiers, attack types (Portscan, Brute Force, FTP Patator, DNS Amplification, DDoS), risk levels, status indicators, and actionable block options. This structured presentation of live threat data validates the system’s classification accuracy and provides security analysts with immediate, actionable forensic details for incident investigation and response.

Figure 10: Sample table of Detected Testing Attacks

7.8 System Integration and Operational Feasibility

The results confirm successful integration of all system components, with reliable end-to-end operation from packet capture through visualization. Compared to prior studies focusing on isolated models [4], [10], this work demonstrates practical system implementation suitable for real-world environments requiring continuous monitoring and actionable insights.

7.9 Limitations of Current Evaluation

The evaluation is limited to development-scale testing with simulated attacks and constrained client numbers. While functional validation is successful, large-scale deployment with diverse traffic patterns and genuine attacks is needed to fully assess detection accuracy and scalability.

8. CONCLUSIONS

8.1 Summary of Findings

This study successfully designed, developed, and evaluated a real-time flow-based Intrusion Detection and Security Management System (IDSMS). The system integrates live network traffic monitoring, machine learning-based intrusion analysis, centralized data storage, and web-based visualization into a cohesive security platform. Key achievements include:

Successful implementation of a modular architecture separating traffic acquisition, analysis, storage, and visualization components

Effective flow capture and feature extraction from MikroTik gateway environments

High-performance detection using a two-stage ensemble machine learning approach (99.92% binary accuracy, 99.83% multi-class accuracy)

Functional integration of all system components with reliable end-to-end operation

Practical visualization through an interactive web dashboard supporting real-time monitoring and historical analysis

8.2 Contributions to Knowledge and Practice

This research makes several important contributions:

Practical Implementation Focus: Unlike many IDS studies concentrating on algorithmic improvements, this work emphasizes system-level design and operational deployment

Integration Framework: Provides a blueprint for integrating machine learning models into operational security systems

Resource-Constrained Solutions: Demonstrates affordable cybersecurity implementation using open-source tools

Shared-Access Environment Applicability: Addresses the specific needs of public internet access services and educational institutions

8.3 Recommendations for Implementation

Based on the findings, the following implementation recommendations are provided:

Institutional Deployment: Educational institutions should consider similar flow-based IDS implementations for cost-effective network security

Configuration Guidelines: Network administrators should ensure proper traffic mirroring configurations and adequate monitoring node resources

Model Maintenance: Regular retraining of machine learning models with updated attack patterns is essential for sustained effectiveness

Scalability Planning: Organizations should plan for database scaling and distributed processing for larger deployments

8.4 Future Research Directions

The following areas warrant further investigation:

Large-Scale Real-World Deployment: Testing in operational networks with diverse traffic patterns and genuine attacks

Automated Response Integration: Implementing dynamic firewall rule enforcement and automated mitigation actions

Adaptive Learning Approaches: Exploring federated and continuous learning for model improvement

Performance Benchmarking: Comparative evaluation against commercial and open-source IDS solutions

Extended Network Environments: Adaptation for cloud-based, SDN, and 5G network infrastructures

8.5 Final Remarks

The developed IDSMS provides a practical foundation for intrusion detection in resource-constrained environments, particularly relevant for educational institutions and small enterprises in developing economies. By bridging the gap between theoretical models and operational systems, this research contributes to more accessible and implementable cybersecurity solutions. The modular design ensures adaptability to evolving network threats and technologies, supporting long-term security management needs.

REFERENCES

[1] Satheesh, N., Rathnamma, M. V., Rajeshkumar, G., et al. (2020). Flow-based anomaly intrusion detection using machine learning model with software defined networking for OpenFlow network. Microprocessors and Microsystems, 79, 103285.

[2] Pontes, C. F. T., de Souza, M. M. C., Gondim, J. J. C., Bishop, M., & Marotta, M. A. (2021). A New Method for Flow-Based Network Intrusion Detection Using the Inverse Potts Model. IEEE Transactions on Network and Service Management, 18(2),

1125–1136.

[3] Segura-Garcia, J., Garcia-Pineda, M., & Felici-Castell, S. (2021). HeaderHunter: A flow-based, real-time intrusion detection system for IEEE 802.11 networks. Sensors, 21(4), 1140.

[4] Sarhan, M., Layeghy, S., & Portmann, M. (2022). Towards a Standard Feature Set for Network Intrusion Detection System Datasets. Mobile Networks and Applications, 27(1), 357–370.

[5] Ferrag, M. A., Friha, O., Hamouda, D., Maglaras, L., & Janicke, H. (2022). Edge-IIoTset: A New Comprehensive Realistic Cyber Security Dataset of IoT and IIoT Applications for Centralized and Federated Learning. IEEE Access.

[6] Vishwakarma, M., & Kesswani, N. (2022). DIDS: A deep neural network based real-time intrusion detection system for IoT. Decision Analytics Journal, 5, 100142.

[7] Rathore, H., et al. (2023). An anomaly-based IDS for IoT using NetFlow features and deep learning. Neural Computing and Applications.

[8] Fraihat, S., et al. (2023). Intelligent intrusion detection in IoT networks using NetFlow-based traffic and ML (IoT-focused). Internet of Things.

[9] Jablaoui, R., Cheikhrouhou, O., Hamdi, M., & Liouane, N. (2025). Deep learning enabled intrusion detection system for IoT security. EURASIP Journal on Wireless Communications and Networking, 2025(1).

[10] Kurniabudi, Stiawan, D., Darmawijoyo, Idris, M. Y., & Bamhdi, A. M. (2020). CICIDS-2017 Dataset Feature Analysis with Information Gain for Anomaly Detection. IEEE Access, 8, 132911–132921.

[11] Maseer, Z. K., Yusof, R., Bahaman, N., Mostafa, S. A., & Foozy, C. F. M. (2021). Benchmarking of Machine Learning for Anomaly Based Intrusion Detection Systems in the CICIDS2017 Dataset. IEEE Access, 9, 22351–22370.

[12] Komisarek, M., & Pawlicki, M. (2022). A novel, refined dataset for real-time Network Intrusion Detection. In Proceedings of the 17th International Conference on Availability, Reliability and Security (ARES 2022).

[13] de Oliveira, J. A., Gonçalves, V. P., Meneguette, R. I., de Sousa Jr, R. T., Guidoni, D. L., Oliveira, J. C. M., & Rocha Filho, G. P. (2023). F-NIDS — A Network Intrusion Detection System based on federated learning. Computer Networks, 236, 110010.

[14] Mohammad, R., Saeed, F., Almazroi, A. A., Alsubaei, F. S., & Almazroi, A. A. (2024). Enhancing Intrusion Detection Systems Using a Deep Learning and Data Augmentation Approach. Systems, 12(3), 79.

[15] Alitbi, Z. K., Hosseini Seno, S. A., Ghaemi Bafghi, A., & Zabihzadeh, D. (2025). A Generalized and Real-Time Network Intrusion Detection System Through Incremental Feature Encoding and Similarity Embedding Learning. Sensors, 25(16), 4961.

[16] Mutambik, I. (2024). An efficient flow-based anomaly detection system for enhanced security in IoT networks. Sensors, 24(22), 7408.

[17] Callegari, C., Giordano, S., & Pagano, M. (2024). A real-time deep learning-based approach for detecting network attacks. Big Data Research, 36, 100446.

[18] Gheni, H. Q., & Al-Yaseen, W. L. (2024). Two-step data clustering for improved intrusion detection system using CICIoT2023 dataset. e-Prime — Advances in Electrical Engineering, Electronics and Energy, 9, 100673.

[19] Sayed, N., Baskar, S., Ramachandran, M., et al. (2023). Augmenting IoT intrusion detection system performance using deep neural network. CMC—Computers, Materials & Continua, 74(1).

[20] Manocchio, L. D., Layeghy, S., Lo, W. W., et al. (2023). FlowTransformer: A transformer framework for flow-based network intrusion detection systems. Expert Systems with Applications, 241, 122564.

[21] Doriguzzi-Corin, R., & Siracusa, D. (2023). FLAD: Adaptive federated learning for DDoS attack detection. Computers & Security, 137, 103597.

[22] Nugraha, K. D., Kota, V. R., & Rupp, M. (2024). Leveraging federated learning and variational autoencoders for an enhanced anomaly detection system. IEEE NetSoft 2024.

[23] Devine, M., Ardakani, S. P., Al-Khafajiy, M., & James, Y. (2025). Federated machine learning to enable intrusion detection systems in IoT networks. Electronics, 14(6), 1176.

[24] Buyuktanir, O., Nandi, D., & Goyal, N. (2025). A review of federated learning applications in intrusion detection systems. Cluster Computing.

[25] Saligue, R. Y., & Saligue, E. T. (2025). Real-World Traffic Analysis in Pisonet using DTW and Anomaly Detection. In 2025 7th International Conference on Innovative Data Communication Technologies and Application (ICIDCA). IEEE.

[26] Saligue, R. Y., Saligue, E. T., Teofilo, L. M., Sy, K. N. L., & Cerna, P. D. (2025). Dynamic Analysis and Visualization of Network Traffic Patterns using Machine Learning and Seasonal Decomposition Techniques. In 2025 5th International Conference on Soft Computing for Security Applications (ICSCSA). IEEE.

Manuscript table 1, flattened text follows.

Stage ; Task ; Accuracy ; ROC-AUC

Stage A ; Binary Classification (Benign vs Attack) ; 99.92% ; 0.99987

Stage B ; Multi-class Classification (14 attack types) ; 99.83% ; -

Two-Stage Ensemble Machine Learning for Network Intrusion Detection.docx

Historical source manuscript: setup/Training/Two-Stage Ensemble Machine Learning for Network Intrusion Detection.docx. This appendix preserves source text and does not endorse its numerical results or treat its architecture as the current deployment.

Two-Stage Ensemble Machine Learning for Network Intrusion Detection

Authors

1Affiliation/Institution/University

email1

Introduction

The increasing complexity and frequency of cyber threats have made Intrusion Detection Systems (IDS) a critical component in ensuring network security. Traditional signature-based IDS often fail to detect new or evolving attacks, particularly zero-day exploits, due to their reliance on predefined patterns (Buczak & Guven, 2016). To overcome these limitations, researchers have turned toward machine learning-based anomaly detection techniques capable of recognizing novel attack behaviors (Sommer & Paxson, 2010). However, many existing machine learning approaches struggle with class imbalance, probability miscalibration, and threshold optimization, which can lead to unreliable detection in real-world environments (Niculescu-Mizil & Caruana, 2005; Zadrozny & Elkan, 2002).

Benchmark datasets have played a crucial role in evaluating IDS performance. Early datasets such as KDD CUP 99 (Tavallaee et al., 2009) and UNSW-NB15 (Moustafa & Slay, 2015) were instrumental but limited by redundancy and outdated attack types. The CIC-IDS2017 dataset (Sharafaldin et al., 2018) addressed these shortcomings by providing realistic traffic with modern attack profiles, including DoS, DDoS, and PortScan, among others (Ring et al., 2019).

Ensemble learning techniques, including Random Forest (Breiman, 2001), Gradient Boosting (Friedman, 2001), and XGBoost (Chen & Guestrin, 2016), have demonstrated strong performance in IDS applications by combining multiple classifiers to reduce bias and variance. Studies by Kevric et al. (2017) and Aljawarneh et al. (2018) showed that combining tree-based models can improve detection rates, while Almseidin et al. (2017) highlighted the superior accuracy of ensemble methods compared to single classifiers.

Building upon these insights, this study proposes a two-stage ensemble learning model designed to improve both binary detection and multi-class attack classification through calibrated stacking and optimized threshold tuning. This model aims to achieve higher detection accuracy and robustness against imbalanced network traffic, thereby addressing key limitations in existing IDS frameworks.

Objectives of the Study

This study aims to develop and evaluate a two-stage ensemble-based Intrusion Detection System (IDS) capable of improving detection accuracy, probability calibration, and attack categorization performance. Specifically, the objectives are to:

Design a two-stage IDS framework where Stage A performs binary classification (benign vs. attack) and Stage B performs multi-class classification (specific attack types).

Implement a calibrated stacking ensemble integrating Random Forest, Gradient Boosting, and XGBoost classifiers for reliable probability estimation.

Conduct a comprehensive performance evaluation using the CIC-IDS2017 dataset, focusing on metrics such as precision, recall, F1-score, and ROC-AUC.

Compare the proposed approach with existing IDS models to demonstrate its effectiveness and efficiency in real-world cybersecurity environments.

Materials and Methods

Dataset and Preprocessing

We utilized the CIC-IDS2017 dataset (Sharafaldin et al., 2018), which contains realistic network traffic with both benign and attack flows. The dataset includes 2.8 million samples across 80 features, capturing various attack scenarios over multiple days. Following the dataset characterization by Sharafaldin et al. (2018), we focused on network-layer attacks detectable through flow statistics. Haider et al. (2017) discussed the importance of realistic dataset generation using fuzzy qualitative modeling, which aligns with our use of the CIC-IDS2017 dataset to ensure real-world applicability.

Figure 1. Dataset distribution across traffic types for training and testing sets (CIC-IDS2017)

Data preprocessing involved several steps. First, we normalized column names and handled missing values by replacing them with zeros. Numerical features were standardized using StandardScaler, while non-numeric identifiers were excluded to prevent data leakage. The label column was processed using Unicode normalization to ensure consistency across different attack naming conventions.

Feature Selection

Based on the analysis by Garcia et al. (2014), we selected 78 numerical features representing network flow characteristics. Features related to pure identifiers (IP addresses, ports, timestamps) were excluded, while behavioral features like flow bytes per second, packets per second, and inter-arrival times were retained. This approach aligns with feature selection strategies discussed by Aljawarneh et al. (2018) for handling high-dimensional network data.

Model Architecture

Stage A: Binary Classification (Benign vs Attack)

We implemented a stacking ensemble combining Random Forests (Breiman, 2001), Gradient Boosting (Friedman, 2001), and XGBoost (Chen & Guestrin, 2016). The ensemble employs logistic regression as a meta-classifier, following the stacked generalization framework (Wolpert, 1992). To address probability calibration, we incorporated CalibratedClassifierCV with sigmoid calibration (Niculescu-Mizil & Caruana, 2005; Platt, 1999), which has shown effectiveness in cybersecurity applications (Zadrozny & Elkan, 2001).

The binary classifier is formulated as:

Stage B: Multi-class Classification (Attack Categorization)

For attack type classification, we employed a Random Forest classifier (Breiman, 2001) trained exclusively on attack samples. This separation prevents label leakage and ensures that the multi-class classifier focuses on distinguishing between different attack patterns rather than benign traffic.

Threshold Optimization

Experimental Setup

The dataset was split using stratified sampling with 70% for training and 30% for testing, ensuring proportional representation of all attack types in both sets. We implemented the system in Python using scikit-learn, with hyperparameters optimized for balanced performance and computational efficiency. The experiments were conducted with FAST_MODE enabled to facilitate rapid iteration while maintaining competitive performance.

Ethical Considerations and Data Source Licensing

This study used the publicly available CIC-IDS2017 dataset (Sharafaldin et al., 2018), which was collected and distributed by the Canadian Institute for Cybersecurity (CIC) for academic research purposes. The dataset does not contain any personally identifiable information (PII) or sensitive user data, ensuring compliance with research ethics and privacy standards. All analyses were conducted in accordance with institutional data use guidelines.

Results and Discussion

Stage A: Binary Classification Performance

Our stacking ensemble achieved exceptional performance in distinguishing between benign and attack traffic. As shown in Table 1, the system attained 99.92% accuracy with balanced precision and recall across both classes. The ROC-AUC score of 0.99987 demonstrates near-perfect separability between benign and malicious traffic.

Figure 2. ROC curve for Stage A showing near-perfect class separability (AUC = 0.99987).

Table 1 Stage A Binary Classification Results

Figure 3. Stage A binary classification performance showing precision, recall, and F1-score for benign and attack classes.

Threshold optimization yielded an optimal operating point at 0.256, providing balanced performance between false positives and false negatives. This calibrated threshold significantly outperforms the default 0.5 threshold, particularly in recall for attack detection.

Figure 4. Precision–Recall curve for Stage A showing optimal threshold selection (0.256).

Stage B: Multi-class Attack Classification

The multi-class classifier demonstrated outstanding performance across 14 attack categories, achieving 99.83% overall accuracy. As detailed in Table 2, most attack types were classified with near-perfect accuracy, particularly high-volume attacks like DDoS (100%), DoS HULK (100%), and PortScan (100%).

Table 2 Stage B Multi-class Classification Performance (Selected Attacks)

Figure 5. Stage B multi-class attack classification accuracies across 14 attack types.

The confusion matrix revealed minimal misclassification between attack types, with most errors occurring in low-sample categories. This performance surpasses recently reported results in the literature, demonstrating the effectiveness of our two-stage approach.

Figure 6. Confusion matrix for Stage B showing classification performance across network-layer attacks.

Comparative Analysis

Our results significantly outperform existing approaches in the literature. As shown in Table 3, our method achieves superior performance compared to recent ensemble-based IDS proposals.

Table 3 Performance Comparison with Recent Works

The performance improvement can be attributed to several factors: (1) effective probability calibration addressing dataset imbalance (Niculescu-Mizil & Caruana, 2005), (2) optimal threshold tuning maximizing F1-score (Zadrozny & Elkan, 2002), and (3) the two-stage architecture preventing interference between binary and multi-class classification tasks.

Computational Efficiency

Despite the sophisticated ensemble architecture, our system maintains practical computational requirements. Stage A training required approximately 3.3 hours, while Stage B training completed in 43 seconds. Inference performance supports real-time deployment, with the capability to process thousands of network flows per second. This efficiency aligns with the requirements discussed by Shone et al. (2018) for practical intrusion detection systems.

Figure 7. Computational efficiency comparison between training and inference stages.

Conclusion and Recommendation

This study proposed a two-stage ensemble learning framework for network intrusion detection, designed to improve both detection accuracy and classification reliability. By combining calibrated stacking ensembles in Stage A and Random Forest-based multi-class classification in Stage B, the model effectively separated benign from malicious traffic and accurately identified specific attack types. Evaluation using the CIC-IDS2017 dataset demonstrated state-of-the-art performance, achieving 99.92% binary accuracy and 99.83% multi-class accuracy across 14 attack categories.

The success of the proposed model can be attributed to three main factors: (1) the separation of binary and multi-class tasks, which reduced class interference; (2) the implementation of probability calibration and threshold optimization, enhancing decision reliability; and (3) the integration of ensemble-based methods, which reduced overfitting and improved generalization. Comparative analysis with existing intrusion detection models confirmed that the proposed approach outperformed recent ensemble and deep learning methods in terms of accuracy, recall, and computational efficiency.

For future research, the following directions are recommended:

Integration of Payload Features: Extend the model to detect web-based and application-layer attacks through deep packet inspection and content-aware features.

Adaptive and Online Learning: Incorporate incremental learning mechanisms to enable continuous adaptation to evolving attack behaviors and zero-day threats.

Deployment Optimization: Investigate model compression and lightweight ensemble configurations to ensure efficient deployment in resource-constrained environments, such as IoT and edge devices.

Cross-Domain Evaluation: Test the framework on alternative datasets and real network traffic to assess generalization across different cybersecurity contexts.

Although the proposed two-stage ensemble model achieved high accuracy and robustness, several limitations must be acknowledged. First, the experiments were conducted solely on the CIC-IDS2017 dataset, which, despite its realism, may not fully represent emerging or application-layer attack patterns. Second, the study focused primarily on network-layer traffic and did not incorporate payload-based or encrypted flow features. Finally, computational resource requirements may limit the scalability of the model in large-scale or real-time environments without optimization. These limitations will guide the direction of future work.

REFERENCES

Sharafaldin, I., Lashkari, A. H., & Ghorbani, A. A. (2018). Toward generating a new intrusion detection dataset and intrusion traffic characterization. Proceedings of the 4th International Conference on Information Systems Security and Privacy, 108–116. https://doi.org/10.5220/0006639801080116

Moustafa, N., & Slay, J. (2015). UNSW-NB15: A comprehensive data set for network intrusion detection systems. 2015 Military Communications and Information Systems Conference (MilCIS), 1–6. https://doi.org/10.1109/MilCIS.2015.7348942

Tavallaee, M., Bagheri, E., Lu, W., & Ghorbani, A. A. (2009). A detailed analysis of the KDD CUP 99 data set. 2009 IEEE Symposium on Computational Intelligence for Security and Defense Applications, 1–6. https://doi.org/10.1109/CISDA.2009.5356528

Breiman, L. (2001). Random forests. Machine Learning, 45(1), 5–32. https://doi.org/10.1023/A:1010933404324

Friedman, J. H. (2001). Greedy function approximation: A gradient boosting machine. The Annals of Statistics, 29(5), 1189–1232. https://doi.org/10.1214/aos/1013203451

Chen, T., & Guestrin, C. (2016). XGBoost: A scalable tree boosting system. Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, 785–794. https://doi.org/10.1145/2939672.2939785

Wolpert, D. H. (1992). Stacked generalization. Neural Networks, 5(2), 241–259. https://doi.org/10.1016/S0893-6080(05)80023-1

Niculescu-Mizil, A., & Caruana, R. (2005). Predicting good probabilities with supervised learning. Proceedings of the 22nd International Conference on Machine Learning, 625–632. https://doi.org/10.1145/1102351.1102430

Platt, J. (1999). Probabilistic outputs for support vector machines and comparisons to regularized likelihood methods. Advances in Large Margin Classifiers, 10(3), 61–74.

Zadrozny, B., & Elkan, C. (2002). Transforming classifier scores into accurate multiclass probability estimates. Proceedings of the 8th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, 694–699. https://doi.org/10.1145/775047.775151

Buczak, A. L., & Guven, E. (2016). A survey of data mining and machine learning methods for cyber security intrusion detection. IEEE Communications Surveys & Tutorials, 18(2), 1153–1176. https://doi.org/10.1109/COMST.2015.2494502

Ring, M., Wunderlich, S., Scheuring, D., Landes, D., & Hotho, A. (2019). A survey of network-based intrusion detection data sets. Computers & Security, 86, 147–167. https://doi.org/10.1016/j.cose.2019.06.005

Khraisat, A., Gondal, I., Vamplew, P., & Kamruzzaman, J. (2019). Survey of intrusion detection systems: techniques, datasets and challenges. Cybersecurity, 2(1), 1–22. https://doi.org/10.1186/s42400-019-0038-7

Zadrozny, B., & Elkan, C. (2001). Obtaining calibrated probability estimates from decision trees and naive Bayesian classifiers. Proceedings of the 18th International Conference on Machine Learning, 609–616.

Garcia, S., Zunino, A., & Campo, M. (2014). Survey on network-based botnet detection methods. Security and Communication Networks, 7(5), 878–903. https://doi.org/10.1002/sec.800

Aljawarneh, S., Aldwairi, M., & Yassein, M. B. (2018). Anomaly-based intrusion detection system through feature selection analysis and building hybrid efficient model. Journal of Computational Science, 25, 152–160. https://doi.org/10.1016/j.jocs.2017.03.006

Vinayakumar, R., Alazab, M., Soman, K. P., Poornachandran, P., Al-Nemrat, A., & Venkatraman, S. (2019). Deep learning approach for intelligent intrusion detection system. IEEE Access, 7, 41525–41550. https://doi.org/10.1109/ACCESS.2019.2895334

Kasongo, S. M., & Sun, Y. (2020). A deep learning method with wrapper based feature extraction for wireless intrusion detection system. Computers & Security, 92, 101752. https://doi.org/10.1016/j.cose.2020.101752

Sommer, R., & Paxson, V. (2010). Outside the closed world: On using machine learning for network intrusion detection. 2010 IEEE Symposium on Security and Privacy, 305–316. https://doi.org/10.1109/SP.2010.25

Yin, C., Zhu, Y., Fei, J., & He, X. (2017). A deep learning approach for intrusion detection using recurrent neural networks. IEEE Access, 5, 21954–21961. https://doi.org/10.1109/ACCESS.2017.2762418

Ahmed, M., Mahmood, A. N., & Hu, J. (2016). A survey of network anomaly detection techniques. Journal of Network and Computer Applications, 60, 19–31. https://doi.org/10.1016/j.jnca.2015.11.016

Mirsky, Y., Doitshman, T., Elovici, Y., & Shabtai, A. (2018). Kitsune: An ensemble of autoencoders for online network intrusion detection. Proceedings of the 2018 Network and Distributed System Security Symposium. https://doi.org/10.14722/ndss.2018.23204

Shone, N., Ngoc, T. N., Phai, V. D., & Shi, Q. (2018). A deep learning approach to network intrusion detection. IEEE Transactions on Emerging Topics in Computational Intelligence, 2(1), 41-50. https://doi.org/10.1109/TETCI.2017.2772792

Javaid, A., Niyaz, Q., Sun, W., & Alam, M. (2016). A deep learning approach for network intrusion detection system. *Proceedings of the 9th EAI International Conference on Bio-inspired Information and Communications Technologies*, 21-26. https://doi.org/10.4108/eai.3-12-2015.2262516

Haider, W., Hu, J., Slay, J., Turnbull, B. P., & Xie, Y. (2017). Generating realistic intrusion detection system dataset based on fuzzy qualitative modeling. Journal of Network and Computer Applications, 87, 185-192. https://doi.org/10.1016/j.jnca.2017.03.018

Sperotto, A., Schaffrath, G., Sadre, R., Morariu, C., Pras, A., & Stiller, B. (2010). An overview of IP flow-based intrusion detection. IEEE Communications Surveys & Tutorials, 12(3), 343-356. https://doi.org/10.1109/SURV.2010.032210.00054

Wang, W., & Lu, Z. (2013). Cyber security in the smart grid: Survey and challenges. Computer Networks, 57(5), 1344-1371. https://doi.org/10.1016/j.comnet.2012.12.017

Almseidin, M., Alzubi, M., Kovacs, S., & Alkasassbeh, M. (2017). Evaluation of machine learning algorithms for intrusion detection system. 2017 IEEE 15th International Symposium on Intelligent Systems and Informatics (SISY), 000277-000282. https://doi.org/10.1109/SISY.2017.8080566

Kevric, J., Jukic, S., & Subasi, A. (2017). An effective combining classifier approach using tree algorithms for network intrusion detection. Neural Computing and Applications, 28(1), 1051-1058. https://doi.org/10.1007/s00521-016-2418-1

Chandola, V., Banerjee, A., & Kumar, V. (2009). Anomaly detection: A survey. ACM Computing Surveys, 41(3), 1-58. https://doi.org/10.1145/1541880.1541882

Manuscript table 1, flattened text follows.

A B S T R A C T ;

This study presents a two-stage ensemble machine learning framework for network intrusion detection systems (IDS), designed to enhance detection accuracy and classification reliability in increasingly complex cyber environments. The proposed model operates in two stages: Stage A performs binary classification to distinguish benign traffic from malicious activity using a calibrated stacking ensemble of Random Forest, Gradient Boosting, and XGBoost classifiers; Stage B conducts multi-class attack categorization through a Random Forest classifier trained exclusively on attack samples. The system was evaluated using the CIC-IDS2017 dataset, comprising over 2.8 million network traffic records across multiple attack scenarios. Preprocessing steps included feature normalization, handling of missing values, and selection of 78 flow-based numerical features. Experimental results revealed that the two-stage ensemble achieved 99.92% accuracy in binary classification and 99.83% accuracy in multi-class classification across 14 attack types. The model demonstrated near-perfect ROC-AUC scores (0.99987) and effectively mitigated class imbalance through calibrated probability estimation and optimized threshold tuning. Comparative analysis showed that the proposed system outperformed existing ensemble and deep learning approaches in both accuracy and computational efficiency. These findings highlight the potential of multi-level ensemble learning in improving IDS performance for modern network infrastructures. Future work will focus on adaptive learning for emerging threats and integration with real-time network defense systems. ; ; K E Y W O R D S; ; ensemble learning, ; intrusion detection, ; machine learning, ; network security, ; random forest

Manuscript table 2, flattened text follows.

Metric ; Benign ; Attack ; Weighted Avg

Precision ; 99.97% ; 99.73% ; 99.92%

Recall ; 99.93% ; 99.86% ; 99.92%

F1-Score ; 99.95% ; 99.79% ; 99.92%

Manuscript table 3, flattened text follows.

Metric ; ; DDoS ; DoS HULK ; PortScan ; DoS GoldenEye ; FTP-Patator ; Overall

Precision ; ; 100.0% ; 99.96% ; 99.99% ; 99.61% ; 99.83%

Recall ; ; 100.0% ; 99.99% ; 99.97% ; 99.67% ; 99.83%

F1-Score ; ; 100.0% ; 99.97% ; 99.98% ; 99.64% ; 99.83%

Support ; ; 38,428 ; 69,409 ; 47,614 ; 3,063 ; 2,376 ; 167,294

Manuscript table 4, flattened text follows.

Study ; Approach ; Binary Accuracy ; Multi-class Accuracy

Our Work ; Two-stage Ensemble ; 99.92% ; 99.83%

Kevric et al. (2017) ; Combining Classifier Approach ; 99.20% ; 98.10%

Aljawarneh et al. (2018) ; Hybrid Efficient Model ; 99.50% ; 98.70%

Vinayakumar et al. (2019) ; Deep Learning Approach ; 99.10% ; 97.90%

Kasongo & Sun (2020) ; Deep Learning with Feature Extraction ; 99.30% ; 98.20%

25. Complete project-file inventory and uses

This inventory covers first-party source, configuration, tests, static website assets, setup material, datasets, archives, and generated artifacts visible in the project. Each file is named with its purpose and byte size. Installed PHP dependencies, JavaScript dependencies, Git internals, virtual environments, private environment files, and private runtime session/log contents are grouped rather than expanded. These are reproducible dependency or sensitive runtime stores, not hundreds of additional custom IDS modules. Inventory sizes describe this local snapshot only.

File inventory: Root project files

.editorconfig. 252 bytes. Shared editor formatting settings.

.gitattributes. 288 bytes. Version-control line-ending or file handling attributes.

.gitignore. 369 bytes. Version-control exclusion rules for private and generated files.

artisan. 425 bytes. Laravel command-line bootstrap for migrations, seeders, caches, and other framework commands.

composer.json. 2,876 bytes. PHP dependency ranges, autoload mappings, and development/setup commands.

composer.lock. 346,400 bytes. Resolved PHP dependency versions for reproducible installs.

package-lock.json. 92,272 bytes. Resolved npm dependencies and integrity metadata.

package.json. 414 bytes. Frontend build tooling and scripts.

phpunit.xml. 1,284 bytes. Isolated PHP test environment configuration.

README.md. 3,911 bytes. Framework scaffold overview; operational IDS setup is documented separately under setup.

vite.config.js. 436 bytes. Frontend build entrypoints and development watcher configuration.

File inventory: app

app/Http/Controllers/Controller.php. 77 bytes. Laravel request controller; methods and behavior documented in the controller register.

app/Http/Controllers/DashboardController.php. 9,936 bytes. Laravel request controller; methods and behavior documented in the controller register.

app/Http/Controllers/IntrusionLogController.php. 333 bytes. Laravel request controller; methods and behavior documented in the controller register.

app/Http/Controllers/LiveMonitorController.php. 443 bytes. Laravel request controller; methods and behavior documented in the controller register.

app/Http/Controllers/LogController.php. 2,796 bytes. Laravel request controller; methods and behavior documented in the controller register.

app/Http/Controllers/LoginController.php. 1,733 bytes. Laravel request controller; methods and behavior documented in the controller register.

app/Http/Controllers/ManageThreatController.php. 7,507 bytes. Laravel request controller; methods and behavior documented in the controller register.

app/Http/Controllers/SystemStatusController.php. 8,296 bytes. Laravel request controller; methods and behavior documented in the controller register.

app/Http/Controllers/ThreatReportsController.php. 5,088 bytes. Laravel request controller; methods and behavior documented in the controller register.

app/Http/Middleware/CheckSession.php. 1,237 bytes. Custom website session and role authorization middleware.

app/Models/IntrusionLog.php. 712 bytes. Eloquent persistence model with fillable fields, casting, accessors, or relationships.

app/Models/ThreatIncident.php. 970 bytes. Eloquent persistence model with fillable fields, casting, accessors, or relationships.

app/Models/User.php. 1,009 bytes. Eloquent persistence model with fillable fields, casting, accessors, or relationships.

app/Models/UsersAccount.php. 1,247 bytes. Eloquent persistence model with fillable fields, casting, accessors, or relationships.

app/Providers/AppServiceProvider.php. 526 bytes. Application service-provider lifecycle hooks.

File inventory: bootstrap

bootstrap/app.php. 715 bytes. Laravel application or service-provider bootstrap.

bootstrap/providers.php. 64 bytes. Laravel application or service-provider bootstrap.

File inventory: config

config/app.php. 4,272 bytes. Laravel app configuration; consumes environment settings as registered earlier.

config/auth.php. 4,029 bytes. Laravel auth configuration; consumes environment settings as registered earlier.

config/cache.php. 3,683 bytes. Laravel cache configuration; consumes environment settings as registered earlier.

config/database.php. 6,960 bytes. Laravel database configuration; consumes environment settings as registered earlier.

config/filesystems.php. 2,500 bytes. Laravel filesystems configuration; consumes environment settings as registered earlier.

config/logging.php. 4,327 bytes. Laravel logging configuration; consumes environment settings as registered earlier.

config/mail.php. 3,614 bytes. Laravel mail configuration; consumes environment settings as registered earlier.

config/queue.php. 4,199 bytes. Laravel queue configuration; consumes environment settings as registered earlier.

config/sanctum.php. 3,037 bytes. Laravel sanctum configuration; consumes environment settings as registered earlier.

config/services.php. 1,039 bytes. Laravel services configuration; consumes environment settings as registered earlier.

config/session.php. 7,847 bytes. Laravel session configuration; consumes environment settings as registered earlier.

File inventory: database

database/.gitignore. 10 bytes. Supporting project or template asset; not a separately launched IDS component.

database/factories/UserFactory.php. 1,075 bytes. Framework test/sample account data factory.

database/migrations/0001_01_01_000000_create_users_table.php. 1,473 bytes. Versioned database schema or historical data transformation; see migration register.

database/migrations/0001_01_01_000001_create_cache_table.php. 849 bytes. Versioned database schema or historical data transformation; see migration register.

database/migrations/0001_01_01_000002_create_jobs_table.php. 1,812 bytes. Versioned database schema or historical data transformation; see migration register.

database/migrations/2025_12_14_101328_create_intrusion_logs_table.php. 1,684 bytes. Versioned database schema or historical data transformation; see migration register.

database/migrations/2025_12_14_102846_create_personal_access_tokens_table.php. 863 bytes. Versioned database schema or historical data transformation; see migration register.

database/migrations/2025_12_17_123456_create_users_account_table.php. 846 bytes. Versioned database schema or historical data transformation; see migration register.

database/migrations/2025_12_19_000000_add_status_to_intrusion_logs_table.php. 755 bytes. Versioned database schema or historical data transformation; see migration register.

database/migrations/2026_06_23_000001_update_intrusion_logs_status_enum.php. 545 bytes. Versioned database schema or historical data transformation; see migration register.

database/migrations/2026_06_23_000002_backfill_intrusion_logs_timestamps.php. 676 bytes. Versioned database schema or historical data transformation; see migration register.

database/migrations/2026_06_23_000003_reclassify_pending_intrusion_logs_to_blocked.php. 636 bytes. Versioned database schema or historical data transformation; see migration register.

database/migrations/2026_06_24_000001_update_demo_dst_ip_in_intrusion_logs.php. 564 bytes. Versioned database schema or historical data transformation; see migration register.

database/migrations/2026_08_02_000001_add_pppoe_name_to_intrusion_logs.php. 1,113 bytes. Versioned database schema or historical data transformation; see migration register.

database/migrations/2026_08_02_000002_create_threat_incidents_table.php. 3,448 bytes. Versioned database schema or historical data transformation; see migration register.

database/migrations/2026_10_29_000001_add_role_to_users_account_table.php. 596 bytes. Versioned database schema or historical data transformation; see migration register.

database/seeders/DatabaseSeeder.php. 530 bytes. Database initialization or dedicated account seed operation.

database/seeders/ViewOnlyUserSeeder.php. 477 bytes. Database initialization or dedicated account seed operation.

File inventory: ngrok

ngrok/ngrok.exe. 31,770,400 bytes. Optional ngrok tunneling executable; no main service automatically launches it.

File inventory: public

public/.htaccess. 740 bytes. Supporting project or template asset; not a separately launched IDS component.

public/admin/.browserslistrc. 46 bytes. Supporting project or template asset; not a separately launched IDS component.

public/admin/.eslintignore. 18 bytes. Supporting project or template asset; not a separately launched IDS component.

public/admin/.eslintrc.json. 531 bytes. Dependency, template, icon, or tooling metadata; context determined by its directory.

public/admin/.github/workflows/handle-new-issue-comment.yml. 371 bytes. Supporting project or template asset; not a separately launched IDS component.

public/admin/.github/workflows/handle-new-issue.yml. 1,718 bytes. Supporting project or template asset; not a separately launched IDS component.

public/admin/.github/workflows/issue-staler.yml. 686 bytes. Supporting project or template asset; not a separately launched IDS component.

public/admin/.gitignore. 763 bytes. Supporting project or template asset; not a separately launched IDS component.

public/admin/.prettierignore. 42 bytes. Supporting project or template asset; not a separately launched IDS component.

public/admin/.prettierrc.json. 479 bytes. Dependency, template, icon, or tooling metadata; context determined by its directory.

public/admin/assets/css/demo.css. 2,543 bytes. Prebuilt website or template stylesheet.

public/admin/assets/img/avatars/1.png. 14,015 bytes. Static website image, branding, illustration, favicon, or vendor icon asset.

public/admin/assets/img/avatars/5.png. 20,488 bytes. Static website image, branding, illustration, favicon, or vendor icon asset.

public/admin/assets/img/avatars/6.png. 15,198 bytes. Static website image, branding, illustration, favicon, or vendor icon asset.

public/admin/assets/img/avatars/7.png. 15,180 bytes. Static website image, branding, illustration, favicon, or vendor icon asset.

public/admin/assets/img/backgrounds/18.jpg. 88,783 bytes. Static website image, branding, illustration, favicon, or vendor icon asset.

public/admin/assets/img/elements/1.jpg. 18,135 bytes. Static website image, branding, illustration, favicon, or vendor icon asset.

public/admin/assets/img/elements/11.jpg. 19,087 bytes. Static website image, branding, illustration, favicon, or vendor icon asset.

public/admin/assets/img/elements/12.jpg. 28,075 bytes. Static website image, branding, illustration, favicon, or vendor icon asset.

public/admin/assets/img/elements/13.jpg. 12,929 bytes. Static website image, branding, illustration, favicon, or vendor icon asset.

public/admin/assets/img/elements/17.jpg. 19,716 bytes. Static website image, branding, illustration, favicon, or vendor icon asset.

public/admin/assets/img/elements/18.jpg. 31,223 bytes. Static website image, branding, illustration, favicon, or vendor icon asset.

public/admin/assets/img/elements/19.jpg. 15,553 bytes. Static website image, branding, illustration, favicon, or vendor icon asset.

public/admin/assets/img/elements/2.jpg. 13,332 bytes. Static website image, branding, illustration, favicon, or vendor icon asset.

public/admin/assets/img/elements/20.jpg. 16,593 bytes. Static website image, branding, illustration, favicon, or vendor icon asset.

public/admin/assets/img/elements/3.jpg. 24,285 bytes. Static website image, branding, illustration, favicon, or vendor icon asset.

public/admin/assets/img/elements/4.jpg. 22,875 bytes. Static website image, branding, illustration, favicon, or vendor icon asset.

public/admin/assets/img/elements/5.jpg. 22,938 bytes. Static website image, branding, illustration, favicon, or vendor icon asset.

public/admin/assets/img/elements/7.jpg. 21,183 bytes. Static website image, branding, illustration, favicon, or vendor icon asset.

public/admin/assets/img/favicon/favicon.ico. 1,393 bytes. Static website image, branding, illustration, favicon, or vendor icon asset.

public/admin/assets/img/icons/brands/asana.png. 2,236 bytes. Static website image, branding, illustration, favicon, or vendor icon asset.

public/admin/assets/img/icons/brands/behance.png. 1,731 bytes. Static website image, branding, illustration, favicon, or vendor icon asset.

public/admin/assets/img/icons/brands/dribbble.png. 2,848 bytes. Static website image, branding, illustration, favicon, or vendor icon asset.

public/admin/assets/img/icons/brands/facebook.png. 681 bytes. Static website image, branding, illustration, favicon, or vendor icon asset.

public/admin/assets/img/icons/brands/github.png. 2,169 bytes. Static website image, branding, illustration, favicon, or vendor icon asset.

public/admin/assets/img/icons/brands/google.png. 1,932 bytes. Static website image, branding, illustration, favicon, or vendor icon asset.

public/admin/assets/img/icons/brands/instagram.png. 3,128 bytes. Static website image, branding, illustration, favicon, or vendor icon asset.

public/admin/assets/img/icons/brands/mailchimp.png. 1,405 bytes. Static website image, branding, illustration, favicon, or vendor icon asset.

public/admin/assets/img/icons/brands/slack.png. 2,550 bytes. Static website image, branding, illustration, favicon, or vendor icon asset.

public/admin/assets/img/icons/brands/twitter.png. 1,564 bytes. Static website image, branding, illustration, favicon, or vendor icon asset.

public/admin/assets/img/icons/unicons/cc-primary.png. 702 bytes. Static website image, branding, illustration, favicon, or vendor icon asset.

public/admin/assets/img/icons/unicons/cc-success.png. 776 bytes. Static website image, branding, illustration, favicon, or vendor icon asset.

public/admin/assets/img/icons/unicons/cc-warning.png. 689 bytes. Static website image, branding, illustration, favicon, or vendor icon asset.

public/admin/assets/img/icons/unicons/chart-success.png. 1,528 bytes. Static website image, branding, illustration, favicon, or vendor icon asset.

public/admin/assets/img/icons/unicons/chart.png. 1,491 bytes. Static website image, branding, illustration, favicon, or vendor icon asset.

public/admin/assets/img/icons/unicons/paypal.png. 1,090 bytes. Static website image, branding, illustration, favicon, or vendor icon asset.

public/admin/assets/img/icons/unicons/wallet-info.png. 936 bytes. Static website image, branding, illustration, favicon, or vendor icon asset.

public/admin/assets/img/icons/unicons/wallet.png. 920 bytes. Static website image, branding, illustration, favicon, or vendor icon asset.

public/admin/assets/img/illustrations/girl-doing-yoga-light.png. 219,096 bytes. Static website image, branding, illustration, favicon, or vendor icon asset.

public/admin/assets/img/illustrations/man-with-laptop-light.png. 8,826 bytes. Static website image, branding, illustration, favicon, or vendor icon asset.

public/admin/assets/img/illustrations/page-misc-error-light.png. 139,086 bytes. Static website image, branding, illustration, favicon, or vendor icon asset.

public/admin/assets/img/layouts/layout-container-light.png. 62,469 bytes. Static website image, branding, illustration, favicon, or vendor icon asset.

public/admin/assets/img/layouts/layout-fluid-light.png. 58,375 bytes. Static website image, branding, illustration, favicon, or vendor icon asset.

public/admin/assets/img/layouts/layout-without-menu-light.png. 45,251 bytes. Static website image, branding, illustration, favicon, or vendor icon asset.

public/admin/assets/img/layouts/layout-without-navbar-light.png. 63,976 bytes. Static website image, branding, illustration, favicon, or vendor icon asset.

public/admin/assets/js/config.js. 714 bytes. Browser interaction, charting, template, or frontend tooling JavaScript.

public/admin/assets/js/dashboards-analytics.js. 15,032 bytes. Browser interaction, charting, template, or frontend tooling JavaScript.

public/admin/assets/js/extended-ui-perfect-scrollbar.js. 1,081 bytes. Browser interaction, charting, template, or frontend tooling JavaScript.

public/admin/assets/js/form-basic-inputs.js. 188 bytes. Browser interaction, charting, template, or frontend tooling JavaScript.

public/admin/assets/js/main.js. 3,846 bytes. Browser interaction, charting, template, or frontend tooling JavaScript.

public/admin/assets/js/pages-account-settings-account.js. 852 bytes. Browser interaction, charting, template, or frontend tooling JavaScript.

public/admin/assets/js/ui-modals.js. 1,068 bytes. Browser interaction, charting, template, or frontend tooling JavaScript.

public/admin/assets/js/ui-popover.js. 443 bytes. Browser interaction, charting, template, or frontend tooling JavaScript.

public/admin/assets/js/ui-toasts.js. 1,305 bytes. Browser interaction, charting, template, or frontend tooling JavaScript.

public/admin/build-config.js. 1,893 bytes. Browser interaction, charting, template, or frontend tooling JavaScript.

public/admin/CHANGELOG.md. 140 bytes. Supporting documentation, distribution notes, or license text.

public/admin/documentation.html. 395 bytes. Static website template, demonstration page, or distribution placeholder; not necessarily a Laravel route.

public/admin/fonts/boxicons.scss. 753 bytes. Template styling source or Sass mixin, variable, component, or style test; not detector logic.

public/admin/gulpfile.js. 3,234 bytes. Browser interaction, charting, template, or frontend tooling JavaScript.

public/admin/html/auth-forgot-password-basic.html. 9,954 bytes. Static website template, demonstration page, or distribution placeholder; not necessarily a Laravel route.

public/admin/html/auth-login-basic.html. 11,190 bytes. Static website template, demonstration page, or distribution placeholder; not necessarily a Laravel route.

public/admin/html/auth-register-basic.html. 11,253 bytes. Static website template, demonstration page, or distribution placeholder; not necessarily a Laravel route.

public/admin/html/cards-basic.html. 60,444 bytes. Static website template, demonstration page, or distribution placeholder; not necessarily a Laravel route.

public/admin/html/extended-ui-perfect-scrollbar.html. 36,208 bytes. Static website template, demonstration page, or distribution placeholder; not necessarily a Laravel route.

public/admin/html/extended-ui-text-divider.html. 34,173 bytes. Static website template, demonstration page, or distribution placeholder; not necessarily a Laravel route.

public/admin/html/form-layouts-horizontal.html. 39,286 bytes. Static website template, demonstration page, or distribution placeholder; not necessarily a Laravel route.

public/admin/html/form-layouts-vertical.html. 37,238 bytes. Static website template, demonstration page, or distribution placeholder; not necessarily a Laravel route.

public/admin/html/forms-basic-inputs.html. 55,855 bytes. Static website template, demonstration page, or distribution placeholder; not necessarily a Laravel route.

public/admin/html/forms-input-groups.html. 57,340 bytes. Static website template, demonstration page, or distribution placeholder; not necessarily a Laravel route.

public/admin/html/icons-boxicons.html. 37,576 bytes. Static website template, demonstration page, or distribution placeholder; not necessarily a Laravel route.

public/admin/html/index.html. 58,115 bytes. Static website template, demonstration page, or distribution placeholder; not necessarily a Laravel route.

public/admin/html/layouts-blank.html. 3,564 bytes. Static website template, demonstration page, or distribution placeholder; not necessarily a Laravel route.

public/admin/html/layouts-container.html. 29,794 bytes. Static website template, demonstration page, or distribution placeholder; not necessarily a Laravel route.

public/admin/html/layouts-fluid.html. 29,773 bytes. Static website template, demonstration page, or distribution placeholder; not necessarily a Laravel route.

public/admin/html/layouts-without-menu.html. 10,798 bytes. Static website template, demonstration page, or distribution placeholder; not necessarily a Laravel route.

public/admin/html/layouts-without-navbar.html. 25,106 bytes. Static website template, demonstration page, or distribution placeholder; not necessarily a Laravel route.

public/admin/html/pages-account-settings-account.html. 42,861 bytes. Static website template, demonstration page, or distribution placeholder; not necessarily a Laravel route.

public/admin/html/pages-account-settings-connections.html. 41,724 bytes. Static website template, demonstration page, or distribution placeholder; not necessarily a Laravel route.

public/admin/html/pages-account-settings-notifications.html. 36,476 bytes. Static website template, demonstration page, or distribution placeholder; not necessarily a Laravel route.

public/admin/html/pages-misc-error.html. 4,343 bytes. Static website template, demonstration page, or distribution placeholder; not necessarily a Laravel route.

public/admin/html/pages-misc-under-maintenance.html. 4,400 bytes. Static website template, demonstration page, or distribution placeholder; not necessarily a Laravel route.

public/admin/html/tables-basic.html. 207,223 bytes. Static website template, demonstration page, or distribution placeholder; not necessarily a Laravel route.

public/admin/html/ui-accordion.html. 37,511 bytes. Static website template, demonstration page, or distribution placeholder; not necessarily a Laravel route.

public/admin/html/ui-alerts.html. 32,868 bytes. Static website template, demonstration page, or distribution placeholder; not necessarily a Laravel route.

public/admin/html/ui-badges.html. 38,927 bytes. Static website template, demonstration page, or distribution placeholder; not necessarily a Laravel route.

public/admin/html/ui-buttons.html. 52,015 bytes. Static website template, demonstration page, or distribution placeholder; not necessarily a Laravel route.

public/admin/html/ui-carousel.html. 34,528 bytes. Static website template, demonstration page, or distribution placeholder; not necessarily a Laravel route.

public/admin/html/ui-collapse.html. 36,286 bytes. Static website template, demonstration page, or distribution placeholder; not necessarily a Laravel route.

public/admin/html/ui-dropdowns.html. 80,935 bytes. Static website template, demonstration page, or distribution placeholder; not necessarily a Laravel route.

public/admin/html/ui-footer.html. 33,037 bytes. Static website template, demonstration page, or distribution placeholder; not necessarily a Laravel route.

public/admin/html/ui-list-groups.html. 52,225 bytes. Static website template, demonstration page, or distribution placeholder; not necessarily a Laravel route.

public/admin/html/ui-modals.html. 80,038 bytes. Static website template, demonstration page, or distribution placeholder; not necessarily a Laravel route.

public/admin/html/ui-navbar.html. 36,310 bytes. Static website template, demonstration page, or distribution placeholder; not necessarily a Laravel route.

public/admin/html/ui-offcanvas.html. 46,914 bytes. Static website template, demonstration page, or distribution placeholder; not necessarily a Laravel route.

public/admin/html/ui-pagination-breadcrumbs.html. 44,392 bytes. Static website template, demonstration page, or distribution placeholder; not necessarily a Laravel route.

public/admin/html/ui-progress.html. 46,084 bytes. Static website template, demonstration page, or distribution placeholder; not necessarily a Laravel route.

public/admin/html/ui-spinners.html. 35,440 bytes. Static website template, demonstration page, or distribution placeholder; not necessarily a Laravel route.

public/admin/html/ui-tabs-pills.html. 46,556 bytes. Static website template, demonstration page, or distribution placeholder; not necessarily a Laravel route.

public/admin/html/ui-toasts.html. 46,469 bytes. Static website template, demonstration page, or distribution placeholder; not necessarily a Laravel route.

public/admin/html/ui-tooltips-popovers.html. 35,528 bytes. Static website template, demonstration page, or distribution placeholder; not necessarily a Laravel route.

public/admin/html/ui-typography.html. 47,495 bytes. Static website template, demonstration page, or distribution placeholder; not necessarily a Laravel route.

public/admin/index.html. 4,848 bytes. Static website template, demonstration page, or distribution placeholder; not necessarily a Laravel route.

public/admin/js/bootstrap.js. 919 bytes. Browser interaction, charting, template, or frontend tooling JavaScript.

public/admin/js/helpers.js. 24,011 bytes. Browser interaction, charting, template, or frontend tooling JavaScript.

public/admin/js/menu.js. 17,223 bytes. Browser interaction, charting, template, or frontend tooling JavaScript.

public/admin/libs/apex-charts/apex-charts.scss. 2,044 bytes. Template styling source or Sass mixin, variable, component, or style test; not detector logic.

public/admin/libs/apex-charts/apexcharts.js. 71 bytes. Browser interaction, charting, template, or frontend tooling JavaScript.

public/admin/libs/highlight/highlight-github.scss. 61 bytes. Template styling source or Sass mixin, variable, component, or style test; not detector logic.

public/admin/libs/highlight/highlight.js. 132 bytes. Browser interaction, charting, template, or frontend tooling JavaScript.

public/admin/libs/highlight/highlight.scss. 65 bytes. Template styling source or Sass mixin, variable, component, or style test; not detector logic.

public/admin/libs/jquery/jquery.js. 82 bytes. Browser interaction, charting, template, or frontend tooling JavaScript.

public/admin/libs/masonry/masonry.js. 167 bytes. Browser interaction, charting, template, or frontend tooling JavaScript.

public/admin/libs/perfect-scrollbar/perfect-scrollbar.js. 103 bytes. Browser interaction, charting, template, or frontend tooling JavaScript.

public/admin/libs/perfect-scrollbar/perfect-scrollbar.scss. 2,393 bytes. Template styling source or Sass mixin, variable, component, or style test; not detector logic.

public/admin/libs/popper/popper.js. 208 bytes. Browser interaction, charting, template, or frontend tooling JavaScript.

public/admin/LICENSE.md. 1,102 bytes. Supporting documentation, distribution notes, or license text.

public/admin/package-lock.json. 960,174 bytes. Dependency, template, icon, or tooling metadata; context determined by its directory.

public/admin/package.json. 3,206 bytes. Dependency, template, icon, or tooling metadata; context determined by its directory.

public/admin/README.md. 13,325 bytes. Supporting documentation, distribution notes, or license text.

public/admin/scss/_bootstrap-extended/_accordion.scss. 1,148 bytes. Template styling source or Sass mixin, variable, component, or style test; not detector logic.

public/admin/scss/_bootstrap-extended/_alert.scss. 267 bytes. Template styling source or Sass mixin, variable, component, or style test; not detector logic.

public/admin/scss/_bootstrap-extended/_badge.scss. 387 bytes. Template styling source or Sass mixin, variable, component, or style test; not detector logic.

public/admin/scss/_bootstrap-extended/_breadcrumb.scss. 897 bytes. Template styling source or Sass mixin, variable, component, or style test; not detector logic.

public/admin/scss/_bootstrap-extended/_button-group.scss. 1,807 bytes. Template styling source or Sass mixin, variable, component, or style test; not detector logic.

public/admin/scss/_bootstrap-extended/_buttons.scss. 2,633 bytes. Template styling source or Sass mixin, variable, component, or style test; not detector logic.

public/admin/scss/_bootstrap-extended/_card.scss. 3,840 bytes. Template styling source or Sass mixin, variable, component, or style test; not detector logic.

public/admin/scss/_bootstrap-extended/_carousel.scss. 576 bytes. Template styling source or Sass mixin, variable, component, or style test; not detector logic.

public/admin/scss/_bootstrap-extended/_close.scss. 132 bytes. Template styling source or Sass mixin, variable, component, or style test; not detector logic.

public/admin/scss/_bootstrap-extended/_dropdown.scss. 1,297 bytes. Template styling source or Sass mixin, variable, component, or style test; not detector logic.

public/admin/scss/_bootstrap-extended/_forms.scss. 280 bytes. Template styling source or Sass mixin, variable, component, or style test; not detector logic.

public/admin/scss/_bootstrap-extended/_functions.scss. 4,614 bytes. Template styling source or Sass mixin, variable, component, or style test; not detector logic.

public/admin/scss/_bootstrap-extended/_include.scss. 546 bytes. Template styling source or Sass mixin, variable, component, or style test; not detector logic.

public/admin/scss/_bootstrap-extended/_list-group.scss. 1,069 bytes. Template styling source or Sass mixin, variable, component, or style test; not detector logic.

public/admin/scss/_bootstrap-extended/_mixins.scss. 429 bytes. Template styling source or Sass mixin, variable, component, or style test; not detector logic.

public/admin/scss/_bootstrap-extended/_modal.scss. 2,165 bytes. Template styling source or Sass mixin, variable, component, or style test; not detector logic.

public/admin/scss/_bootstrap-extended/_nav.scss. 6,399 bytes. Template styling source or Sass mixin, variable, component, or style test; not detector logic.

public/admin/scss/_bootstrap-extended/_navbar.scss. 928 bytes. Template styling source or Sass mixin, variable, component, or style test; not detector logic.

public/admin/scss/_bootstrap-extended/_offcanvas.scss. 225 bytes. Template styling source or Sass mixin, variable, component, or style test; not detector logic.

public/admin/scss/_bootstrap-extended/_pagination.scss. 2,015 bytes. Template styling source or Sass mixin, variable, component, or style test; not detector logic.

public/admin/scss/_bootstrap-extended/_popover.scss. 657 bytes. Template styling source or Sass mixin, variable, component, or style test; not detector logic.

public/admin/scss/_bootstrap-extended/_progress.scss. 828 bytes. Template styling source or Sass mixin, variable, component, or style test; not detector logic.

public/admin/scss/_bootstrap-extended/_reboot.scss. 648 bytes. Template styling source or Sass mixin, variable, component, or style test; not detector logic.

public/admin/scss/_bootstrap-extended/_root.scss. 107 bytes. Template styling source or Sass mixin, variable, component, or style test; not detector logic.

public/admin/scss/_bootstrap-extended/_spinners.scss. 363 bytes. Template styling source or Sass mixin, variable, component, or style test; not detector logic.

public/admin/scss/_bootstrap-extended/_tables.scss. 1,493 bytes. Template styling source or Sass mixin, variable, component, or style test; not detector logic.

public/admin/scss/_bootstrap-extended/_toasts.scss. 1,212 bytes. Template styling source or Sass mixin, variable, component, or style test; not detector logic.

public/admin/scss/_bootstrap-extended/_tooltip.scss. 236 bytes. Template styling source or Sass mixin, variable, component, or style test; not detector logic.

public/admin/scss/_bootstrap-extended/_type.scss. 11 bytes. Template styling source or Sass mixin, variable, component, or style test; not detector logic.

public/admin/scss/_bootstrap-extended/_utilities-ltr.scss. 6,432 bytes. Template styling source or Sass mixin, variable, component, or style test; not detector logic.

public/admin/scss/_bootstrap-extended/_utilities.scss. 18,470 bytes. Template styling source or Sass mixin, variable, component, or style test; not detector logic.

public/admin/scss/_bootstrap-extended/_variables.scss. 39,278 bytes. Template styling source or Sass mixin, variable, component, or style test; not detector logic.

public/admin/scss/_bootstrap-extended/forms/_floating-labels.scss. 293 bytes. Template styling source or Sass mixin, variable, component, or style test; not detector logic.

public/admin/scss/_bootstrap-extended/forms/_form-control.scss. 309 bytes. Template styling source or Sass mixin, variable, component, or style test; not detector logic.

public/admin/scss/_bootstrap-extended/forms/_form-range.scss. 1,047 bytes. Template styling source or Sass mixin, variable, component, or style test; not detector logic.

public/admin/scss/_bootstrap-extended/forms/_form-select.scss. 143 bytes. Template styling source or Sass mixin, variable, component, or style test; not detector logic.

public/admin/scss/_bootstrap-extended/forms/_input-group.scss. 1,375 bytes. Template styling source or Sass mixin, variable, component, or style test; not detector logic.

public/admin/scss/_bootstrap-extended/forms/_labels.scss. 419 bytes. Template styling source or Sass mixin, variable, component, or style test; not detector logic.

public/admin/scss/_bootstrap-extended/forms/_validation.scss. 1,310 bytes. Template styling source or Sass mixin, variable, component, or style test; not detector logic.

public/admin/scss/_bootstrap-extended/mixins/_alert.scss. 1,058 bytes. Template styling source or Sass mixin, variable, component, or style test; not detector logic.

public/admin/scss/_bootstrap-extended/mixins/_badge.scss. 259 bytes. Template styling source or Sass mixin, variable, component, or style test; not detector logic.

public/admin/scss/_bootstrap-extended/mixins/_buttons.scss. 4,697 bytes. Template styling source or Sass mixin, variable, component, or style test; not detector logic.

public/admin/scss/_bootstrap-extended/mixins/_caret.scss. 872 bytes. Template styling source or Sass mixin, variable, component, or style test; not detector logic.

public/admin/scss/_bootstrap-extended/mixins/_dropdown.scss. 781 bytes. Template styling source or Sass mixin, variable, component, or style test; not detector logic.

public/admin/scss/_bootstrap-extended/mixins/_forms.scss. 6,741 bytes. Template styling source or Sass mixin, variable, component, or style test; not detector logic.

public/admin/scss/_bootstrap-extended/mixins/_list-group.scss. 1,412 bytes. Template styling source or Sass mixin, variable, component, or style test; not detector logic.

public/admin/scss/_bootstrap-extended/mixins/_misc.scss. 2,560 bytes. Template styling source or Sass mixin, variable, component, or style test; not detector logic.

public/admin/scss/_bootstrap-extended/mixins/_navs.scss. 1,005 bytes. Template styling source or Sass mixin, variable, component, or style test; not detector logic.

public/admin/scss/_bootstrap-extended/mixins/_pagination.scss. 689 bytes. Template styling source or Sass mixin, variable, component, or style test; not detector logic.

public/admin/scss/_bootstrap-extended/mixins/_progress.scss. 563 bytes. Template styling source or Sass mixin, variable, component, or style test; not detector logic.

public/admin/scss/_bootstrap-extended/mixins/_table-variants.scss. 1,339 bytes. Template styling source or Sass mixin, variable, component, or style test; not detector logic.

public/admin/scss/_bootstrap-extended/mixins/_toasts.scss. 1,045 bytes. Template styling source or Sass mixin, variable, component, or style test; not detector logic.

public/admin/scss/_bootstrap-extended.scss. 1,346 bytes. Template styling source or Sass mixin, variable, component, or style test; not detector logic.

public/admin/scss/_bootstrap.scss. 1,788 bytes. Template styling source or Sass mixin, variable, component, or style test; not detector logic.

public/admin/scss/_colors.scss. 1,574 bytes. Template styling source or Sass mixin, variable, component, or style test; not detector logic.

public/admin/scss/_components/_app-brand.scss. 1,354 bytes. Template styling source or Sass mixin, variable, component, or style test; not detector logic.

public/admin/scss/_components/_avatar.scss. 2,697 bytes. Template styling source or Sass mixin, variable, component, or style test; not detector logic.

public/admin/scss/_components/_base.scss. 3,022 bytes. Template styling source or Sass mixin, variable, component, or style test; not detector logic.

public/admin/scss/_components/_common.scss. 4,139 bytes. Template styling source or Sass mixin, variable, component, or style test; not detector logic.

public/admin/scss/_components/_footer.scss. 663 bytes. Template styling source or Sass mixin, variable, component, or style test; not detector logic.

public/admin/scss/_components/_include.scss. 351 bytes. Template styling source or Sass mixin, variable, component, or style test; not detector logic.

public/admin/scss/_components/_layout.scss. 12,458 bytes. Template styling source or Sass mixin, variable, component, or style test; not detector logic.

public/admin/scss/_components/_menu.scss. 7,402 bytes. Template styling source or Sass mixin, variable, component, or style test; not detector logic.

public/admin/scss/_components/_mixins.scss. 152 bytes. Template styling source or Sass mixin, variable, component, or style test; not detector logic.

public/admin/scss/_components/_text-divider.scss. 1,669 bytes. Template styling source or Sass mixin, variable, component, or style test; not detector logic.

public/admin/scss/_components/_variables.scss. 3,036 bytes. Template styling source or Sass mixin, variable, component, or style test; not detector logic.

public/admin/scss/_components/mixins/_app-brand.scss. 521 bytes. Template styling source or Sass mixin, variable, component, or style test; not detector logic.

public/admin/scss/_components/mixins/_avatar.scss. 473 bytes. Template styling source or Sass mixin, variable, component, or style test; not detector logic.

public/admin/scss/_components/mixins/_footer.scss. 894 bytes. Template styling source or Sass mixin, variable, component, or style test; not detector logic.

public/admin/scss/_components/mixins/_menu.scss. 3,414 bytes. Template styling source or Sass mixin, variable, component, or style test; not detector logic.

public/admin/scss/_components/mixins/_misc.scss. 109 bytes. Template styling source or Sass mixin, variable, component, or style test; not detector logic.

public/admin/scss/_components/mixins/_navbar.scss. 1,977 bytes. Template styling source or Sass mixin, variable, component, or style test; not detector logic.

public/admin/scss/_components/mixins/_text-divider.scss. 405 bytes. Template styling source or Sass mixin, variable, component, or style test; not detector logic.

public/admin/scss/_components.scss. 303 bytes. Template styling source or Sass mixin, variable, component, or style test; not detector logic.

public/admin/scss/_custom-variables/_bootstrap-extended.scss. 488 bytes. Template styling source or Sass mixin, variable, component, or style test; not detector logic.

public/admin/scss/_custom-variables/_components.scss. 430 bytes. Template styling source or Sass mixin, variable, component, or style test; not detector logic.

public/admin/scss/_custom-variables/_libs.scss. 458 bytes. Template styling source or Sass mixin, variable, component, or style test; not detector logic.

public/admin/scss/_custom-variables/_pages.scss. 457 bytes. Template styling source or Sass mixin, variable, component, or style test; not detector logic.

public/admin/scss/_custom-variables/_support.scss. 117 bytes. Template styling source or Sass mixin, variable, component, or style test; not detector logic.

public/admin/scss/_theme/_common.scss. 2,118 bytes. Template styling source or Sass mixin, variable, component, or style test; not detector logic.

public/admin/scss/_theme/_theme.scss. 2,884 bytes. Template styling source or Sass mixin, variable, component, or style test; not detector logic.

public/admin/scss/core.scss. 91 bytes. Template styling source or Sass mixin, variable, component, or style test; not detector logic.

public/admin/scss/pages/page-account-settings.scss. 303 bytes. Template styling source or Sass mixin, variable, component, or style test; not detector logic.

public/admin/scss/pages/page-auth.scss. 15,656 bytes. Template styling source or Sass mixin, variable, component, or style test; not detector logic.

public/admin/scss/pages/page-icons.scss. 703 bytes. Template styling source or Sass mixin, variable, component, or style test; not detector logic.

public/admin/scss/pages/page-misc.scss. 419 bytes. Template styling source or Sass mixin, variable, component, or style test; not detector logic.

public/admin/scss/theme-default.scss. 3,038 bytes. Template styling source or Sass mixin, variable, component, or style test; not detector logic.

public/admin/tasks/build.js. 5,225 bytes. Browser interaction, charting, template, or frontend tooling JavaScript.

public/admin/tasks/prod.js. 1,492 bytes. Browser interaction, charting, template, or frontend tooling JavaScript.

public/admin/webpack.config.js. 3,769 bytes. Browser interaction, charting, template, or frontend tooling JavaScript.

public/admin/yarn.lock. 335,900 bytes. Supporting project or template asset; not a separately launched IDS component.

public/favicon.ico. 0 bytes. Static website image, branding, illustration, favicon, or vendor icon asset.

public/images/logo.png. 51,825 bytes. Static website image, branding, illustration, favicon, or vendor icon asset.

public/images/pic1.jpg. 46,475 bytes. Static website image, branding, illustration, favicon, or vendor icon asset.

public/images/pic2.jpg. 96,473 bytes. Static website image, branding, illustration, favicon, or vendor icon asset.

public/images/profile.jpeg. 7,060 bytes. Static website image, branding, illustration, favicon, or vendor icon asset.

public/images/profile.webp. 13,604 bytes. Static website image, branding, illustration, favicon, or vendor icon asset.

public/index.php. 543 bytes. Laravel public HTTP front controller.

public/landing/assets/css/style.css. 29,960 bytes. Prebuilt website or template stylesheet.

public/landing/assets/images/3d-render-1-min.jpg. 103,519 bytes. Static website image, branding, illustration, favicon, or vendor icon asset.

public/landing/assets/images/about_2-min.jpg. 129,857 bytes. Static website image, branding, illustration, favicon, or vendor icon asset.

public/landing/assets/images/arch-line-reverse.svg. 221 bytes. Static website image, branding, illustration, favicon, or vendor icon asset.

public/landing/assets/images/arch-line.svg. 216 bytes. Static website image, branding, illustration, favicon, or vendor icon asset.

public/landing/assets/images/astronaut.svg. 17,757 bytes. Static website image, branding, illustration, favicon, or vendor icon asset.

public/landing/assets/images/block_squares.svg. 507 bytes. Static website image, branding, illustration, favicon, or vendor icon asset.

public/landing/assets/images/business/business-img-1-min.jpg. 96,019 bytes. Static website image, branding, illustration, favicon, or vendor icon asset.

public/landing/assets/images/business/business-img-10-min.jpg. 69,225 bytes. Static website image, branding, illustration, favicon, or vendor icon asset.

public/landing/assets/images/business/business-img-2-min.jpg. 89,467 bytes. Static website image, branding, illustration, favicon, or vendor icon asset.

public/landing/assets/images/business/business-img-3-min.jpg. 123,425 bytes. Static website image, branding, illustration, favicon, or vendor icon asset.

public/landing/assets/images/business/business-img-4-min.jpg. 75,090 bytes. Static website image, branding, illustration, favicon, or vendor icon asset.

public/landing/assets/images/business/business-img-5-min.jpg. 65,036 bytes. Static website image, branding, illustration, favicon, or vendor icon asset.

public/landing/assets/images/business/business-img-6-min.jpg. 118,255 bytes. Static website image, branding, illustration, favicon, or vendor icon asset.

public/landing/assets/images/business/business-img-7-min.jpg. 61,980 bytes. Static website image, branding, illustration, favicon, or vendor icon asset.

public/landing/assets/images/business/business-img-8-min.jpg. 52,185 bytes. Static website image, branding, illustration, favicon, or vendor icon asset.

public/landing/assets/images/business/business-img-9-min.jpg. 67,941 bytes. Static website image, branding, illustration, favicon, or vendor icon asset.

public/landing/assets/images/card-expenses.png. 42,266 bytes. Static website image, branding, illustration, favicon, or vendor icon asset.

public/landing/assets/images/Facebook_icon.svg. 1,953 bytes. Static website image, branding, illustration, favicon, or vendor icon asset.

public/landing/assets/images/Google_icon.svg. 742 bytes. Static website image, branding, illustration, favicon, or vendor icon asset.

public/landing/assets/images/hero-bg-1-min.jpg. 151,462 bytes. Static website image, branding, illustration, favicon, or vendor icon asset.

public/landing/assets/images/hero-bg-2-min.jpg. 193,295 bytes. Static website image, branding, illustration, favicon, or vendor icon asset.

public/landing/assets/images/hero-img-1-min.jpg. 184,823 bytes. Static website image, branding, illustration, favicon, or vendor icon asset.

public/landing/assets/images/hero-img-2-min.jpg. 164,809 bytes. Static website image, branding, illustration, favicon, or vendor icon asset.

public/landing/assets/images/img-1-min.jpg. 89,423 bytes. Static website image, branding, illustration, favicon, or vendor icon asset.

public/landing/assets/images/img-10-min.jpg. 290,517 bytes. Static website image, branding, illustration, favicon, or vendor icon asset.

public/landing/assets/images/img-11-min.jpg. 47,824 bytes. Static website image, branding, illustration, favicon, or vendor icon asset.

public/landing/assets/images/img-12-min.jpg. 168,283 bytes. Static website image, branding, illustration, favicon, or vendor icon asset.

public/landing/assets/images/img-13-min.jpg. 159,881 bytes. Static website image, branding, illustration, favicon, or vendor icon asset.

public/landing/assets/images/img-14-min.jpg. 37,106 bytes. Static website image, branding, illustration, favicon, or vendor icon asset.

public/landing/assets/images/img-15-min.jpg. 75,142 bytes. Static website image, branding, illustration, favicon, or vendor icon asset.

public/landing/assets/images/img-16-min.jpg. 152,491 bytes. Static website image, branding, illustration, favicon, or vendor icon asset.

public/landing/assets/images/img-2-min.jpg. 17,595 bytes. Static website image, branding, illustration, favicon, or vendor icon asset.

public/landing/assets/images/img-3-min.jpg. 53,386 bytes. Static website image, branding, illustration, favicon, or vendor icon asset.

public/landing/assets/images/img-4-min.jpg. 224,764 bytes. Static website image, branding, illustration, favicon, or vendor icon asset.

public/landing/assets/images/img-5-min.jpg. 259,938 bytes. Static website image, branding, illustration, favicon, or vendor icon asset.

public/landing/assets/images/img-6-min.jpg. 152,410 bytes. Static website image, branding, illustration, favicon, or vendor icon asset.

public/landing/assets/images/img-7-min.jpg. 154,178 bytes. Static website image, branding, illustration, favicon, or vendor icon asset.

public/landing/assets/images/img-8-min.jpg. 109,381 bytes. Static website image, branding, illustration, favicon, or vendor icon asset.

public/landing/assets/images/img-9-min.jpg. 193,946 bytes. Static website image, branding, illustration, favicon, or vendor icon asset.

public/landing/assets/images/img1-min.jpg. 169,835 bytes. Static website image, branding, illustration, favicon, or vendor icon asset.

public/landing/assets/images/logo/actual-size/logo-air-bnb__black.svg. 5,853 bytes. Static website image, branding, illustration, favicon, or vendor icon asset.

public/landing/assets/images/logo/actual-size/logo-american-apparel__black.svg. 11,875 bytes. Static website image, branding, illustration, favicon, or vendor icon asset.

public/landing/assets/images/logo/actual-size/logo-apple__black.svg. 1,560 bytes. Static website image, branding, illustration, favicon, or vendor icon asset.

public/landing/assets/images/logo/actual-size/logo-ebay__black-1.svg. 4,475 bytes. Static website image, branding, illustration, favicon, or vendor icon asset.

public/landing/assets/images/logo/actual-size/logo-ebay__black.svg. 2,089 bytes. Static website image, branding, illustration, favicon, or vendor icon asset.

public/landing/assets/images/logo/actual-size/logo-google__black.svg. 2,792 bytes. Static website image, branding, illustration, favicon, or vendor icon asset.

public/landing/assets/images/logo/actual-size/logo-ibm__black.svg. 2,512 bytes. Static website image, branding, illustration, favicon, or vendor icon asset.

public/landing/assets/images/logo/actual-size/logo-invision__black.svg. 3,393 bytes. Static website image, branding, illustration, favicon, or vendor icon asset.

public/landing/assets/images/logo/actual-size/logo-microsoft_black.svg. 6,573 bytes. Static website image, branding, illustration, favicon, or vendor icon asset.

public/landing/assets/images/logo/actual-size/logo-netflix__black.svg. 1,450 bytes. Static website image, branding, illustration, favicon, or vendor icon asset.

public/landing/assets/images/logo/actual-size/logo-uber__black.svg. 3,842 bytes. Static website image, branding, illustration, favicon, or vendor icon asset.

public/landing/assets/images/logo/actual-size/logo-youtube__black.svg. 5,489 bytes. Static website image, branding, illustration, favicon, or vendor icon asset.

public/landing/assets/images/logo/logo-air-bnb.svg. 5,862 bytes. Static website image, branding, illustration, favicon, or vendor icon asset.

public/landing/assets/images/logo/logo-air-bnb__black.svg. 5,860 bytes. Static website image, branding, illustration, favicon, or vendor icon asset.

public/landing/assets/images/logo/logo-american-apparel.svg. 11,894 bytes. Static website image, branding, illustration, favicon, or vendor icon asset.

public/landing/assets/images/logo/logo-american-apparel__black.svg. 11,892 bytes. Static website image, branding, illustration, favicon, or vendor icon asset.

public/landing/assets/images/logo/logo-apple.svg. 1,593 bytes. Static website image, branding, illustration, favicon, or vendor icon asset.

public/landing/assets/images/logo/logo-apple__black.svg. 1,593 bytes. Static website image, branding, illustration, favicon, or vendor icon asset.

public/landing/assets/images/logo/logo-ebay-1.svg. 4,515 bytes. Static website image, branding, illustration, favicon, or vendor icon asset.

public/landing/assets/images/logo/logo-ebay.svg. 2,130 bytes. Static website image, branding, illustration, favicon, or vendor icon asset.

public/landing/assets/images/logo/logo-ebay__black-1.svg. 4,512 bytes. Static website image, branding, illustration, favicon, or vendor icon asset.

public/landing/assets/images/logo/logo-ebay__black.svg. 2,122 bytes. Static website image, branding, illustration, favicon, or vendor icon asset.

public/landing/assets/images/logo/logo-google.svg. 2,818 bytes. Static website image, branding, illustration, favicon, or vendor icon asset.

public/landing/assets/images/logo/logo-google__black.svg. 2,810 bytes. Static website image, branding, illustration, favicon, or vendor icon asset.

public/landing/assets/images/logo/logo-ibm.svg. 2,563 bytes. Static website image, branding, illustration, favicon, or vendor icon asset.

public/landing/assets/images/logo/logo-ibm__black.svg. 2,561 bytes. Static website image, branding, illustration, favicon, or vendor icon asset.

public/landing/assets/images/logo/logo-invision.svg. 3,414 bytes. Static website image, branding, illustration, favicon, or vendor icon asset.

public/landing/assets/images/logo/logo-invision__black.svg. 3,402 bytes. Static website image, branding, illustration, favicon, or vendor icon asset.

public/landing/assets/images/logo/logo-microsoft.svg. 6,606 bytes. Static website image, branding, illustration, favicon, or vendor icon asset.

public/landing/assets/images/logo/logo-microsoft__black.svg. 6,599 bytes. Static website image, branding, illustration, favicon, or vendor icon asset.

public/landing/assets/images/logo/logo-netflix.svg. 1,465 bytes. Static website image, branding, illustration, favicon, or vendor icon asset.

public/landing/assets/images/logo/logo-netflix__black.svg. 1,463 bytes. Static website image, branding, illustration, favicon, or vendor icon asset.

public/landing/assets/images/logo/logo-uber.svg. 3,871 bytes. Static website image, branding, illustration, favicon, or vendor icon asset.

public/landing/assets/images/logo/logo-uber__black.svg. 3,859 bytes. Static website image, branding, illustration, favicon, or vendor icon asset.

public/landing/assets/images/logo/logo-youtube.svg. 5,507 bytes. Static website image, branding, illustration, favicon, or vendor icon asset.

public/landing/assets/images/logo/logo-youtube__black.svg. 5,500 bytes. Static website image, branding, illustration, favicon, or vendor icon asset.

public/landing/assets/images/logo-dark.svg. 4,240 bytes. Static website image, branding, illustration, favicon, or vendor icon asset.

public/landing/assets/images/logo-light.svg. 4,228 bytes. Static website image, branding, illustration, favicon, or vendor icon asset.

public/landing/assets/images/person-sq-1-min.jpg. 82,709 bytes. Static website image, branding, illustration, favicon, or vendor icon asset.

public/landing/assets/images/person-sq-2-min.jpg. 61,907 bytes. Static website image, branding, illustration, favicon, or vendor icon asset.

public/landing/assets/images/person-sq-3-min.jpg. 67,050 bytes. Static website image, branding, illustration, favicon, or vendor icon asset.

public/landing/assets/images/person-sq-4-min.jpg. 86,300 bytes. Static website image, branding, illustration, favicon, or vendor icon asset.

public/landing/assets/images/person-sq-5-min.jpg. 105,995 bytes. Static website image, branding, illustration, favicon, or vendor icon asset.

public/landing/assets/images/person-sq-6-min.jpg. 122,815 bytes. Static website image, branding, illustration, favicon, or vendor icon asset.

public/landing/assets/images/person-sq-7-min.jpg. 74,641 bytes. Static website image, branding, illustration, favicon, or vendor icon asset.

public/landing/assets/images/person-sq-8-min.jpg. 97,315 bytes. Static website image, branding, illustration, favicon, or vendor icon asset.

public/landing/assets/images/person-sq-9-min.jpg. 100,200 bytes. Static website image, branding, illustration, favicon, or vendor icon asset.

public/landing/assets/images/portfolio-details-img-1-min.jpg. 309,620 bytes. Static website image, branding, illustration, favicon, or vendor icon asset.

public/landing/assets/images/portfolio-details-img-2-min.jpg. 75,171 bytes. Static website image, branding, illustration, favicon, or vendor icon asset.

public/landing/assets/images/portfolio-details-img-3-min.jpg. 121,185 bytes. Static website image, branding, illustration, favicon, or vendor icon asset.

public/landing/assets/images/portfolio-details-img-4-min.jpg. 49,593 bytes. Static website image, branding, illustration, favicon, or vendor icon asset.

public/landing/assets/images/portfolio-details-img-5-min.jpg. 286,291 bytes. Static website image, branding, illustration, favicon, or vendor icon asset.

public/landing/assets/images/portfolio-details-img-6-min.jpg. 273,225 bytes. Static website image, branding, illustration, favicon, or vendor icon asset.

public/landing/assets/images/quote.svg. 309 bytes. Static website image, branding, illustration, favicon, or vendor icon asset.

public/landing/assets/images/scribble-1.svg. 4,232 bytes. Static website image, branding, illustration, favicon, or vendor icon asset.

public/landing/assets/images/scribble-2.svg. 1,212 bytes. Static website image, branding, illustration, favicon, or vendor icon asset.

public/landing/assets/js/custom.js. 16,397 bytes. Browser interaction, charting, template, or frontend tooling JavaScript.

public/landing/assets/vendors/aos/aos.css. 28,023 bytes. Prebuilt website or template stylesheet.

public/landing/assets/vendors/aos/aos.js. 14,690 bytes. Browser interaction, charting, template, or frontend tooling JavaScript.

public/landing/assets/vendors/bootstrap/bootstrap.bundle.min.js. 80,721 bytes. Browser interaction, charting, template, or frontend tooling JavaScript.

public/landing/assets/vendors/bootstrap/bootstrap.min.css. 232,802 bytes. Prebuilt website or template stylesheet.

public/landing/assets/vendors/bootstrap/bs-darkmode.js. 2,544 bytes. Browser interaction, charting, template, or frontend tooling JavaScript.

public/landing/assets/vendors/bootstrap/scss/_accordion.scss. 5,077 bytes. Template styling source or Sass mixin, variable, component, or style test; not detector logic.

public/landing/assets/vendors/bootstrap/scss/_alert.scss. 2,073 bytes. Template styling source or Sass mixin, variable, component, or style test; not detector logic.

public/landing/assets/vendors/bootstrap/scss/_badge.scss. 1,118 bytes. Template styling source or Sass mixin, variable, component, or style test; not detector logic.

public/landing/assets/vendors/bootstrap/scss/_breadcrumb.scss. 1,751 bytes. Template styling source or Sass mixin, variable, component, or style test; not detector logic.

public/landing/assets/vendors/bootstrap/scss/_button-group.scss. 3,201 bytes. Template styling source or Sass mixin, variable, component, or style test; not detector logic.

public/landing/assets/vendors/bootstrap/scss/_buttons.scss. 7,057 bytes. Template styling source or Sass mixin, variable, component, or style test; not detector logic.

public/landing/assets/vendors/bootstrap/scss/_card.scss. 6,979 bytes. Template styling source or Sass mixin, variable, component, or style test; not detector logic.

public/landing/assets/vendors/bootstrap/scss/_carousel.scss. 5,862 bytes. Template styling source or Sass mixin, variable, component, or style test; not detector logic.

public/landing/assets/vendors/bootstrap/scss/_close.scss. 2,018 bytes. Template styling source or Sass mixin, variable, component, or style test; not detector logic.

public/landing/assets/vendors/bootstrap/scss/_containers.scss. 1,201 bytes. Template styling source or Sass mixin, variable, component, or style test; not detector logic.

public/landing/assets/vendors/bootstrap/scss/_dropdown.scss. 8,093 bytes. Template styling source or Sass mixin, variable, component, or style test; not detector logic.

public/landing/assets/vendors/bootstrap/scss/_forms.scss. 256 bytes. Template styling source or Sass mixin, variable, component, or style test; not detector logic.

public/landing/assets/vendors/bootstrap/scss/_functions.scss. 10,554 bytes. Template styling source or Sass mixin, variable, component, or style test; not detector logic.

public/landing/assets/vendors/bootstrap/scss/_grid.scss. 683 bytes. Template styling source or Sass mixin, variable, component, or style test; not detector logic.

public/landing/assets/vendors/bootstrap/scss/_helpers.scss. 353 bytes. Template styling source or Sass mixin, variable, component, or style test; not detector logic.

public/landing/assets/vendors/bootstrap/scss/_images.scss. 1,158 bytes. Template styling source or Sass mixin, variable, component, or style test; not detector logic.

public/landing/assets/vendors/bootstrap/scss/_list-group.scss. 6,852 bytes. Template styling source or Sass mixin, variable, component, or style test; not detector logic.

public/landing/assets/vendors/bootstrap/scss/_maps.scss. 6,005 bytes. Template styling source or Sass mixin, variable, component, or style test; not detector logic.

public/landing/assets/vendors/bootstrap/scss/_mixins.scss. 875 bytes. Template styling source or Sass mixin, variable, component, or style test; not detector logic.

public/landing/assets/vendors/bootstrap/scss/_modal.scss. 7,662 bytes. Template styling source or Sass mixin, variable, component, or style test; not detector logic.

public/landing/assets/vendors/bootstrap/scss/_nav.scss. 5,211 bytes. Template styling source or Sass mixin, variable, component, or style test; not detector logic.

public/landing/assets/vendors/bootstrap/scss/_navbar.scss. 9,155 bytes. Template styling source or Sass mixin, variable, component, or style test; not detector logic.

public/landing/assets/vendors/bootstrap/scss/_offcanvas.scss. 4,653 bytes. Template styling source or Sass mixin, variable, component, or style test; not detector logic.

public/landing/assets/vendors/bootstrap/scss/_pagination.scss. 3,947 bytes. Template styling source or Sass mixin, variable, component, or style test; not detector logic.

public/landing/assets/vendors/bootstrap/scss/_placeholders.scss. 859 bytes. Template styling source or Sass mixin, variable, component, or style test; not detector logic.

public/landing/assets/vendors/bootstrap/scss/_popover.scss. 6,907 bytes. Template styling source or Sass mixin, variable, component, or style test; not detector logic.

public/landing/assets/vendors/bootstrap/scss/_progress.scss. 2,016 bytes. Template styling source or Sass mixin, variable, component, or style test; not detector logic.

public/landing/assets/vendors/bootstrap/scss/_reboot.scss. 12,438 bytes. Template styling source or Sass mixin, variable, component, or style test; not detector logic.

public/landing/assets/vendors/bootstrap/scss/_root.scss. 6,924 bytes. Template styling source or Sass mixin, variable, component, or style test; not detector logic.

public/landing/assets/vendors/bootstrap/scss/_spinners.scss. 2,429 bytes. Template styling source or Sass mixin, variable, component, or style test; not detector logic.

public/landing/assets/vendors/bootstrap/scss/_tables.scss. 4,946 bytes. Template styling source or Sass mixin, variable, component, or style test; not detector logic.

public/landing/assets/vendors/bootstrap/scss/_toasts.scss. 2,490 bytes. Template styling source or Sass mixin, variable, component, or style test; not detector logic.

public/landing/assets/vendors/bootstrap/scss/_tooltip.scss. 4,281 bytes. Template styling source or Sass mixin, variable, component, or style test; not detector logic.

public/landing/assets/vendors/bootstrap/scss/_transitions.scss. 425 bytes. Template styling source or Sass mixin, variable, component, or style test; not detector logic.

public/landing/assets/vendors/bootstrap/scss/_type.scss. 1,420 bytes. Template styling source or Sass mixin, variable, component, or style test; not detector logic.

public/landing/assets/vendors/bootstrap/scss/_utilities.scss. 19,235 bytes. Template styling source or Sass mixin, variable, component, or style test; not detector logic.

public/landing/assets/vendors/bootstrap/scss/_variables-dark.scss. 5,057 bytes. Template styling source or Sass mixin, variable, component, or style test; not detector logic.

public/landing/assets/vendors/bootstrap/scss/_variables.scss. 76,324 bytes. Template styling source or Sass mixin, variable, component, or style test; not detector logic.

public/landing/assets/vendors/bootstrap/scss/bootstrap-grid.scss. 1,183 bytes. Template styling source or Sass mixin, variable, component, or style test; not detector logic.

public/landing/assets/vendors/bootstrap/scss/bootstrap-reboot.scss. 189 bytes. Template styling source or Sass mixin, variable, component, or style test; not detector logic.

public/landing/assets/vendors/bootstrap/scss/bootstrap-utilities.scss. 306 bytes. Template styling source or Sass mixin, variable, component, or style test; not detector logic.

public/landing/assets/vendors/bootstrap/scss/bootstrap.scss. 938 bytes. Template styling source or Sass mixin, variable, component, or style test; not detector logic.

public/landing/assets/vendors/bootstrap/scss/forms/_floating-labels.scss. 2,688 bytes. Template styling source or Sass mixin, variable, component, or style test; not detector logic.

public/landing/assets/vendors/bootstrap/scss/forms/_form-check.scss. 4,854 bytes. Template styling source or Sass mixin, variable, component, or style test; not detector logic.

public/landing/assets/vendors/bootstrap/scss/forms/_form-control.scss. 6,590 bytes. Template styling source or Sass mixin, variable, component, or style test; not detector logic.

public/landing/assets/vendors/bootstrap/scss/forms/_form-range.scss. 2,796 bytes. Template styling source or Sass mixin, variable, component, or style test; not detector logic.

public/landing/assets/vendors/bootstrap/scss/forms/_form-select.scss. 2,488 bytes. Template styling source or Sass mixin, variable, component, or style test; not detector logic.

public/landing/assets/vendors/bootstrap/scss/forms/_form-text.scss. 219 bytes. Template styling source or Sass mixin, variable, component, or style test; not detector logic.

public/landing/assets/vendors/bootstrap/scss/forms/_input-group.scss. 3,899 bytes. Template styling source or Sass mixin, variable, component, or style test; not detector logic.

public/landing/assets/vendors/bootstrap/scss/forms/_labels.scss. 1,142 bytes. Template styling source or Sass mixin, variable, component, or style test; not detector logic.

public/landing/assets/vendors/bootstrap/scss/forms/_validation.scss. 478 bytes. Template styling source or Sass mixin, variable, component, or style test; not detector logic.

public/landing/assets/vendors/bootstrap/scss/helpers/_clearfix.scss. 37 bytes. Template styling source or Sass mixin, variable, component, or style test; not detector logic.

public/landing/assets/vendors/bootstrap/scss/helpers/_color-bg.scss. 403 bytes. Template styling source or Sass mixin, variable, component, or style test; not detector logic.

public/landing/assets/vendors/bootstrap/scss/helpers/_colored-links.scss. 1,755 bytes. Template styling source or Sass mixin, variable, component, or style test; not detector logic.

public/landing/assets/vendors/bootstrap/scss/helpers/_focus-ring.scss. 385 bytes. Template styling source or Sass mixin, variable, component, or style test; not detector logic.

public/landing/assets/vendors/bootstrap/scss/helpers/_icon-link.scss. 605 bytes. Template styling source or Sass mixin, variable, component, or style test; not detector logic.

public/landing/assets/vendors/bootstrap/scss/helpers/_position.scss. 621 bytes. Template styling source or Sass mixin, variable, component, or style test; not detector logic.

public/landing/assets/vendors/bootstrap/scss/helpers/_ratio.scss. 399 bytes. Template styling source or Sass mixin, variable, component, or style test; not detector logic.

public/landing/assets/vendors/bootstrap/scss/helpers/_stacks.scss. 245 bytes. Template styling source or Sass mixin, variable, component, or style test; not detector logic.

public/landing/assets/vendors/bootstrap/scss/helpers/_stretched-link.scss. 223 bytes. Template styling source or Sass mixin, variable, component, or style test; not detector logic.

public/landing/assets/vendors/bootstrap/scss/helpers/_text-truncation.scss. 73 bytes. Template styling source or Sass mixin, variable, component, or style test; not detector logic.

public/landing/assets/vendors/bootstrap/scss/helpers/_visually-hidden.scss. 136 bytes. Template styling source or Sass mixin, variable, component, or style test; not detector logic.

public/landing/assets/vendors/bootstrap/scss/helpers/_vr.scss. 160 bytes. Template styling source or Sass mixin, variable, component, or style test; not detector logic.

public/landing/assets/vendors/bootstrap/scss/mixins/_alert.scss. 525 bytes. Template styling source or Sass mixin, variable, component, or style test; not detector logic.

public/landing/assets/vendors/bootstrap/scss/mixins/_backdrop.scss. 328 bytes. Template styling source or Sass mixin, variable, component, or style test; not detector logic.

public/landing/assets/vendors/bootstrap/scss/mixins/_banner.scss. 223 bytes. Template styling source or Sass mixin, variable, component, or style test; not detector logic.

public/landing/assets/vendors/bootstrap/scss/mixins/_border-radius.scss. 2,031 bytes. Template styling source or Sass mixin, variable, component, or style test; not detector logic.

public/landing/assets/vendors/bootstrap/scss/mixins/_box-shadow.scss. 398 bytes. Template styling source or Sass mixin, variable, component, or style test; not detector logic.

public/landing/assets/vendors/bootstrap/scss/mixins/_breakpoints.scss. 4,580 bytes. Template styling source or Sass mixin, variable, component, or style test; not detector logic.

public/landing/assets/vendors/bootstrap/scss/mixins/_buttons.scss. 3,220 bytes. Template styling source or Sass mixin, variable, component, or style test; not detector logic.

public/landing/assets/vendors/bootstrap/scss/mixins/_caret.scss. 1,587 bytes. Template styling source or Sass mixin, variable, component, or style test; not detector logic.

public/landing/assets/vendors/bootstrap/scss/mixins/_clearfix.scss. 147 bytes. Template styling source or Sass mixin, variable, component, or style test; not detector logic.

public/landing/assets/vendors/bootstrap/scss/mixins/_color-mode.scss. 447 bytes. Template styling source or Sass mixin, variable, component, or style test; not detector logic.

public/landing/assets/vendors/bootstrap/scss/mixins/_color-scheme.scss. 167 bytes. Template styling source or Sass mixin, variable, component, or style test; not detector logic.

public/landing/assets/vendors/bootstrap/scss/mixins/_container.scss. 410 bytes. Template styling source or Sass mixin, variable, component, or style test; not detector logic.

public/landing/assets/vendors/bootstrap/scss/mixins/_deprecate.scss. 613 bytes. Template styling source or Sass mixin, variable, component, or style test; not detector logic.

public/landing/assets/vendors/bootstrap/scss/mixins/_forms.scss. 4,578 bytes. Template styling source or Sass mixin, variable, component, or style test; not detector logic.

public/landing/assets/vendors/bootstrap/scss/mixins/_gradients.scss. 1,956 bytes. Template styling source or Sass mixin, variable, component, or style test; not detector logic.

public/landing/assets/vendors/bootstrap/scss/mixins/_grid.scss. 4,734 bytes. Template styling source or Sass mixin, variable, component, or style test; not detector logic.

public/landing/assets/vendors/bootstrap/scss/mixins/_image.scss. 395 bytes. Template styling source or Sass mixin, variable, component, or style test; not detector logic.

public/landing/assets/vendors/bootstrap/scss/mixins/_list-group.scss. 581 bytes. Template styling source or Sass mixin, variable, component, or style test; not detector logic.

public/landing/assets/vendors/bootstrap/scss/mixins/_lists.scss. 168 bytes. Template styling source or Sass mixin, variable, component, or style test; not detector logic.

public/landing/assets/vendors/bootstrap/scss/mixins/_pagination.scss. 387 bytes. Template styling source or Sass mixin, variable, component, or style test; not detector logic.

public/landing/assets/vendors/bootstrap/scss/mixins/_reset-text.scss. 495 bytes. Template styling source or Sass mixin, variable, component, or style test; not detector logic.

public/landing/assets/vendors/bootstrap/scss/mixins/_resize.scss. 202 bytes. Template styling source or Sass mixin, variable, component, or style test; not detector logic.

public/landing/assets/vendors/bootstrap/scss/mixins/_table-variants.scss. 1,101 bytes. Template styling source or Sass mixin, variable, component, or style test; not detector logic.

public/landing/assets/vendors/bootstrap/scss/mixins/_text-truncate.scss. 168 bytes. Template styling source or Sass mixin, variable, component, or style test; not detector logic.

public/landing/assets/vendors/bootstrap/scss/mixins/_transition.scss. 661 bytes. Template styling source or Sass mixin, variable, component, or style test; not detector logic.

public/landing/assets/vendors/bootstrap/scss/mixins/_utilities.scss. 3,384 bytes. Template styling source or Sass mixin, variable, component, or style test; not detector logic.

public/landing/assets/vendors/bootstrap/scss/mixins/_visually-hidden.scss. 1,110 bytes. Template styling source or Sass mixin, variable, component, or style test; not detector logic.

public/landing/assets/vendors/bootstrap/scss/tests/jasmine.js. 449 bytes. Browser interaction, charting, template, or frontend tooling JavaScript.

public/landing/assets/vendors/bootstrap/scss/tests/mixins/_auto-import-of-variables-dark.test.scss. 276 bytes. Template styling source or Sass mixin, variable, component, or style test; not detector logic.

public/landing/assets/vendors/bootstrap/scss/tests/mixins/_color-modes.test.scss. 1,859 bytes. Template styling source or Sass mixin, variable, component, or style test; not detector logic.

public/landing/assets/vendors/bootstrap/scss/tests/mixins/_media-query-color-mode-full.test.scss. 246 bytes. Template styling source or Sass mixin, variable, component, or style test; not detector logic.

public/landing/assets/vendors/bootstrap/scss/tests/mixins/_utilities.test.scss. 8,773 bytes. Template styling source or Sass mixin, variable, component, or style test; not detector logic.

public/landing/assets/vendors/bootstrap/scss/tests/sass-true/register.js. 377 bytes. Browser interaction, charting, template, or frontend tooling JavaScript.

public/landing/assets/vendors/bootstrap/scss/tests/sass-true/runner.js. 450 bytes. Browser interaction, charting, template, or frontend tooling JavaScript.

public/landing/assets/vendors/bootstrap/scss/tests/utilities/_api.test.scss. 1,462 bytes. Template styling source or Sass mixin, variable, component, or style test; not detector logic.

public/landing/assets/vendors/bootstrap/scss/utilities/_api.scss. 1,737 bytes. Template styling source or Sass mixin, variable, component, or style test; not detector logic.

public/landing/assets/vendors/bootstrap-icons/font/bootstrap-icons.css. 98,255 bytes. Prebuilt website or template stylesheet.

public/landing/assets/vendors/bootstrap-icons/font/bootstrap-icons.json. 52,358 bytes. Dependency, template, icon, or tooling metadata; context determined by its directory.

public/landing/assets/vendors/bootstrap-icons/font/bootstrap-icons.min.css. 85,875 bytes. Prebuilt website or template stylesheet.

public/landing/assets/vendors/bootstrap-icons/font/bootstrap-icons.scss. 57,755 bytes. Template styling source or Sass mixin, variable, component, or style test; not detector logic.

public/landing/assets/vendors/bootstrap-icons/font/fonts/bootstrap-icons.woff. 176,032 bytes. Static web-font or icon-font binary used by website styling.

public/landing/assets/vendors/bootstrap-icons/font/fonts/bootstrap-icons.woff2. 130,396 bytes. Static web-font or icon-font binary used by website styling.

public/landing/assets/vendors/glightbox/glightbox.min.css. 13,749 bytes. Prebuilt website or template stylesheet.

public/landing/assets/vendors/glightbox/glightbox.min.js. 56,300 bytes. Browser interaction, charting, template, or frontend tooling JavaScript.

public/landing/assets/vendors/gsap/gsap.min.js. 72,213 bytes. Browser interaction, charting, template, or frontend tooling JavaScript.

public/landing/assets/vendors/imagesloaded/imagesloaded.pkgd.min.js. 5,485 bytes. Browser interaction, charting, template, or frontend tooling JavaScript.

public/landing/assets/vendors/index.html. 207 bytes. Static website template, demonstration page, or distribution placeholder; not necessarily a Laravel route.

public/landing/assets/vendors/isotope/isotope.pkgd.min.js. 35,445 bytes. Browser interaction, charting, template, or frontend tooling JavaScript.

public/landing/assets/vendors/PHPMailer/index.html. 207 bytes. Bundled third-party mail library or supporting metadata; no active IDS mail workflow established.

public/landing/assets/vendors/PHPMailer/src/DSNConfigurator.php. 6,883 bytes. Bundled third-party mail library or supporting metadata; no active IDS mail workflow established.

public/landing/assets/vendors/PHPMailer/src/Exception.php. 1,256 bytes. Bundled third-party mail library or supporting metadata; no active IDS mail workflow established.

public/landing/assets/vendors/PHPMailer/src/index.html. 207 bytes. Bundled third-party mail library or supporting metadata; no active IDS mail workflow established.

public/landing/assets/vendors/PHPMailer/src/OAuth.php. 3,791 bytes. Bundled third-party mail library or supporting metadata; no active IDS mail workflow established.

public/landing/assets/vendors/PHPMailer/src/OAuthTokenProvider.php. 1,538 bytes. Bundled third-party mail library or supporting metadata; no active IDS mail workflow established.

public/landing/assets/vendors/PHPMailer/src/PHPMailer.php. 182,896 bytes. Bundled third-party mail library or supporting metadata; no active IDS mail workflow established.

public/landing/assets/vendors/PHPMailer/src/POP3.php. 12,275 bytes. Bundled third-party mail library or supporting metadata; no active IDS mail workflow established.

public/landing/assets/vendors/PHPMailer/src/SMTP.php. 48,717 bytes. Bundled third-party mail library or supporting metadata; no active IDS mail workflow established.

public/landing/assets/vendors/purecounter/purecounter.js. 7,190 bytes. Browser interaction, charting, template, or frontend tooling JavaScript.

public/landing/assets/vendors/swiper/swiper-bundle.min.css. 18,432 bytes. Prebuilt website or template stylesheet.

public/landing/assets/vendors/swiper/swiper-bundle.min.js. 150,447 bytes. Browser interaction, charting, template, or frontend tooling JavaScript.

public/landing/index.html. 66,211 bytes. Static website template, demonstration page, or distribution placeholder; not necessarily a Laravel route.

public/landing/Readme.md. 594 bytes. Supporting documentation, distribution notes, or license text.

public/ngrok.exe. 31,770,400 bytes. Optional ngrok tunneling executable; no main service automatically launches it.

public/Nova-Bootstrap-1.0.0.zip. 6,840,309 bytes. Packaged historical API or website template distribution; not automatically executed.

public/robots.txt. 24 bytes. Crawler instructions; not an authentication control.

public/sneat-1.0.0.zip. 4,546,185 bytes. Packaged historical API or website template distribution; not automatically executed.

File inventory: resources

resources/css/app.css. 390 bytes. Frontend source asset consumed by the Vite build.

resources/js/app.js. 22 bytes. Frontend source asset consumed by the Vite build.

resources/js/bootstrap.js. 127 bytes. Frontend source asset consumed by the Vite build.

resources/views/admin/dashboard.blade.php. 58,422 bytes. Server-rendered Blade website page or partial; page responsibilities are explained in the website chapter.

resources/views/admin/live-monitor.blade.php. 13,841 bytes. Server-rendered Blade website page or partial; page responsibilities are explained in the website chapter.

resources/views/admin/logs.blade.php. 18,158 bytes. Server-rendered Blade website page or partial; page responsibilities are explained in the website chapter.

resources/views/admin/manage-threats.blade.php. 31,611 bytes. Server-rendered Blade website page or partial; page responsibilities are explained in the website chapter.

resources/views/admin/partials/system-status-alert.blade.php. 909 bytes. Server-rendered Blade website page or partial; page responsibilities are explained in the website chapter.

resources/views/admin/system-status.blade.php. 31,934 bytes. Server-rendered Blade website page or partial; page responsibilities are explained in the website chapter.

resources/views/admin/threat-reports.blade.php. 12,657 bytes. Server-rendered Blade website page or partial; page responsibilities are explained in the website chapter.

resources/views/welcome.blade.php. 44,476 bytes. Server-rendered Blade website page or partial; page responsibilities are explained in the website chapter.

File inventory: routes

routes/api.php. 310 bytes. HTTP or console route registration; determines exposed handler and middleware mapping.

routes/console.php. 210 bytes. HTTP or console route registration; determines exposed handler and middleware mapping.

routes/web.php. 2,509 bytes. HTTP or console route registration; determines exposed handler and middleware mapping.

File inventory: setup

setup/__pycache__/Detection Process.cpython-312.pyc. 13,509 bytes. Generated Python bytecode cache; recreated by Python and not authoritative source.

setup/__pycache__/Detection Process.cpython-314.pyc. 11,460 bytes. Generated Python bytecode cache; recreated by Python and not authoritative source.

setup/__pycache__/Mikrotek PPPoe Sniffer.cpython-312.pyc. 31,769 bytes. Generated Python bytecode cache; recreated by Python and not authoritative source.

setup/__pycache__/Mikrotek PPPoe Sniffer.cpython-314.pyc. 32,349 bytes. Generated Python bytecode cache; recreated by Python and not authoritative source.

setup/__pycache__/netflow_schema.cpython-312.pyc. 567 bytes. Generated Python bytecode cache; recreated by Python and not authoritative source.

setup/__pycache__/Pending Blocker.cpython-312.pyc. 54,675 bytes. Generated Python bytecode cache; recreated by Python and not authoritative source.

setup/__pycache__/Pending Blocker.cpython-314.pyc. 52,479 bytes. Generated Python bytecode cache; recreated by Python and not authoritative source.

setup/data/.gitattributes. 62 bytes. Supporting project or template asset; not a separately launched IDS component.

setup/data/model/ids_ensemble.joblib. 134 bytes. Serialized model artifact or placeholder; inventoried without executing/deserializing. Only the configured filename is loaded at runtime.

setup/data/model/model.joblib. 102,913,220 bytes. Serialized model artifact or placeholder; inventoried without executing/deserializing. Only the configured filename is loaded at runtime.

setup/data/model/netflow_ids.joblib. 57,823,369 bytes. Serialized model artifact or placeholder; inventoried without executing/deserializing. Only the configured filename is loaded at runtime.

setup/data/model/netflow_ids.metrics.json. 3,306 bytes. Stored evaluation metrics and bootstrap-validation warning for a model artifact.

setup/data/README.md. 18 bytes. Setup or model-development guidance.

setup/data/requirements.txt. 197 bytes. Python runtime/development dependency declaration.

setup/Detection Process.py. 9,172 bytes. Current Flask two-stage inference API and direct transactional MySQL observation/incident writer.

setup/ids_ensemble-api-main.zip. 3,311 bytes. Packaged historical API or website template distribution; not automatically executed.

setup/logs/blocked_threat_sync.log. 438 bytes. Supporting project or template asset; not a separately launched IDS component.

setup/logs/merged_ids.log. 51,766,602 bytes. Supporting project or template asset; not a separately launched IDS component.

setup/logs/mikrotik_ids_78.log. 824,072 bytes. Supporting project or template asset; not a separately launched IDS component.

setup/mikroktek cmd.txt. 344 bytes. Manual RouterOS notes for inspecting or removing IDS-owned rules.

setup/Mikrotek PPPoe Sniffer.py. 25,069 bytes. NetFlow v5 UDP collector, PPP identity discovery, numerical aggregation, and inference client.

setup/Pending Blocker.py. 35,566 bytes. Database-polling subscriber firewall reconciler implementing both block and restore.

setup/README.md. 2,025 bytes. Local services, startup sequence, shared database, and incident lifecycle guide.

setup/Start-IDS.bat. 334 bytes. Windows launcher opening the three current Python services in separate consoles.

setup/Training/.ipynb_checkpoints/Check Attr-checkpoint.ipynb. 72 bytes. Automatically saved historical notebook checkpoint; not an active runtime service.

setup/Training/.ipynb_checkpoints/Network Attack Simulation-checkpoint.ipynb. 10,147 bytes. Automatically saved historical notebook checkpoint; not an active runtime service.

setup/Training/.ipynb_checkpoints/Training Model-checkpoint.ipynb. 15,256 bytes. Automatically saved historical notebook checkpoint; not an active runtime service.

setup/Training/__pycache__/train_netflow_model.cpython-312.pyc. 19,131 bytes. Generated Python bytecode cache; recreated by Python and not authoritative source.

setup/Training/__pycache__/training_schema.cpython-312.pyc. 571 bytes. Generated Python bytecode cache; recreated by Python and not authoritative source.

setup/Training/alpha.pdf. 1,018,136 bytes. Supporting project or template asset; not a separately launched IDS component.

setup/Training/alpha_word.docx. 777,734 bytes. Historical research manuscript; its textual contents are preserved in the manuscript appendix.

setup/Training/artifacts/.ipynb_checkpoints/Untitled-checkpoint.ipynb. 72 bytes. Automatically saved historical notebook checkpoint; not an active runtime service.

setup/Training/artifacts/computational_efficiency.pdf. 25,513 bytes. Supporting project or template asset; not a separately launched IDS component.

setup/Training/artifacts/computational_efficiency.png. 97,119 bytes. Static website image, branding, illustration, favicon, or vendor icon asset.

setup/Training/artifacts/dataset_distribution.pdf. 29,002 bytes. Supporting project or template asset; not a separately launched IDS component.

setup/Training/artifacts/dataset_distribution.png. 153,580 bytes. Static website image, branding, illustration, favicon, or vendor icon asset.

setup/Training/artifacts/ids_ensemble.joblib. 102,913,220 bytes. Serialized model artifact or placeholder; inventoried without executing/deserializing. Only the configured filename is loaded at runtime.

setup/Training/artifacts/precision_recall_curve.pdf. 25,751 bytes. Supporting project or template asset; not a separately launched IDS component.

setup/Training/artifacts/precision_recall_curve.png. 132,460 bytes. Static website image, branding, illustration, favicon, or vendor icon asset.

setup/Training/artifacts/research_paper_visualizations.pdf. 46,963 bytes. Supporting project or template asset; not a separately launched IDS component.

setup/Training/artifacts/research_paper_visualizations.png. 753,471 bytes. Static website image, branding, illustration, favicon, or vendor icon asset.

setup/Training/artifacts/roc_curve.pdf. 25,768 bytes. Supporting project or template asset; not a separately launched IDS component.

setup/Training/artifacts/roc_curve.png. 140,608 bytes. Static website image, branding, illustration, favicon, or vendor icon asset.

setup/Training/artifacts/stage_a_performance.pdf. 25,816 bytes. Supporting project or template asset; not a separately launched IDS component.

setup/Training/artifacts/stage_a_performance.png. 148,302 bytes. Static website image, branding, illustration, favicon, or vendor icon asset.

setup/Training/artifacts/stage_b_attack_accuracy.pdf. 30,682 bytes. Supporting project or template asset; not a separately launched IDS component.

setup/Training/artifacts/stage_b_attack_accuracy.png. 194,620 bytes. Static website image, branding, illustration, favicon, or vendor icon asset.

setup/Training/artifacts/stage_b_confusion_matrix.pdf. 34,961 bytes. Supporting project or template asset; not a separately launched IDS component.

setup/Training/artifacts/stage_b_confusion_matrix.png. 336,708 bytes. Static website image, branding, illustration, favicon, or vendor icon asset.

setup/Training/artifacts/Untitled.ipynb. 1,751 bytes. Offline training or inference experiment notebook; summarized in the notebook register.

setup/Training/attack.csv. 58,316,725 bytes. Supporting project or template asset; not a separately launched IDS component.

setup/Training/beta.docx. 21,865 bytes. Historical research manuscript; its textual contents are preserved in the manuscript appendix.

setup/Training/Check Attr.ipynb. 617 bytes. Offline training or inference experiment notebook; summarized in the notebook register.

setup/Training/Dataset/ddos_dataset.pcap_ISCX.csv. 77,349,605 bytes. Offline labeled traffic training/evaluation input; not required for ordinary inference startup.

setup/Training/Dataset/Friday-WorkingHours-Afternoon-PortScan.pcap_ISCX.csv. 76,906,168 bytes. Offline labeled traffic training/evaluation input; not required for ordinary inference startup.

setup/Training/Dataset/Friday-WorkingHours-Morning.pcap.csv. 58,316,725 bytes. Offline labeled traffic training/evaluation input; not required for ordinary inference startup.

setup/Training/Dataset/Monday-WorkingHours.pcap_ISCX.csv. 176,927,918 bytes. Offline labeled traffic training/evaluation input; not required for ordinary inference startup.

setup/Training/Dataset/Thursday-WorkingHours-Afternoon-Infilteration.pcap_ISCX.csv. 83,102,436 bytes. Offline labeled traffic training/evaluation input; not required for ordinary inference startup.

setup/Training/Dataset/Thursday-WorkingHours-Morning-WebAttacks.pcap_ISCX.csv. 52,023,263 bytes. Offline labeled traffic training/evaluation input; not required for ordinary inference startup.

setup/Training/Dataset/Tuesday-WorkingHours.pcap_ISCX.csv. 135,078,995 bytes. Offline labeled traffic training/evaluation input; not required for ordinary inference startup.

setup/Training/Dataset/Wednesday-workingHours.pcap_ISCX.csv. 225,166,395 bytes. Offline labeled traffic training/evaluation input; not required for ordinary inference startup.

setup/Training/Design and Development of a Real-Time Flow-Based Intrusion Detection System Using a Two-Stage Ensemble Model.docx. 1,070,427 bytes. Historical research manuscript; its textual contents are preserved in the manuscript appendix.

setup/Training/Network Attack Simulation.ipynb. 5,832 bytes. Offline training or inference experiment notebook; summarized in the notebook register.

setup/Training/paper 2 development sir jimson.docx. 1,065,687 bytes. Historical research manuscript; its textual contents are preserved in the manuscript appendix.

setup/Training/predicted_attacks_only.csv. 1,390 bytes. Supporting project or template asset; not a separately launched IDS component.

setup/Training/predicted_attacks_thr_080.csv. 4,863 bytes. Supporting project or template asset; not a separately launched IDS component.

setup/Training/README.md. 1,321 bytes. Setup or model-development guidance.

setup/Training/train_netflow_model.py. 15,052 bytes. Maintained fourteen-feature training, validation, serialization, and test metrics pipeline.

setup/Training/Training Model.ipynb. 23,464 bytes. Offline training or inference experiment notebook; summarized in the notebook register.

setup/Training/training_schema.py. 500 bytes. Ordered feature names and mikrotik-netflow-v1 schema identifier.

setup/Training/Two-Stage Ensemble Machine Learning for Network Intrusion Detection.docx. 795,451 bytes. Historical research manuscript; its textual contents are preserved in the manuscript appendix.

setup/Training/Two-Stage Ensemble Machine Learning for Network Intrusion Detection.pdf. 1,354,622 bytes. Supporting project or template asset; not a separately launched IDS component.

setup/Working Backup Pending Blocker.txt. 29,618 bytes. Historical backup of the enforcement script; not the current launched module.

File inventory: tests

tests/Feature/ExampleTest.php. 147 bytes. Regression test source or shared test harness; coverage is documented in the testing chapter.

tests/Feature/PppoeBlockingIdentityTest.php. 4,995 bytes. Regression test source or shared test harness; coverage is documented in the testing chapter.

tests/Feature/ViewOnlyAccessTest.php. 2,423 bytes. Regression test source or shared test harness; coverage is documented in the testing chapter.

tests/Pest.php. 1,584 bytes. Regression test source or shared test harness; coverage is documented in the testing chapter.

tests/TestCase.php. 142 bytes. Regression test source or shared test harness; coverage is documented in the testing chapter.

tests/Unit/ExampleTest.php. 81 bytes. Regression test source or shared test harness; coverage is documented in the testing chapter.

File inventory: tools

tools/build_system_documentation.py. 115,836 bytes. Offline text/PDF documentation generator; not part of website request handling.

Grouped private, dependency, and generated directories

.env: private environment or local operational file; values and contents not published.

.env.example: private environment or local operational file; values and contents not published.

.git: installed dependencies, editor/test state, virtual environment, or version-control metadata; internal files not expanded.

.venv-ids: installed dependencies, editor/test state, virtual environment, or version-control metadata; internal files not expanded.

.venv-model-test: installed dependencies, editor/test state, virtual environment, or version-control metadata; internal files not expanded.

bootstrap/cache: generated framework runtime storage, caches, sessions, views, and logs; contents not published.

commands.txt: private environment or local operational file; values and contents not published.

public/admin/assets/vendor: installed dependencies, editor/test state, virtual environment, or version-control metadata; internal files not expanded.

public/landing/assets/vendors/bootstrap/scss/vendor: installed dependencies, editor/test state, virtual environment, or version-control metadata; internal files not expanded.

setup/.env: private environment or local operational file; values and contents not published.

storage: generated framework runtime storage, caches, sessions, views, and logs; contents not published.

vendor: installed dependencies, editor/test state, virtual environment, or version-control metadata; internal files not expanded.

Documentation output files

public/about_system/IDMS_Complete_System_Documentation.pdf is the final searchable, text-only PDF. public/about_system/IDMS_Complete_System_Documentation.txt is the editable plain-text companion. public/about_system/documentation_manifest.json records source hashes, source coverage counts, exclusions, build metadata, and PDF validation facts. These artifacts are generated documentation, not additional website or detection services.

26. Traceability, exclusions, and documentation completion

This reference individually inventories 611 project files outside the explicitly grouped private, dependency, and generated directories. It registers current Python source members, custom PHP methods, every migration's schema declarations, source-referenced environment variable names, notebook experiments, historical API archive members, and text from all five Word manuscripts. It provides a comprehensive local-source reference while keeping runtime secrets and subscriber records out of a potentially public folder.

Source-relative paths identify evidence locations. Line numbers refer to the inspected snapshot and can change after editing. The manifest stores SHA-256 fingerprints of selected first-party text sources, without copying private values. The PDF is generated from the narrative in this script plus safe local-source extraction. This document's build process reads source and development documents only; it does not contact a router, analyze live traffic, call a hosted classifier, or alter production data.

The reader should distinguish four evidence levels: behavior explicitly implemented in current source; historical claims preserved from project research; defaults that may be overridden privately; and operational outcomes requiring a live verification. A deployment is complete only when its website, schema, Python services, inference model, environment settings, and router policy agree. This reference documents the repository and known workflow; it does not replace that deployment verification.